Security Incident Detection and Response Management System

Sprint I — Services 7, 8, 9

Joseph Awaida

18 October 2026

Contribution report: merge into the common team report before submission.

Repository: https://github.com/j123awaida/435L

Table of contents

Sprint I — Services 7, 8, 9 1

Table of contents 2

1. Title page (Joseph Awaida) 3

2. Table of contents (Joseph Awaida) 3

3. Introduction (Joseph Awaida) 3

4. System architecture (Joseph Awaida) 3

5. Implementation details (Joseph Awaida) 3

6. Database design (Joseph Awaida) 4

7. Error handling and validation (Joseph Awaida) 5

8. API documentation (Joseph Awaida) 5

9. Authentication and authorization (Joseph Awaida) 5

10. GitHub and version control (Joseph Awaida) 6

11. References and AI disclosure (Joseph Awaida) 6

12. Reproduction and handoff (Joseph Awaida) 6

13. Genuine screenshot evidence (Joseph Awaida) 6

Design diagrams (Joseph Awaida) 8

Actual execution evidence (Joseph Awaida) 10

Database structure observed locally (Joseph Awaida) 15

Before/after database records (Joseph Awaida) 18

API reference (Joseph Awaida) 21

Genuine Postman screenshots (Joseph Awaida) 25

Genuine database and GitHub screenshots (Joseph Awaida) 68

Submission status and remaining evidence (Joseph Awaida) 78

1. Title page (Joseph Awaida)

Security Incident Detection and Response Management System Backend. Software Tools Lab, Fall 2026–2027, Sprint I. Contribution: services 7, 8, and 9. Prepared 18 October 2026. GitHub: https://github.com/j123awaida/435L. This document is Joseph Awaida's contribution draft, to be merged into the team's common report with names, page numbers, real Postman screenshots, team history, and CREATES feedback.

2. Table of contents (Joseph Awaida)

The DOCX version contains a Word table-of-contents field. Update the field after merging the common report so page numbers reflect the final layout. Sections 1–11 here are Joseph Awaida's contribution. Other members must identify their own sections in the merged report.

3. Introduction (Joseph Awaida)

Organizations need consistent records of security weaknesses, notifications, and response procedures. These backend services associate discovered vulnerabilities with assets, deliver user alerts, and track ordered response actions. The implementation provides all eighteen required operations for this contribution, persistent relational storage, authentication, role checks, input validation, and runnable Postman scenarios. It does not implement the remaining six services or a frontend.

4. System architecture (Joseph Awaida)

Four independent FastAPI processes and a notification delivery worker run locally: a temporary identity adapter on port 8000, vulnerability management on 8007, notifications on 8008, and response playbooks on 8009. Postman and API clients communicate with each process over HTTP. A shared SQLite file supplies reference identities, assets, tokens, owned service tables, and local audit records. This Sprint I arrangement is appropriate for a local demonstration, but it does not provide independent service databases or the complete team's cross-service API integration.

The identity, asset, and audit interfaces are explicit team integration boundaries. Teammates must replace the temporary adapter and reference tables with agreed APIs for Services 1, 2, and 5. Upstream incident and threat services may create notifications by calling Service 8 with an authorized administrator session. Service 7 automatically queues a Vulnerability Alert for the affected asset owner. A worker delivers queued alerts to Service 8 over HTTP and retries failures; an idempotency receipt prevents duplicate delivery. Incident/threat event producers remain the responsibility of the corresponding teammates. See architecture.mmd for the diagram.

5. Implementation details (Joseph Awaida)

Service 7: vulnerability management

The service creates, lists, retrieves, updates, searches, and deletes vulnerability records. Each record stores title, description, CVE identifier, CVSS, severity, asset, discovery source, lifecycle status, creation time, and resolution time. Creation checks that the asset exists and atomically queues an alert for its owner. A worker delivers it through the Service 8 HTTP API after commit. Search combines exact CVE, severity, asset, and status filters with AND. Empty results return an empty array.

For example, POST /Joseph-Awaida/vulnerabilities accepts {"title":"Exposed remote service","description":"Review required","cve_identifier":"CVE-2026-12345","cvss_score":9.1,"severity":"Critical","asset_id":1,"discovery_source":"Lab scanner"}. A successful creation returns HTTP 201 and the stored object, including its generated id and timestamps. This CVE is synthetic input for exercising the application, not a claim about a published vulnerability.

Severity is stored as a constrained text enum. The caller selects severity independently of CVSS; no automatic severity calculation is claimed. The workflow enforces Open → Under Investigation → In Progress → Resolved → Closed. Moving directly from Open to Closed returns 409. The server sets resolution_date on entering Resolved and preserves it on closure, rather than accepting arbitrary client timestamps. This deliberately protects chronology.

Service 8: notification management

Notifications store recipient, title, message, type, severity, creation time, status, and read date. Administrators create alerts for existing users. Alert types cover incident assignments, security alerts, system events, threat alerts, and vulnerability alerts. Status starts Unread. Marking read sets a server timestamp only once, so repeated requests preserve the original read date.

For example, POST /Joseph-Awaida/notifications accepts {"user_id":3,"title":"Security alert","message":"Review suspicious activity","notification_type":"Security Alert","severity":"High"}. It returns HTTP 201 with the generated notification. GET /Joseph-Awaida/notifications/user/3 returns an array. A valid recipient with no notifications returns []. Analyst and employee access is limited to their own notifications; administrators and auditors can read across users. Only administrator, analyst, and employee roles may mark read, subject to ownership. Search accepts recipient, type, severity, and status. Analysts search only their own records; employees have no search permission.

Service 9: playbooks and response actions

Playbooks store name, description, incident category, severity, creator, and creation date. Their response actions are returned in ascending step order, together with execution history. An action cannot be inserted before an already started step. Actions store description, positive unique step number within a playbook, status, assignee, and completion time. Administrators create/delete playbooks; administrators and analysts add actions and advance action status; auditors only read.

For example, POST /Joseph-Awaida/playbooks accepts {"name":"Malware containment","description":"Isolate and investigate","incident_category":"Malware","severity":"High"}. POST /Joseph-Awaida/playbooks/{id}/actions accepts {"description":"Isolate asset","step_number":1,"assigned_user_id":2}. PUT /Joseph-Awaida/actions/{id}/status accepts {"status":"In Progress"}. Successful updates return the persisted action. Steps progress Pending → In Progress → Completed. A user must be assigned before execution. Earlier steps must be completed before a later action starts. Completion time is generated by the server.

Each action creation/status update creates an immutable history row, including actor, assignee, old/new state, and time. Deleting a playbook cascades to its current actions but preserves execution history with the original action/playbook identifiers. Those history identifiers intentionally lack cascading foreign keys so deleted objects remain traceable.

6. Database design (Joseph Awaida)

The schema is app/schema.sql. SQLite foreign keys are enabled on every connection. Every operational table uses an integer primary key; session tokens use a SHA-256 digest primary key. Users own assets and receive notifications. Assets have many vulnerabilities. Users create playbooks. Playbooks contain many actions. Users can be assigned many actions and generate many history/audit entries. See schema.mmd for the ER diagram.

users is a minimal Service 1 reference with unique username, full name, password hash, role, and account status. assets is a minimal Service 2 reference with name and owner. They are integration scaffolding, not the full user/asset schemas required of the team. sessions holds hashed tokens, users, and expiry. vulnerabilities, notifications, playbooks, actions, and action_history belong to this contribution. audit_logs is a local Service 5

integration boundary and records permission denials and login results. All nine complete team service schemas must be merged by the team.

CHECK constraints validate database enum values, score ranges, and step numbers. UNIQUE(playbook_id, step_number) prevents ambiguous action ordering. Indexes support common vulnerability and notification filters. Parameterized queries prevent SQL injection; table names and update keys originate in trusted code/schema definitions. Transactions protect lifecycle updates from concurrent writes. History rows cannot be updated/deleted through database triggers. Audit rows cannot be updated. No audit modification API is exposed. Successful protected create/update/delete requests are automatically recorded by middleware. An outbox stores alert payloads and delivery status; receipt records store payload digests and original responses for idempotent HTTP delivery. No tokens or credentials are stored in outbox rows.

7. Error handling and validation (Joseph Awaida)

Requests reject unknown fields, empty or whitespace-only text, invalid CVE structure, scores outside 0–10, invalid severities/types/statuses, nonpositive identifiers, duplicate action steps, invalid references, and workflow skips. CVE identifiers must match CVE-YYYY-NNNN or longer numeric suffixes. This is syntactic validation, not a lookup in a CVE registry.

Errors consistently use {"status":"error","message":"..."}. Input errors also contain a sanitized list of field locations and messages. HTTP 401 identifies missing, invalid, expired authentication or inactive accounts; 403 identifies role or ownership denial; 404 indicates a nonexistent entity; 409 indicates constraint/workflow conflicts; 422 indicates malformed/invalid input. Internal SQL details are not returned in constraint errors. Permission violations persist in local audit logs with actor, method, path, related service, and warning severity.

Automated tests exercise all four roles, credentials, inactive accounts, ownership, malformed inputs, lifecycle ordering, action ordering, deletions, and history retention. Actual local execution records, database structure, and before/after data are saved by scripts/capture_evidence.py; evidence/SUMMARY.md records their outcome after execution. Genuine Postman desktop screenshots, database structure/record screenshots, and GitHub commit-history screenshots are embedded in the report appendices with captions identifying Joseph Awaida. The separate JSON records remain supplemental execution evidence.

8. API documentation (Joseph Awaida)

The full API list and request fields are in API.md; generated OpenAPI JSON documents include exact schemas. Interactive documentation is available at each process's /docs. All assignment endpoints include /Joseph-Awaida before the resource path. The Postman collection contains descriptions and assertions for all required operations plus success, failure, authorization, search, and workflow scenarios. Requests save tokens and created identifiers into local environment variables. The environment contains empty password/token fields so no credentials are published.

For reproduction, Postman users enter locally generated development passwords and run folders in order. The submitted screenshots were captured from an actual successful desktop run. Screenshots should show method, named route, request body as applicable, status, response body, and a caption such as “Joseph Awaida — Advance vulnerability to Under Investigation: demonstrates the permitted next lifecycle step.” Hide credentials and bearer tokens.

9. Authentication and authorization (Joseph Awaida)

The temporary identity adapter verifies salted PBKDF2-HMAC-SHA256 password hashes using 600,000 iterations and constant-time digest comparisons. Successful login returns a cryptographically random bearer token with a one-hour expiry. Only token digests are stored. Every service verifies token expiry, current account status, and role against the database on each request. Suspending a user immediately prevents use of an existing session.

admin corresponds to Security Administrator; analyst to Security Analyst; employee to Regular Employee/User; auditor to Auditor. Administrators have all eighteen operations. Analysts can read/update vulnerabilities, read their own notifications, mark their own notifications read, search their own notifications, read playbooks, and add/update response actions. Employees can only read/mark their own notifications. Auditors can read/search vulnerabilities and notifications and read playbooks; they cannot mutate records. Analysts cannot create/delete vulnerabilities, create/delete notifications, or create/delete playbooks. Forbidden role and ownership attempts are audited.

The development identity adapter is not a public registration service and does not allow callers to choose their own roles. The seed command provisions demo identities locally. Full identity registration/account management remains the responsibility of Service 1. Production deployment, distributed token verification, HTTPS termination, identity service contracts, and remote audit forwarding are outside this local Sprint I contribution.

10. GitHub and version control (Joseph Awaida)

Target repository: https://github.com/j123awaida/435L. The source and tests were successfully pushed to the repository on 18 October 2026. Commit history records implementation, tests, and documentation work. No backdated commits or teammate contributions are fabricated. Each teammate must push their own work, and the merged common report must include actual history/contribution screenshots. GitHub upload status must be checked against the actual push result; naming a target repository here is not proof of upload.

11. References and AI disclosure (Joseph Awaida)

Primary assignment: Final Project Fall 2026-2027-Sprint 1 (1).docx supplied by the student. Implementation documentation: https://fastapi.tiangolo.com/ ; https://docs.python.org/3/library/sqlite3.html ; https://docs.python.org/3/library/hashlib.html ; https://docs.pytest.org/ ; https://learning.postman.com/docs/collections/collections-overview/ .

OpenAI Codex assisted in interpreting requirements, implementing code, creating tests, preparing API/Postman artifacts, and drafting this report. Joseph Awaida must review and understand the implementation, reproduce its behavior, capture the required evidence, and accurately disclose assistance in the final team submission. The automated API evidence is produced by the local running implementation; none of it is claimed to be manually executed Postman evidence.

12. Reproduction and handoff (Joseph Awaida)

The README provides setup/run commands. scripts.notification_worker retries automatic vulnerability alerts; scripts.verify_integration checks real HTTP delivery and duplicate protection. scripts.validation_summary creates a public summary from actual JUnit/Newman artifacts. scripts.pdf_report creates a paginated PDF with a generated table of contents, diagrams, actual database structure, and before/after records. scripts.package_submission packages committed public files and excludes local credentials/runtime state.

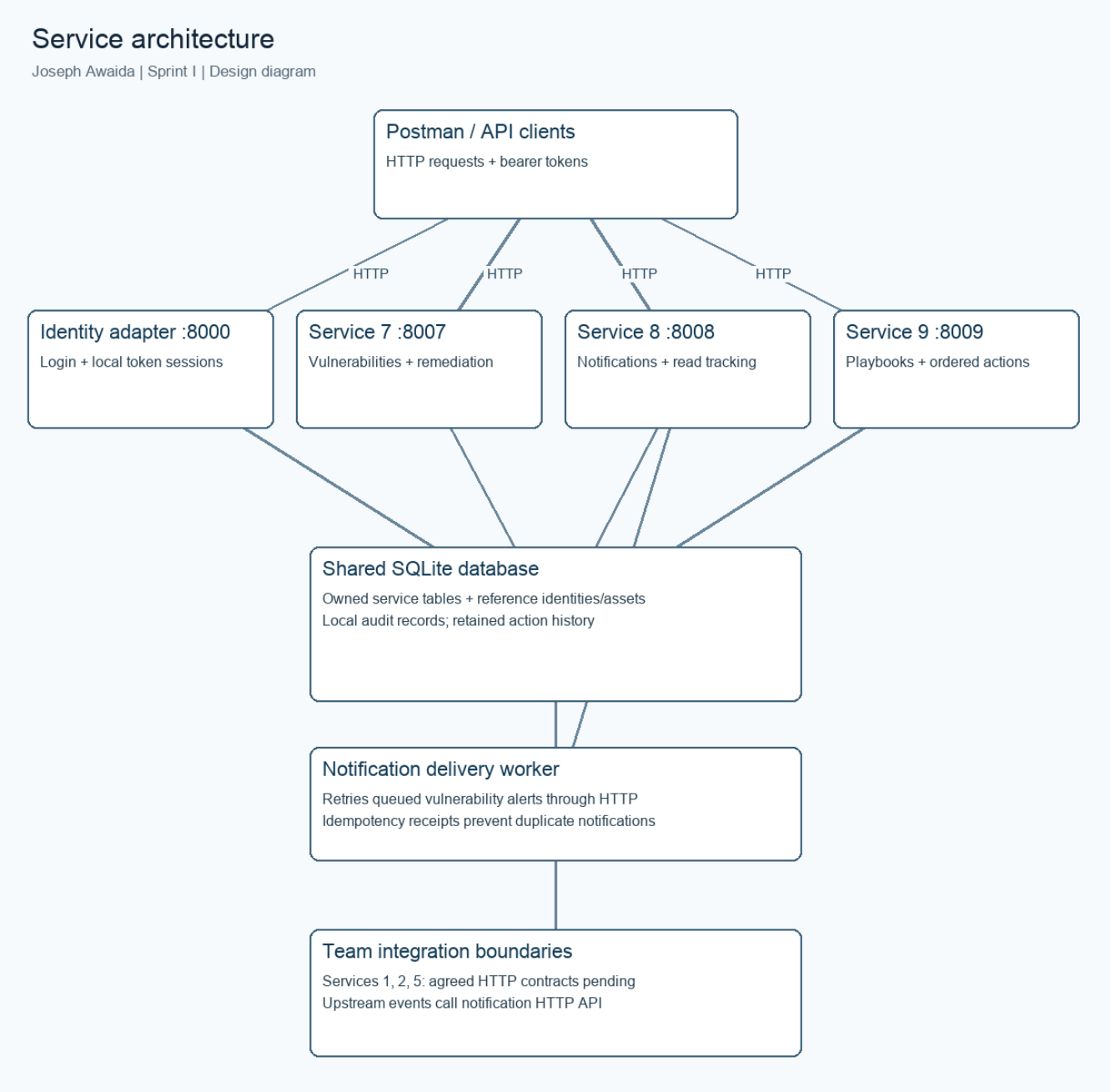
See TEAM-INTEGRATION.md for the upstream notification contract and remaining teammate interfaces, and DEMONSTRATION.md for a practical walkthrough. After access was enabled, the actual Postman desktop client executed the collection successfully and its result/request views were captured through its Electron browser interface. SQLite-Web in read-only mode and headless Chrome supplied actual database and GitHub screenshots. Images are unedited; authentication token bodies and private password request bodies are excluded from public captures. The JSON records and generated design diagrams remain distinct from screenshot evidence.

13. Genuine screenshot evidence (Joseph Awaida)

The screenshot appendices contain real Postman runner requests/results for the 57 scenarios and actual database/GitHub browser captures. The desktop run passed all 84 assertions with zero errors. Every image has a caption naming Joseph Awaida and explaining the operation. Successful login captures show status and response headers instead of publishing bearer tokens. Password-containing resolved request bodies are omitted, while ordinary request bodies and responses are shown. The Postman collection retains the complete request definitions with password variables. Screenshot source URLs, capture times and captions are recorded in evidence/screenshots/postman-manifest.json and browser-manifest.json.

Design diagrams (Joseph Awaida)

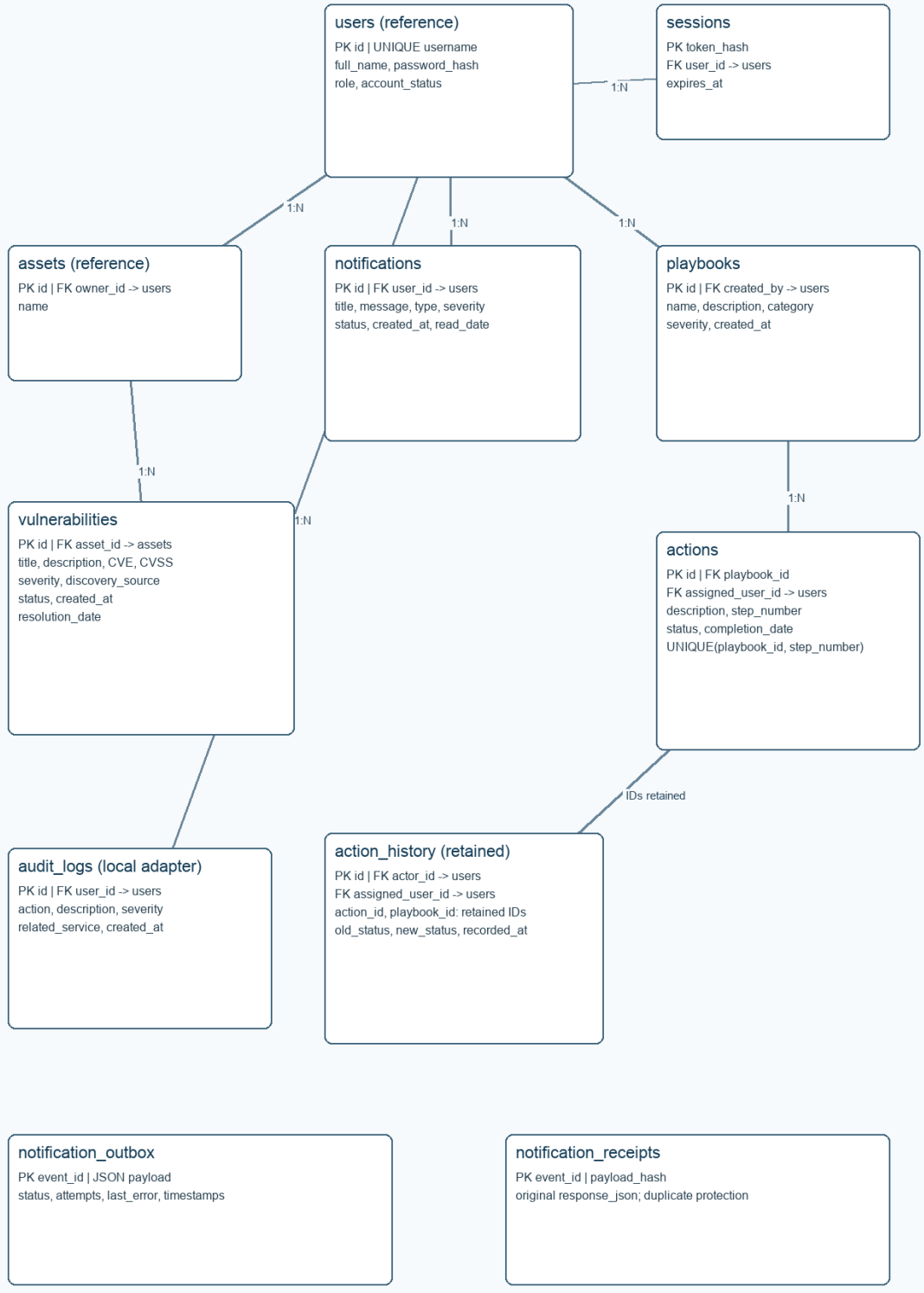
Architecture



Schema

Relational schema and integration references

Joseph Awaida | Sprint I | Design diagram



Actual execution evidence (Joseph Awaida)

Captured from the local running APIs at 2026-10-18T09:55:24.815382+00:00. This appendix contains execution records, not Postman desktop screenshots.

Pytest: 12 tests, 0 failures, 0 errors. Postman Newman: 57 requests and 84 assertions; 0 failed assertions.

Operation	Expected HTTP	Actual HTTP
Login admin	200	200
Login analyst	200	200
Login employee	200	200
Login auditor	200	200
Reject incorrect password	401	401
Create critical vulnerability	201	201
Create second vulnerability	201	201
Retrieve vulnerabilities	200	200
Retrieve vulnerability	200	200
Missing vulnerability	404	404
Search severity=Critical	200	200
Search asset_id=2	200	200
Search cve_identifier=CVE-2026-12345	200	200
Search status=Open	200	200
Reject CVSS above ten	422	422
Reject missing title	422	422
Reject employee access	403	403
Reject missing token	401	401
Reject lifecycle skip	409	409
Advance vulnerability to Under Investigation	200	200
Advance vulnerability to In Progress	200	200
Advance vulnerability to Resolved	200	200
Advance vulnerability to Closed	200	200
Reject analyst deletion	403	403
Delete vulnerability	200	200
Create employee alert	201	201
Create analyst alert	201	201
Retrieve own notifications	200	200
Retrieve empty notifications	200	200

Operation	Expected HTTP	Actual HTTP
Retrieve specific notification	200	200
Reject another user notification	403	403
Missing notification	404	404
Search user_id=3	200	200
Search notification_type=Security%20Alert	200	200
Search severity=High	200	200
Search status=Unread	200	200
Reject auditor marking read	403	403
Mark own notification read	200	200
Repeat mark read	200	200
Reject employee deletion	403	403
Delete notification	200	200
Create playbook	201	201
Create second playbook	201	201
List playbooks	200	200
Retrieve playbook	200	200
Missing playbook	404	404
Add response step 1	201	201
Add response step 2	201	201
Reject duplicate step	409	409
Reject out of order action	409	409
Step 1 In Progress	200	200
Step 1 Completed	200	200
Step 2 In Progress	200	200
Step 2 Completed	200	200
Retrieve completed workflow	200	200
Reject analyst deleting playbook	403	403
Delete playbook	200	200

Joseph Awaida — Create critical vulnerability

POST http://127.0.0.1:8007/Joseph-Awaida/vulnerabilities

Actual request (credentials omitted):

```
{
  "title": "Exposed remote service",
  "description": "Scanner detected an exposed service requiring review.",
  "cve_identifier": "CVE-2026-12345",
  "cvss_score": 9.1,
  "severity": "Critical",
```

```
  "asset_id": 1,
  "discovery_source": "Lab scanner"
}
```

Actual response: HTTP 201

```
{
  "title": "Exposed remote service",
  "description": "Scanner detected an exposed service requiring review.",
  "cve_identifier": "CVE-2026-12345",
  "cvss_score": 9.1,
  "severity": "Critical",
  "asset_id": 1,
  "discovery_source": "Lab scanner",
  "id": 13,
  "status": "Open",
  "created_at": "2026-10-18T09:55:24.670665+00:00",
  "resolution_date": null
}
```

Joseph Awaida — Advance vulnerability to Resolved

PUT http://127.0.0.1:8007/Joseph-Awaida/vulnerabilities/13

Actual request (credentials omitted):

```
{
  "status": "Resolved"
}
```

Actual response: HTTP 200

```
{
  "title": "Exposed remote service",
  "description": "Scanner detected an exposed service requiring review.",
  "cve_identifier": "CVE-2026-12345",
  "cvss_score": 9.1,
  "severity": "Critical",
  "asset_id": 1,
  "discovery_source": "Lab scanner",
  "id": 13,
  "status": "Resolved",
  "created_at": "2026-10-18T09:55:24.670665+00:00",
  "resolution_date": "2026-10-18T09:55:24.709124+00:00"
}
```

Joseph Awaida — Create employee alert

POST http://127.0.0.1:8008/Joseph-Awaida/notifications

Actual request (credentials omitted):

```
{
  "user_id": 3,
  "title": "Security alert",
  "message": "Review the reported suspicious message.",
  "notification_type": "Security Alert",
  "severity": "High"
}
```

Actual response: HTTP 201

```
{
  "user_id": 3,
  "title": "Security alert",
  "message": "Review the reported suspicious message.",
  "notification_type": "Security Alert",
  "severity": "High",
  "id": 19,
  "status": "Unread",
  "created_at": "2026-10-18T09:55:24.724712+00:00",
  "read_date": null
}
```

Joseph Awaida — Mark own notification read

PUT http://127.0.0.1:8008/Joseph-Awaida/notifications/19/read

Actual request (credentials omitted):

```
null
```

Actual response: HTTP 200


```
{
  "user_id": 3,
  "title": "Security alert",
  "message": "Review the reported suspicious message.",
  "notification_type": "Security Alert",
  "severity": "High",
  "id": 19,
  "status": "Read",
  "created_at": "2026-10-18T09:55:24.724712+00:00",
  "read_date": "2026-10-18T09:55:24.753966+00:00"
}
```


Joseph Awaida — Retrieve completed workflow

GET http://127.0.0.1:8009/Joseph-Awaida/playbooks/11

Actual request (credentials omitted):

```
null
```

Actual response: HTTP 200


```
{
  "name": "Malware containment",
  "description": "Isolate and investigate a suspected malware infection.",
  "incident_category": "Malware",
  "severity": "High",
  "id": 11,
  "created_by": 1,
  "created_at": "2026-10-18T09:55:24.768582+00:00",
  "actions": [
    {
      "description": "Isolate asset",
      "step_number": 1,
      "assigned_user_id": 2,
      "id": 1,
      "playbook_id": 11,
      "status": "Completed",
      "completion_date": "2026-10-18T09:55:24.797926+00:00"
    },
    {
      "description": "Investigate artifacts",
      "step_number": 2,
      "assigned_user_id": 2,
      "id": 2,
      "playbook_id": 11,
      "status": "Completed",
      "completion_date": "2026-10-18T09:55:24.805144+00:00"
    }
  ],
  "execution_history": [
    {
      "id": 31,
      "action_id": 1,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": null,
      "new_status": "Pending",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.782169+00:00"
    },
    {
      "id": 32,
      "action_id": 2,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": null,
      "new_status": "Pending",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.785758+00:00"
    },
    {
      "id": 33,
      "action_id": 1,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": "Pending",
      "new_status": "In Progress",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.794424+00:00"
```

```
    },
    {
      "id": 34,
      "action_id": 1,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": "In Progress",
      "new_status": "Completed",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.798280+00:00"
    },
    {
      "id": 35,
      "action_id": 2,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": "Pending",
      "new_status": "In Progress",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.801797+00:00"
    },
    {
      "id": 36,
      "action_id": 2,
      "playbook_id": 11,
      "actor_id": 2,
      "old_status": "In Progress",
      "new_status": "Completed",
      "assigned_user_id": 2,
      "recorded_at": "2026-10-18T09:55:24.805316+00:00"
    }
  ]
}
```


Joseph Awaida — Reject another user notification

GET http://127.0.0.1:8008/Joseph-Awaida/notifications/19

Actual request (credentials omitted):


```
null
```


Actual response: HTTP 403


```
{
  "status": "error",
  "message": "Access to another user\u2019s notifications is forbidden"
}
```

Database structure observed locally (Joseph Awaida)

action_history

Field	Type	Primary key	Required
id	INTEGER	True	False
action_id	INTEGER	False	True
playbook_id	INTEGER	False	True
actor_id	INTEGER	False	True
old_status	TEXT	False	False
new_status	TEXT	False	True
assigned_user_id	INTEGER	False	False
recorded_at	TEXT	False	True

actions

Field	Type	Primary key	Required
id	INTEGER	True	False
playbook_id	INTEGER	False	True
description	TEXT	False	True
step_number	INTEGER	False	True
status	TEXT	False	True
assigned_user_id	INTEGER	False	False
completion_date	TEXT	False	False

assets

Field	Type	Primary key	Required
id	INTEGER	True	False
name	TEXT	False	True
owner_id	INTEGER	False	True

audit_logs

Field	Type	Primary key	Required
id	INTEGER	True	False
user_id	INTEGER	False	False
action	TEXT	False	True
description	TEXT	False	True

Field	Type	Primary key	Required
severity	TEXT	False	True
related_service	TEXT	False	True
created_at	TEXT	False	True

notification_outbox

Field	Type	Primary key	Required
event_id	TEXT	True	False
payload_json	TEXT	False	True
status	TEXT	False	True
attempts	INTEGER	False	True
last_error	TEXT	False	False
created_at	TEXT	False	True
delivered_at	TEXT	False	False

notifications

Field	Type	Primary key	Required
id	INTEGER	True	False
user_id	INTEGER	False	True
title	TEXT	False	True
message	TEXT	False	True
notification_type	TEXT	False	True
severity	TEXT	False	True
status	TEXT	False	True
created_at	TEXT	False	True
read_date	TEXT	False	False

playbooks

Field	Type	Primary key	Required
id	INTEGER	True	False
name	TEXT	False	True
description	TEXT	False	True
incident_category	TEXT	False	True
severity	TEXT	False	True
created_by	INTEGER	False	True

Field	Type	Primary key	Required
created_at	TEXT	False	True

users

Field	Type	Primary key	Required
id	INTEGER	True	False
full_name	TEXT	False	True
username	TEXT	False	True
password_hash	TEXT	False	True
role	TEXT	False	True
account_status	TEXT	False	True

vulnerabilities

Field	Type	Primary key	Required
id	INTEGER	True	False
title	TEXT	False	True
description	TEXT	False	True
cve_identifier	TEXT	False	True
cvss_score	REAL	False	True
severity	TEXT	False	True
asset_id	INTEGER	False	True
discovery_source	TEXT	False	True
status	TEXT	False	True
created_at	TEXT	False	True
resolution_date	TEXT	False	False

Before/after database records (Joseph Awaida)

These records were queried from the actual SQLite database immediately before and after each operation. Concurrent alert delivery may add other notifications. They supplement the required database screenshots.

Advance vulnerability to Resolved

Before — vulnerabilities

id	status	resolution_date
2	Open	None
4	Open	None
6	Open	None
7	Open	None
9	Open	None
10	Open	None
12	Open	None
13	In Progress	None
14	Open	None

After — vulnerabilities

id	status	resolution_date
2	Open	None
4	Open	None
6	Open	None
7	Open	None
9	Open	None
10	Open	None
12	Open	None
13	Resolved	2026-10-18T09:55:24.709124+00:00
14	Open	None

Mark own notification read

Before — notifications

id	status	read_date
2	Unread	None
4	Unread	None
6	Unread	None
7	Unread	None
8	Unread	None

id	status	read_date
9	Unread	None
11	Unread	None
12	Unread	None
13	Unread	None
14	Unread	None
16	Unread	None
17	Unread	None
18	Unread	None
19	Unread	None
20	Unread	None

After — notifications

id	status	read_date
2	Unread	None
4	Unread	None
6	Unread	None
7	Unread	None
8	Unread	None
9	Unread	None
11	Unread	None
12	Unread	None
13	Unread	None
14	Unread	None
16	Unread	None
17	Unread	None
18	Unread	None
19	Read	2026-10-18T09:55:24.753966+00:00
20	Unread	None

Step 1 Completed

Before — actions

id	status	completion_date
1	In Progress	None
2	Pending	None

After — actions

id	status	completion_date
1	Completed	2026-10-18T09:55:24.797926+00:00
2	Pending	None

API reference (Joseph Awaida)

All protected requests require a bearer token. Full request/response schemas are in the exported OpenAPI JSON and interactive `/docs`. Error format: `{"status":"error","message":"..."}`. Common errors: 401 missing/invalid authentication; 403 role/ownership; 404 missing resource; 409 conflicting step or workflow; 422 input validation.

Identity integration adapter (Service 1 boundary)

Health

`GET /Joseph-Awaida/health`

Success: 200. See Postman requests for runnable examples.

Login

`POST /Joseph-Awaida/auth/login`

JSON body: `username, password`. Required: `username, password`.

Success: 200. See Postman requests for runnable examples.

Service 7: Vulnerability Management

Health

`GET /Joseph-Awaida/health`

Success: 200. See Postman requests for runnable examples.

Create vulnerability

`POST /Joseph-Awaida/vulnerabilities`

JSON body: `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source`. Required: `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source`.

Administrator creates a vulnerability against an existing asset; queues an HTTP-delivered alert for the asset owner.

Success: 201. See Postman requests for runnable examples.

Response: VulnerabilityRecord objects with fields `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source, id, status, created_at, resolution_date`.

List vulnerabilities

`GET /Joseph-Awaida/vulnerabilities`

Parameters: limit (query, optional), offset (query, optional)

Administrators, analysts and auditors retrieve records, with limit/offset pagination.

Success: 200. See Postman requests for runnable examples.

Response: array of VulnerabilityRecord objects with fields `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source, id, status, created_at, resolution_date`.

Search vulnerabilities

`GET /Joseph-Awaida/vulnerabilities/search`

Parameters: cve_identifier (query, optional), severity (query, optional), asset_id (query, optional), status (query, optional)

Exact CVE, severity, asset and lifecycle status filters combine with AND. No matches return an empty array.

Success: 200. See Postman requests for runnable examples.

Response: array of VulnerabilityRecord objects with fields `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source, id, status, created_at, resolution_date`.

Retrieve vulnerability

`GET /Joseph-Awaida/vulnerabilities/{id}`

Parameters: id (path, required)

Returns the stored vulnerability or 404. Employees are forbidden.

Success: 200. See Postman requests for runnable examples.

Response: VulnerabilityRecord objects with fields `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source, id, status, created_at, resolution_date`.

Update vulnerability

`PUT /Joseph-Awaida/vulnerabilities/{id}`

Parameters: id (path, required)

JSON body: `description, severity, status, cvss_score`. Required: ``.

Administrator or analyst edits supported fields. Lifecycle advances one step; resolution time is generated by the server.

Success: 200. See Postman requests for runnable examples.

Response: VulnerabilityRecord objects with fields `title, description, cve_identifier, cvss_score, severity, asset_id, discovery_source, id, status, created_at, resolution_date`.

Delete vulnerability

`DELETE /Joseph-Awaida/vulnerabilities/{id}`

Parameters: id (path, required)

Administrator only; deletion of a missing record returns 404.

Success: 200. See Postman requests for runnable examples.

Response: Message objects with fields `message`.

Service 8: Notification and Alert Management

Health

`GET /Joseph-Awaida/health`

Success: 200. See Postman requests for runnable examples.

Create notification

`POST /Joseph-Awaida/notifications`

Parameters: idempotency-key (header, optional)

JSON body: `user_id, title, message, notification_type, severity`. Required: `user_id, title, message, notification_type, severity`.

Administrator or authenticated administrator service session creates a user alert. Optional Idempotency-Key prevents duplicate delivery; payload mismatch returns 409.

Success: 201. See Postman requests for runnable examples.

Response: NotificationRecord objects with fields `user_id, title, message, notification_type, severity, id, status, created_at, read_date`.

Retrieve user notifications

`GET /Joseph-Awaida/notifications/user/{user_id}`

Parameters: user_id (path, required)

Analysts and employees may access only their own notifications. Administrators and auditors may read across users. Existing recipients with no alerts return an empty array.

Success: 200. See Postman requests for runnable examples.

Response: array of NotificationRecord objects with fields `user_id, title, message, notification_type, severity, id, status, created_at, read_date`.

Search notifications

`GET /Joseph-Awaida/notifications/search`

Parameters: user_id (query, optional), notification_type (query, optional), severity (query, optional), status (query, optional)

Filter by user, type, severity and read status. Analysts are restricted to their own records; employees cannot search.

Success: 200. See Postman requests for runnable examples.

Response: array of NotificationRecord objects with fields `user_id, title, message, notification_type, severity, id, status, created_at, read_date`.

Retrieve notification

`GET /Joseph-Awaida/notifications/{id}`

Parameters: id (path, required)

Returns a notification after role and ownership checks, or 404 for an invalid identifier.

Success: 200. See Postman requests for runnable examples.

Response: NotificationRecord objects with fields `user_id, title, message, notification_type, severity, id, status, created_at, read_date`.

Delete notification

`DELETE /Joseph-Awaida/notifications/{id}`

Parameters: id (path, required)

Administrator only. Idempotency receipts remain so retries do not recreate deleted alerts.

Success: 200. See Postman requests for runnable examples.

Response: Message objects with fields `message`.

Mark notification read

`PUT /Joseph-Awaida/notifications/{id}/read`

Parameters: id (path, required)

Administrator or owning analyst/employee marks read. Repeated calls preserve the first read timestamp. Auditors cannot mutate.

Success: 200. See Postman requests for runnable examples.

Response: NotificationRecord objects with fields `user_id, title, message, notification_type, severity, id, status, created_at, read_date`.

Service 9: Security Playbook and Response Automation

Health

`GET /Joseph-Awaida/health`

Success: 200. See Postman requests for runnable examples.

List response playbooks

`GET /Joseph-Awaida/playbooks`

Administrators, analysts and auditors list available playbooks.

Success: 200. See Postman requests for runnable examples.

Response: array of PlaybookRecord objects with fields `name, description, incident_category, severity, id, created_by, created_at`.

Create response playbook

`POST /Joseph-Awaida/playbooks`

JSON body: `name, description, incident_category, severity`. Required: `name, description, incident_category, severity`.

Administrator creates a playbook organized by incident category and severity.

Success: 201. See Postman requests for runnable examples.

Response: PlaybookRecord objects with fields `name, description, incident_category, severity, id, created_by, created_at`.

Retrieve response playbook

`GET /Joseph-Awaida/playbooks/{id}`

Parameters: id (path, required)

Returns playbook metadata, ordered actions and append-only execution history.

Success: 200. See Postman requests for runnable examples.

Response: PlaybookDetail objects with fields `name, description, incident_category, severity, id, created_by, created_at, actions, execution_history`.

Delete response playbook

`DELETE /Joseph-Awaida/playbooks/{id}`

Parameters: id (path, required)

Administrator deletes playbook and current actions; immutable execution history survives.

Success: 200. See Postman requests for runnable examples.

Response: Message objects with fields `message`.

Add response action

`POST /Joseph-Awaida/playbooks/{id}/actions`

Parameters: id (path, required)

JSON body: `description, step_number, assigned_user_id`. Required: `description, step_number`.

Administrator or analyst adds a unique positive step; new steps cannot precede already started actions.

Success: 201. See Postman requests for runnable examples.

Response: ActionRecord objects with fields `description, step_number, assigned_user_id, id, playbook_id, status, completion_date`.

Update response action status

`PUT /Joseph-Awaida/actions/{id}/status`

Parameters: id (path, required)

JSON body: `status, assigned_user_id`. Required: `status`.

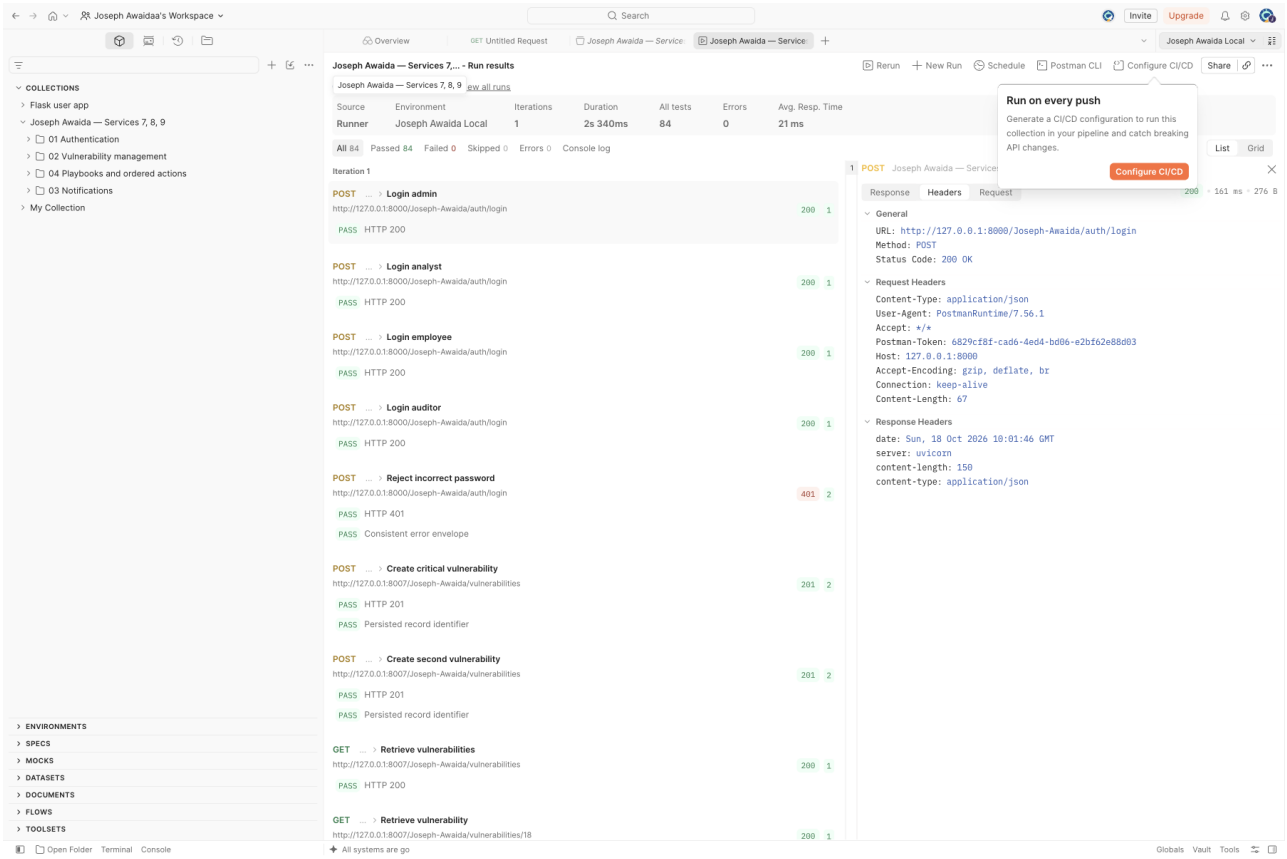
Administrator or analyst advances Pending to In Progress to Completed. Requires an active qualified assignee and completed earlier steps. Completion time is server-generated.

Success: 200. See Postman requests for runnable examples.

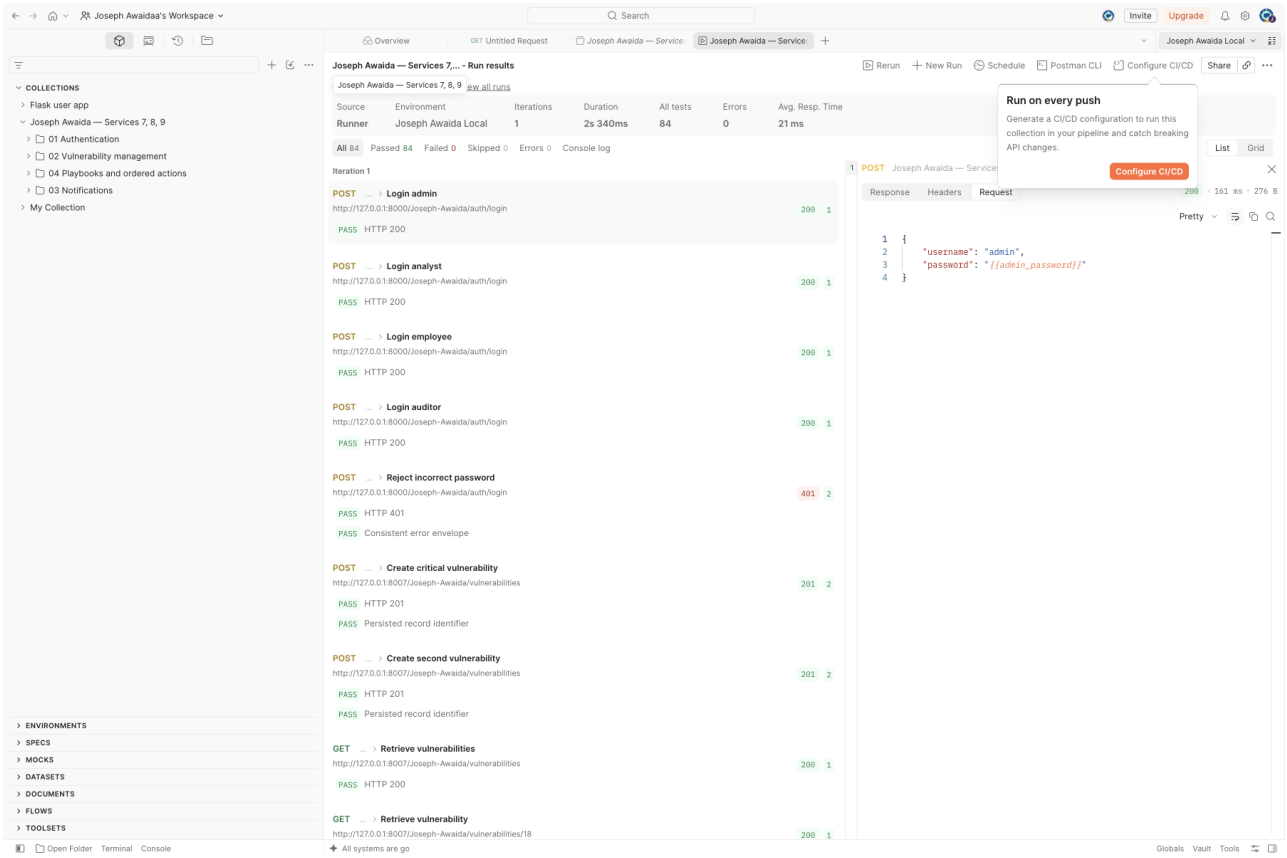
Response: ActionRecord objects with fields `description, step_number, assigned_user_id, id, playbook_id, status, completion_date`.

Genuine Postman screenshots (Joseph Awaida)

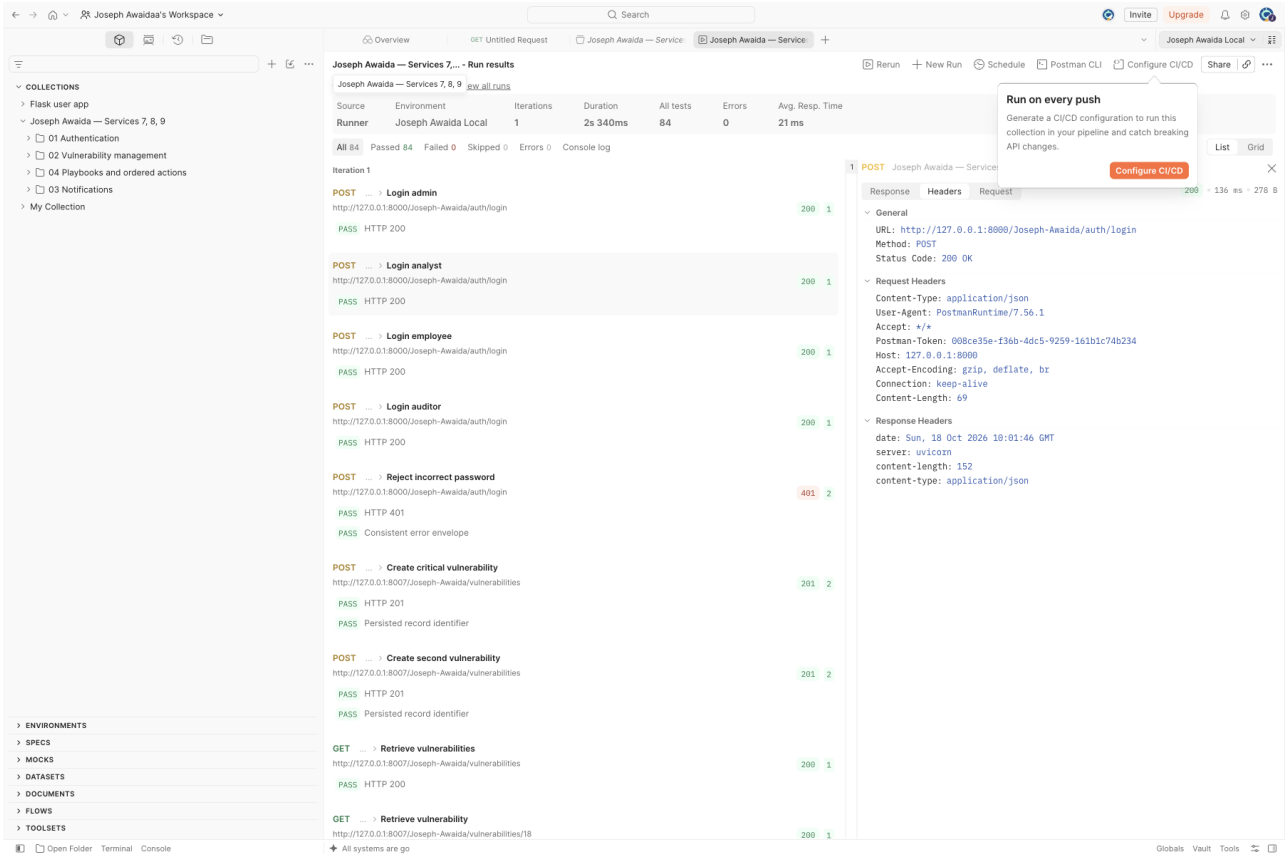
Unedited captures from actual running applications. Captions identify the student, operation and purpose. Token-bearing authentication responses use the Headers view; private password bodies are omitted.



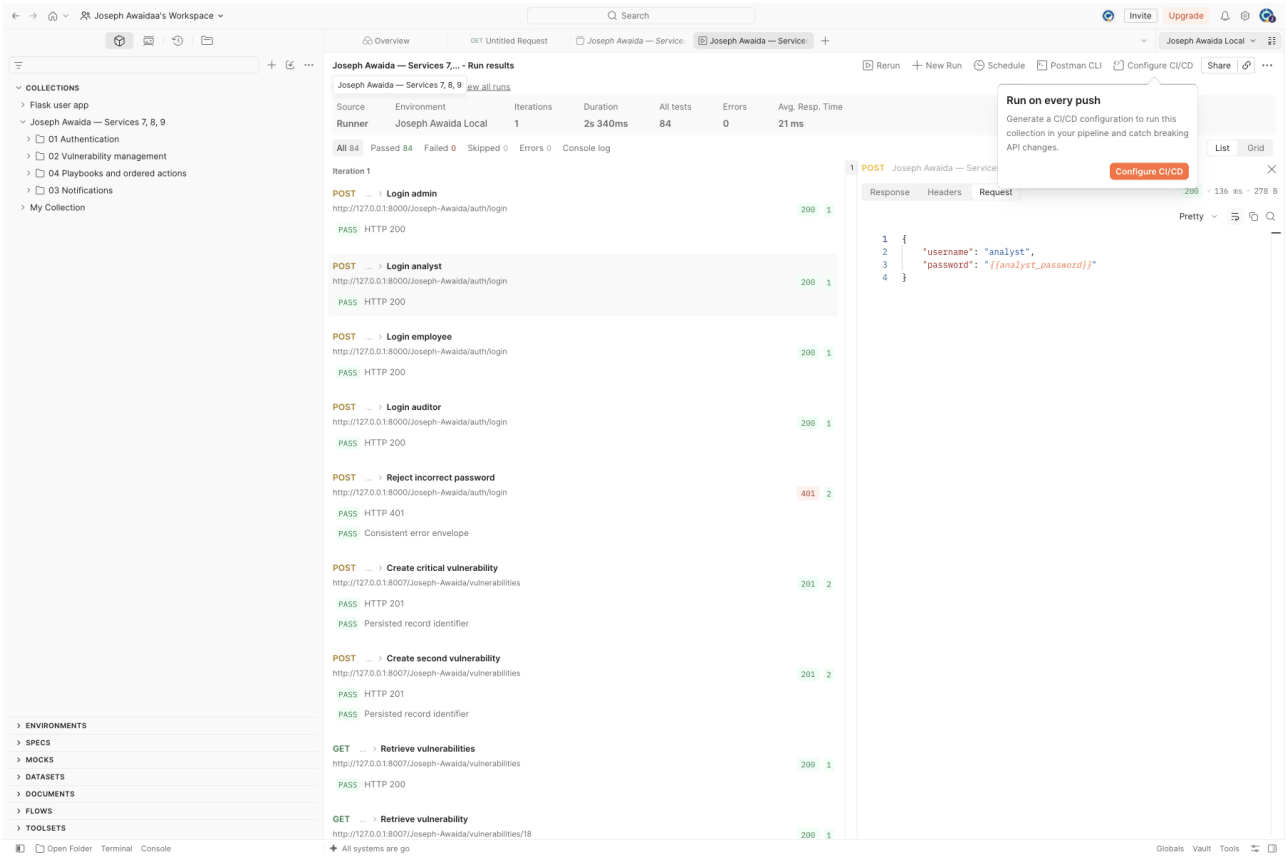
Joseph Awaida — Login admin: actual POST result and HTTP status (response body omitted to protect the authentication token).



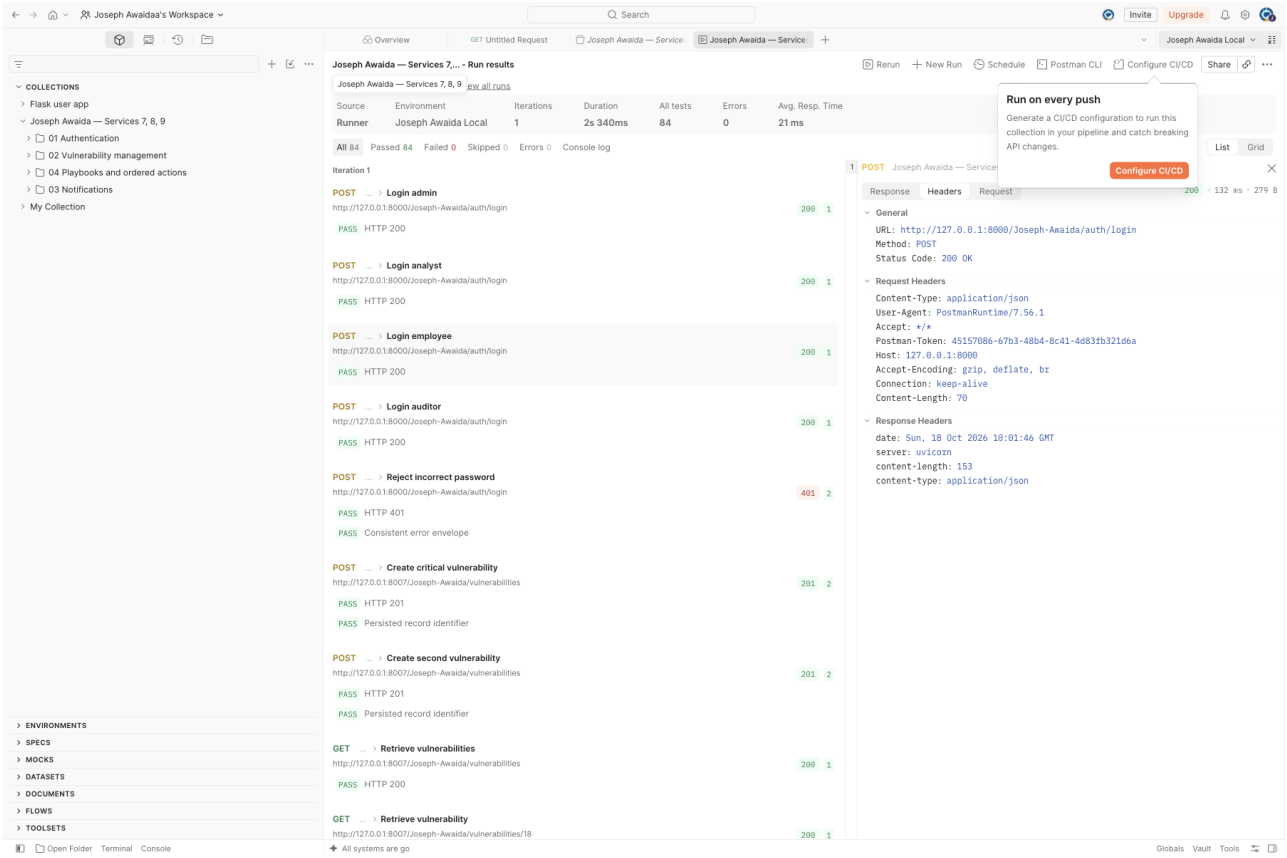
Joseph Awaida — Login admin: actual request body shown in the Postman runner.



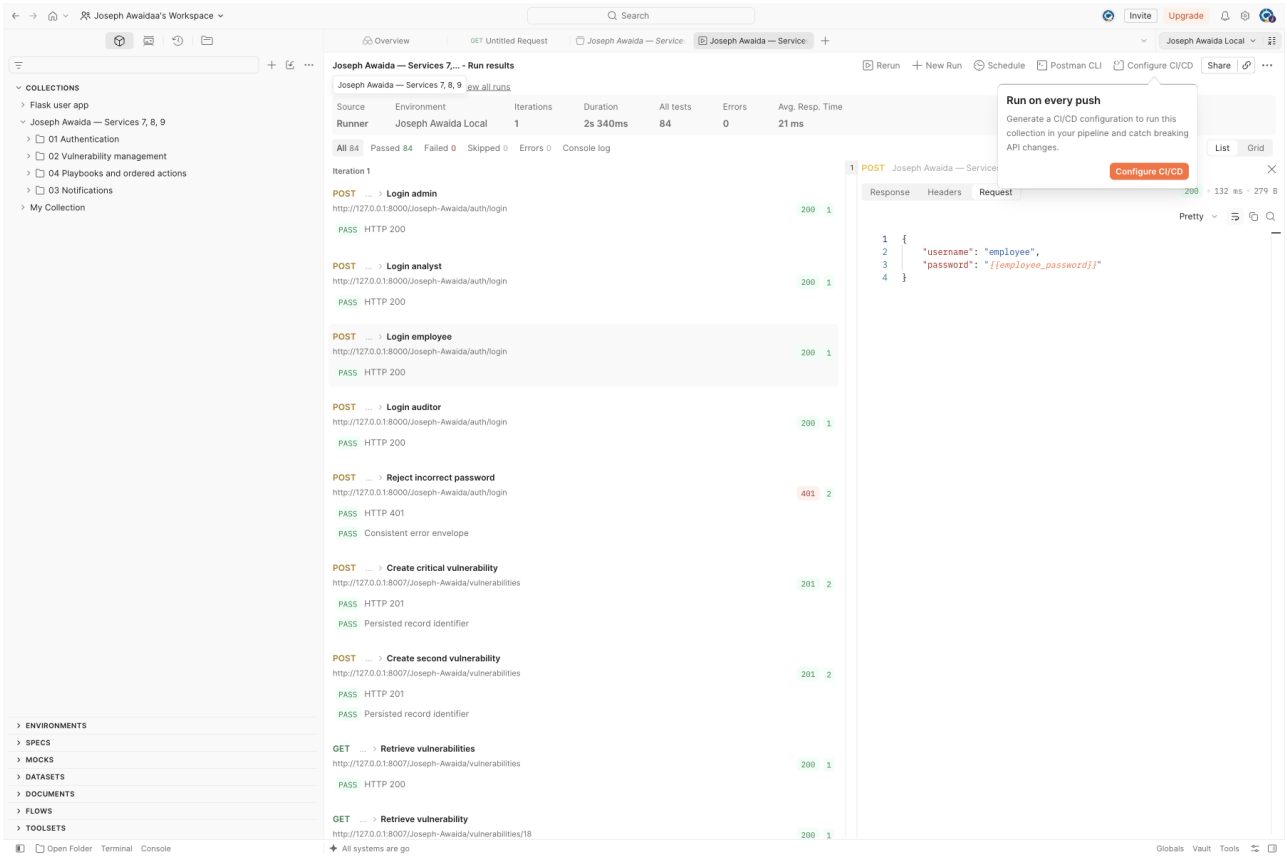
Joseph Awaida — Login analyst: actual POST result and HTTP status (response body omitted to protect the authentication token).



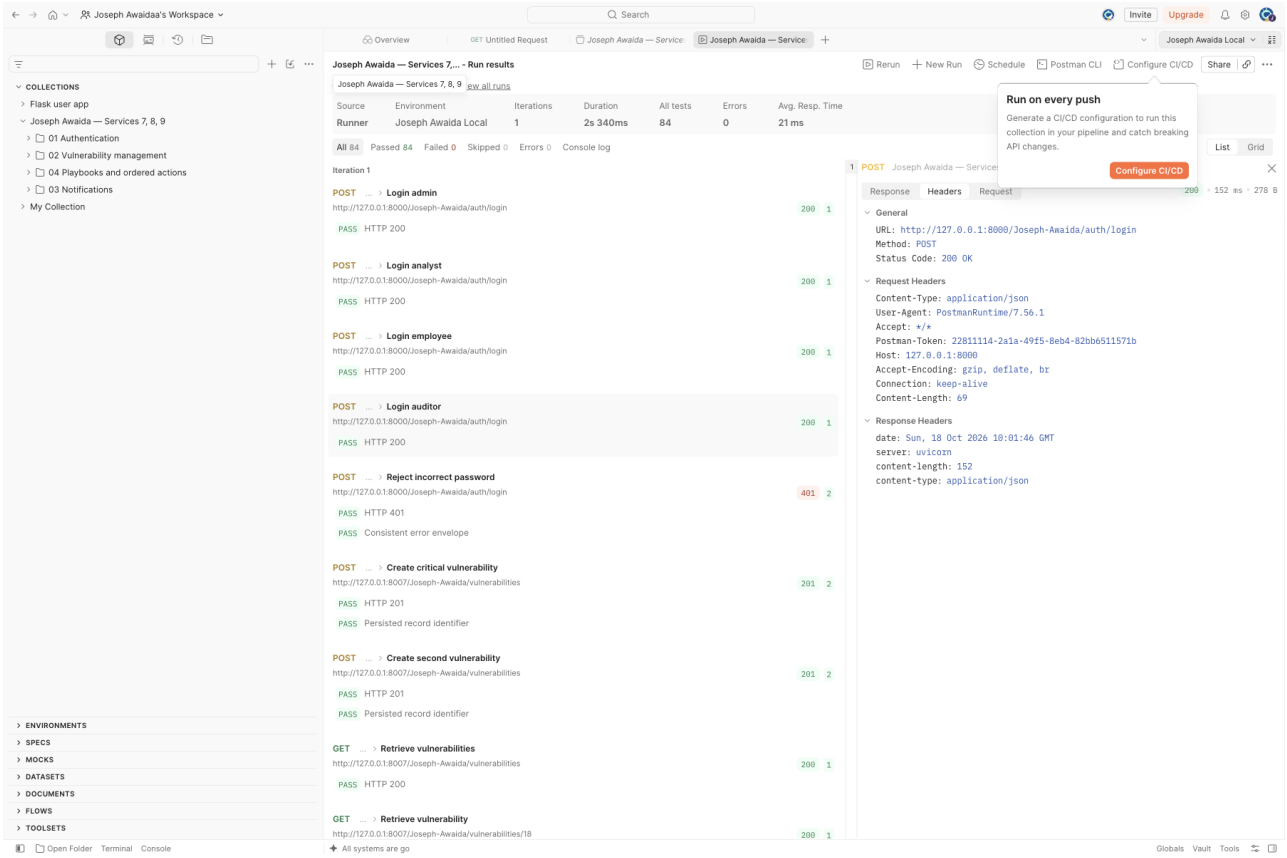
Joseph Awaida — Login analyst: actual request body shown in the Postman runner.



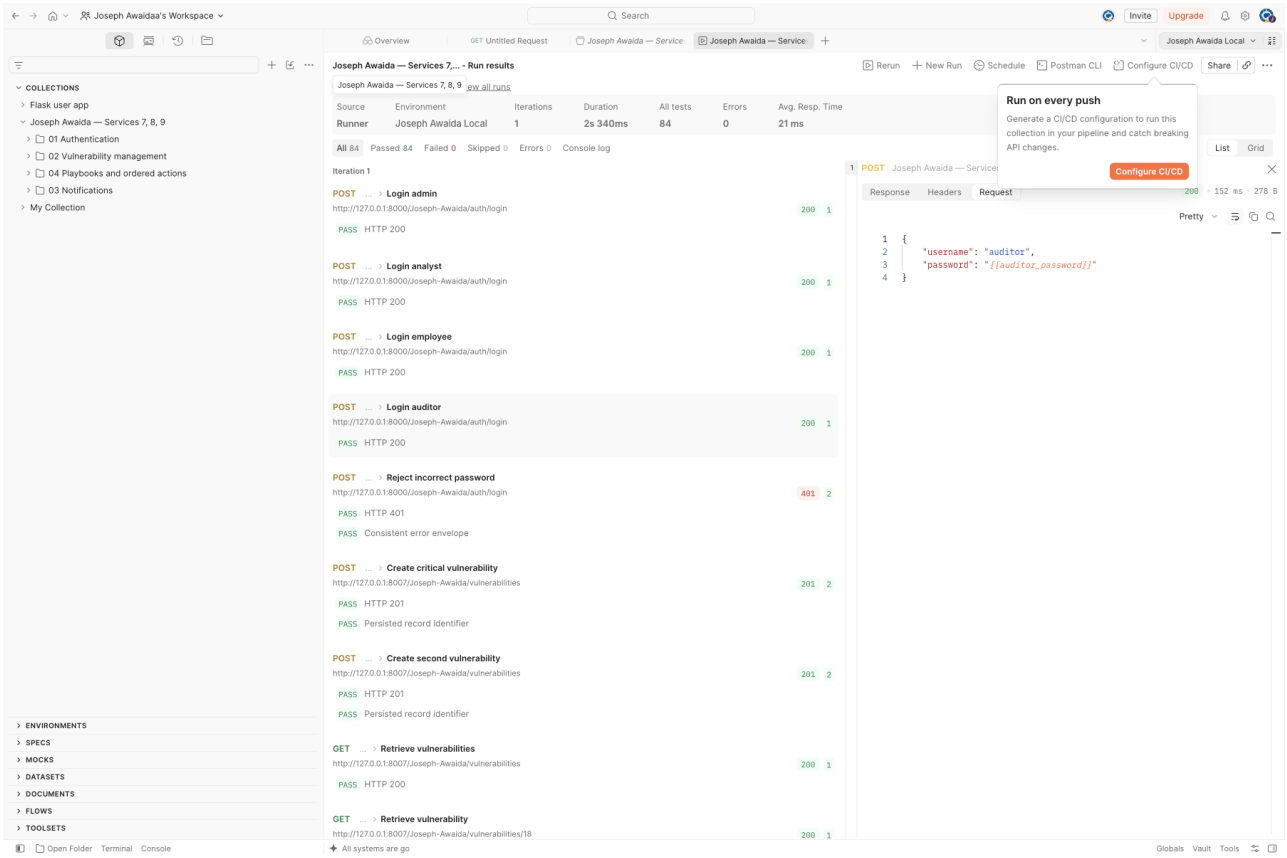
Joseph Awaida — Login employee: actual POST result and HTTP status (response body omitted to protect the authentication token).



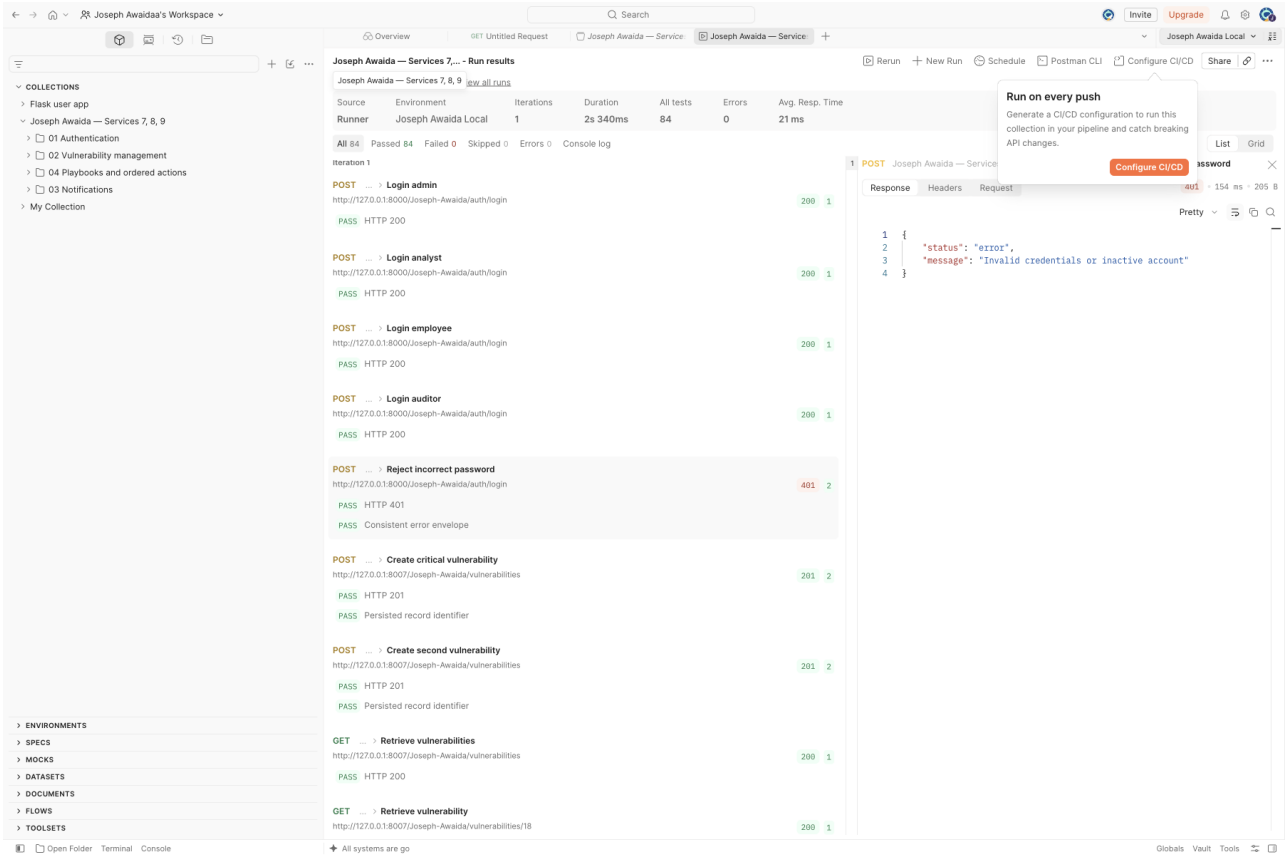
Joseph Awaida — Login employee: actual request body shown in the Postman runner.



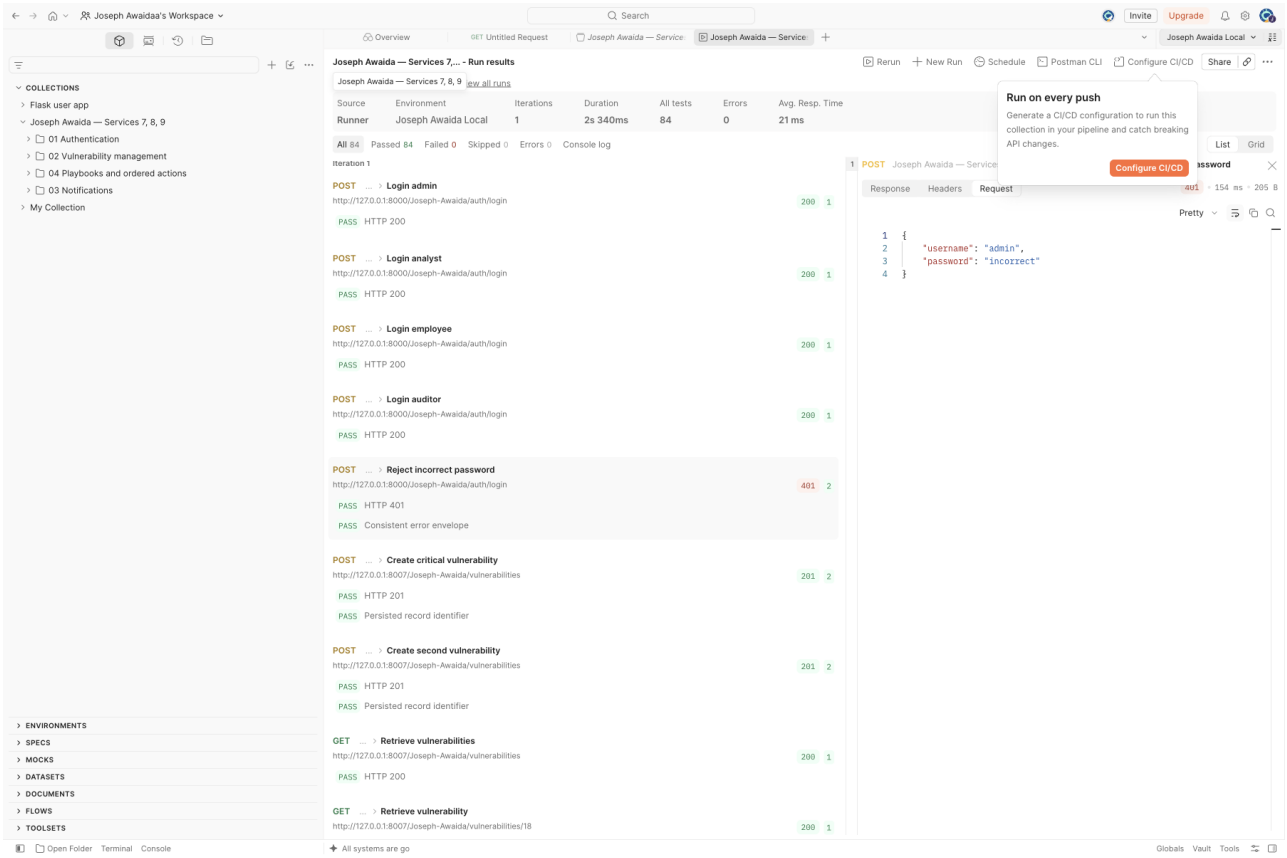
Joseph Awaida — Login auditor: actual POST result and HTTP status (response body omitted to protect the authentication token).



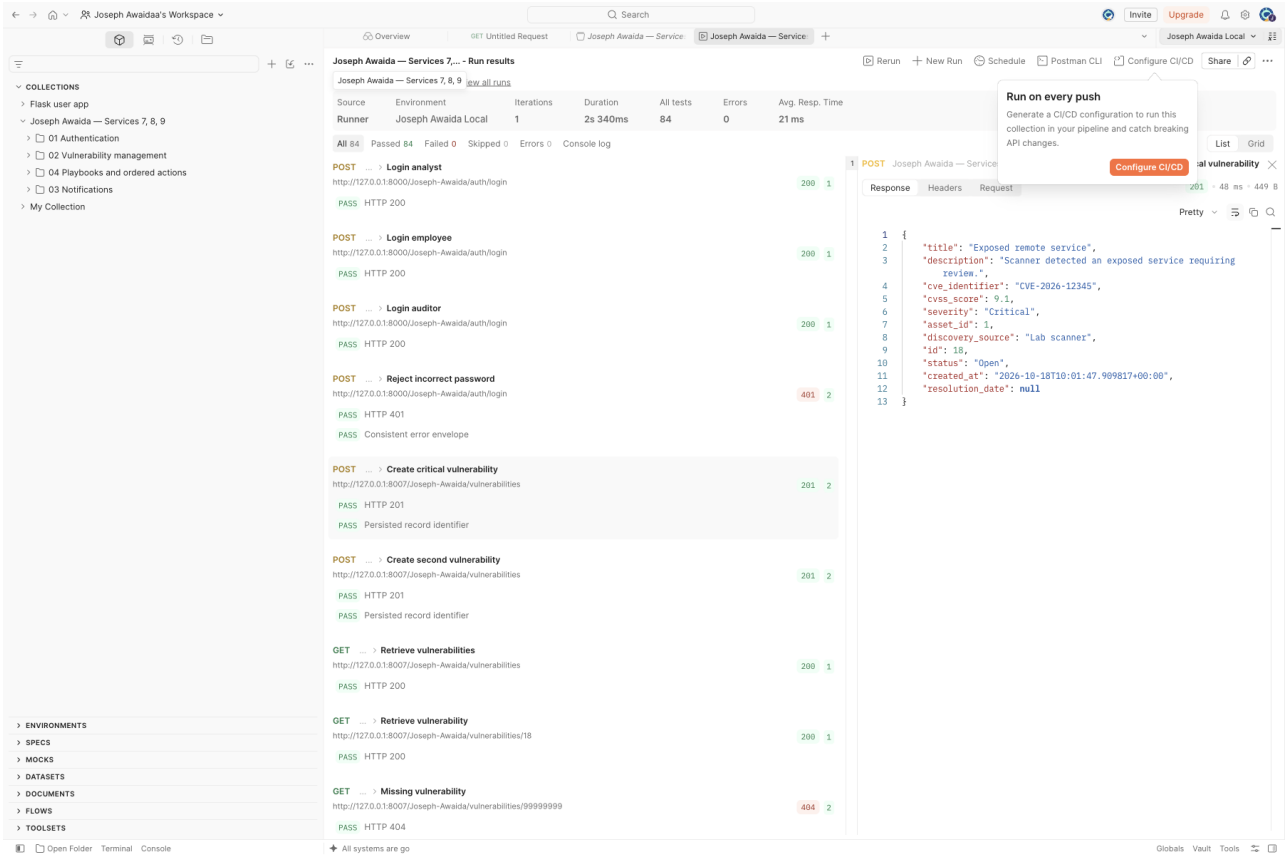
Joseph Awaida — Login auditor: actual request body shown in the Postman runner.



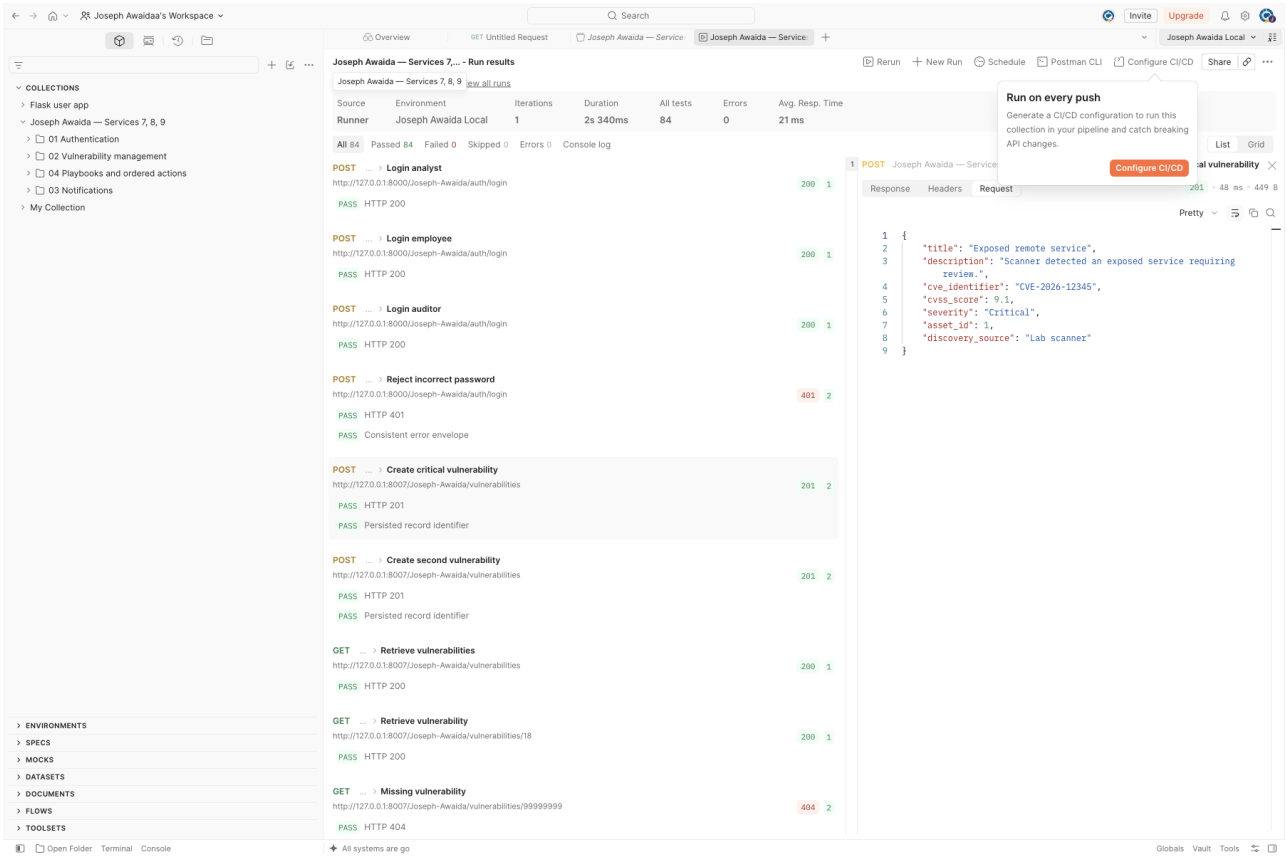
Joseph Awaida — Reject incorrect password: actual POST result and HTTP status.



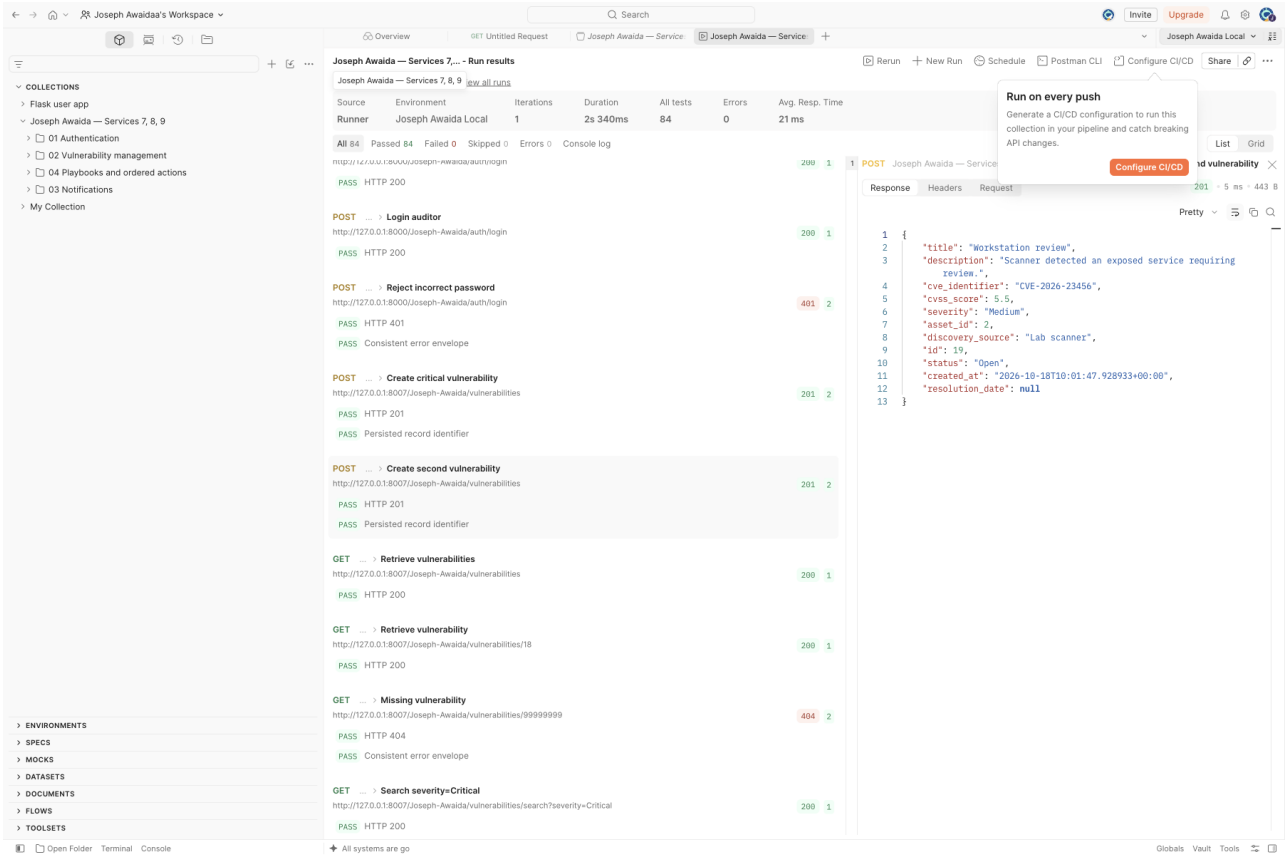
Joseph Awaida — Reject incorrect password: actual request body shown in the Postman runner.



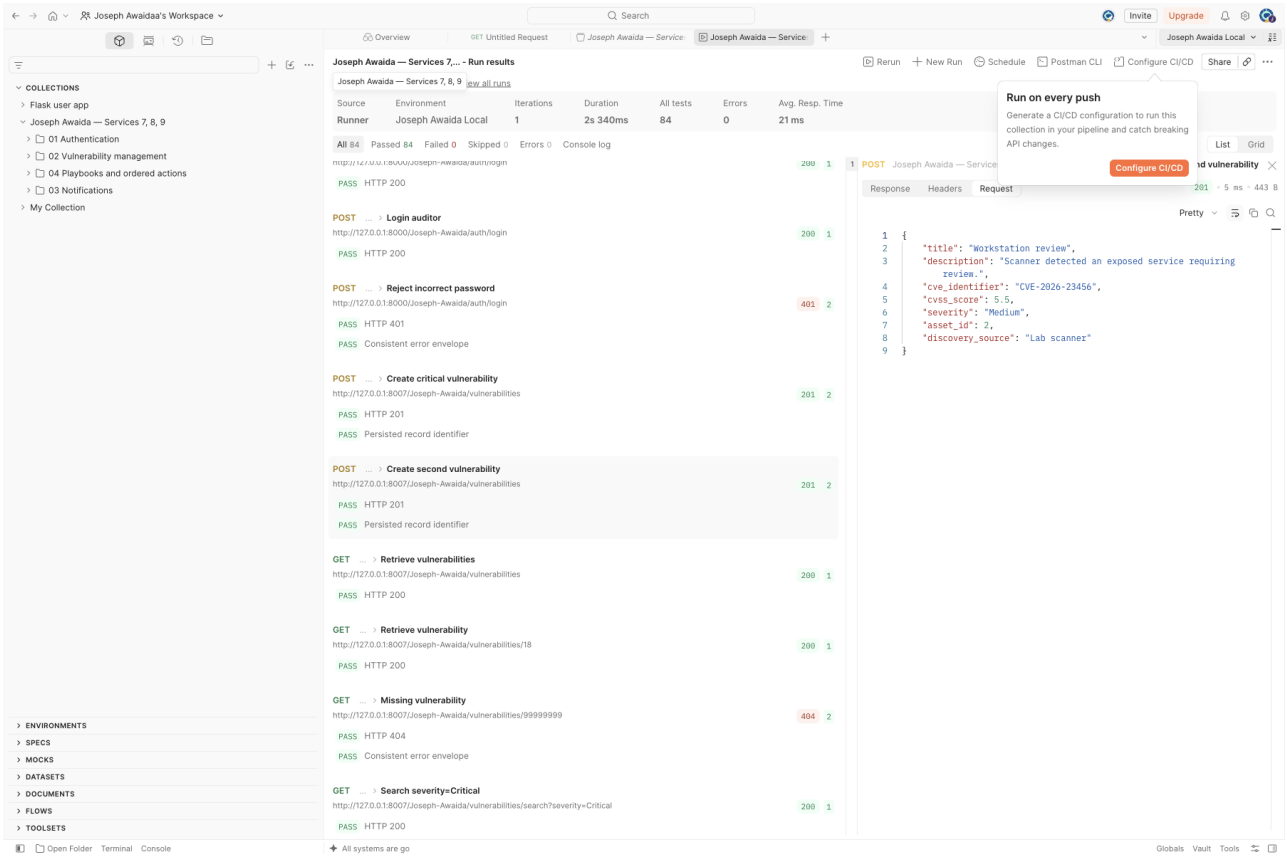
Joseph Awaida — Create critical vulnerability: actual POST result and HTTP status.



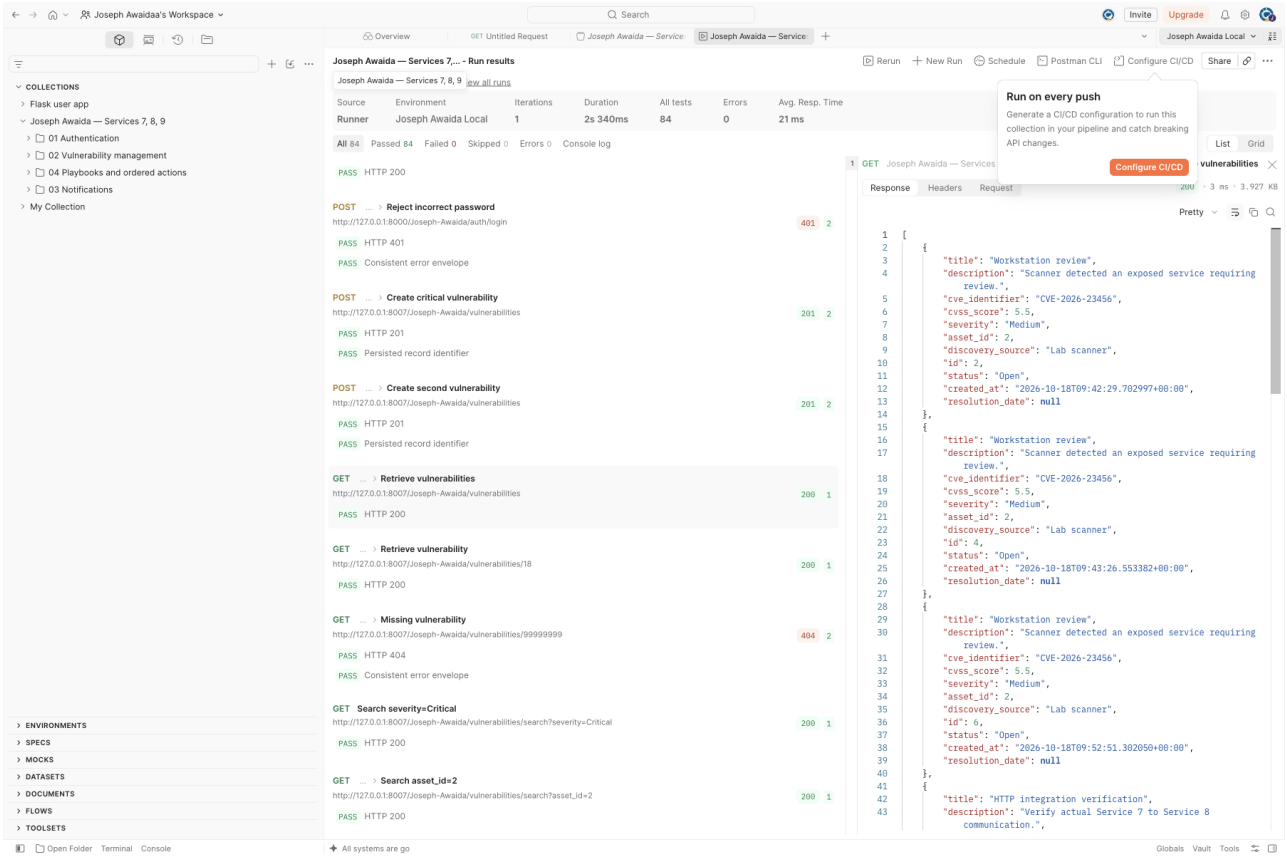
Joseph Awaida — Create critical vulnerability: actual request body shown in the Postman runner.



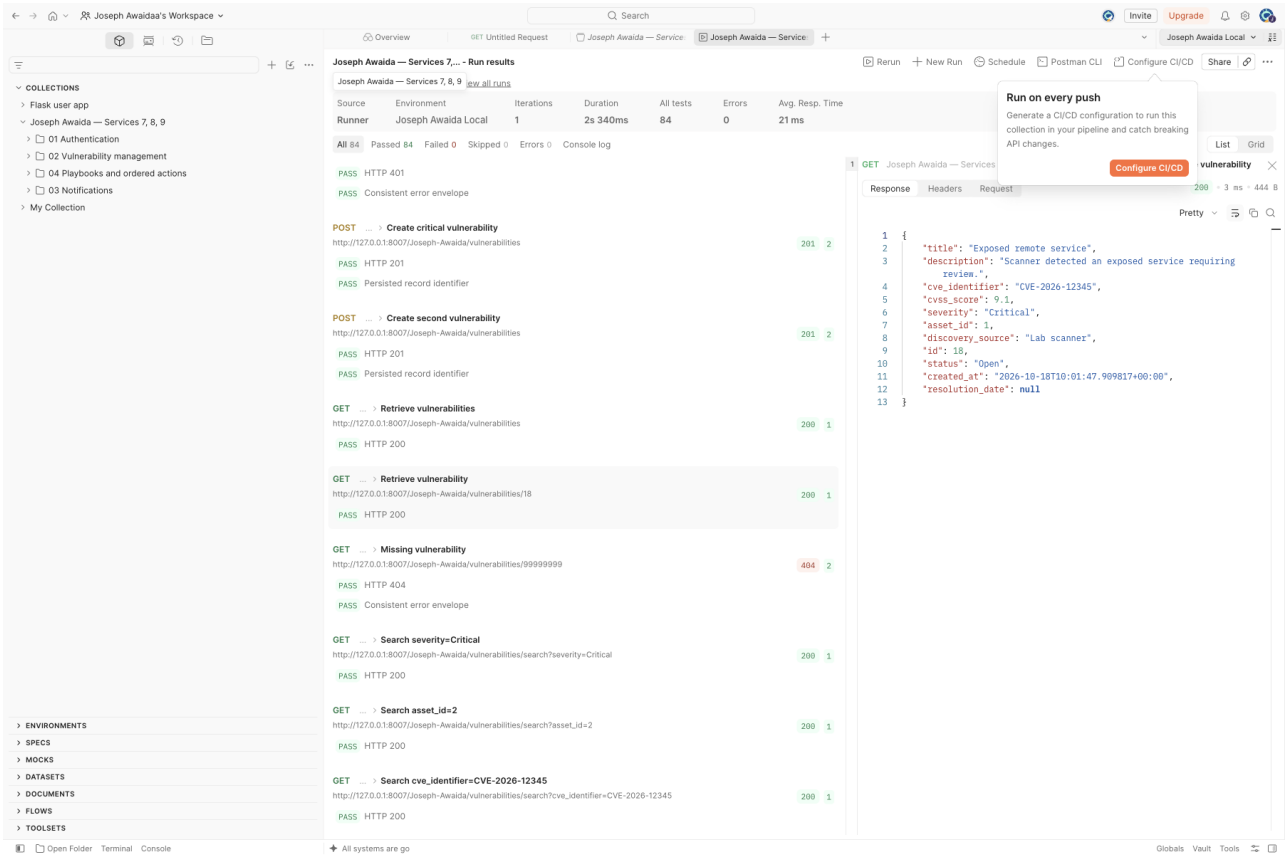
Joseph Awaida — Create second vulnerability: actual POST result and HTTP status.



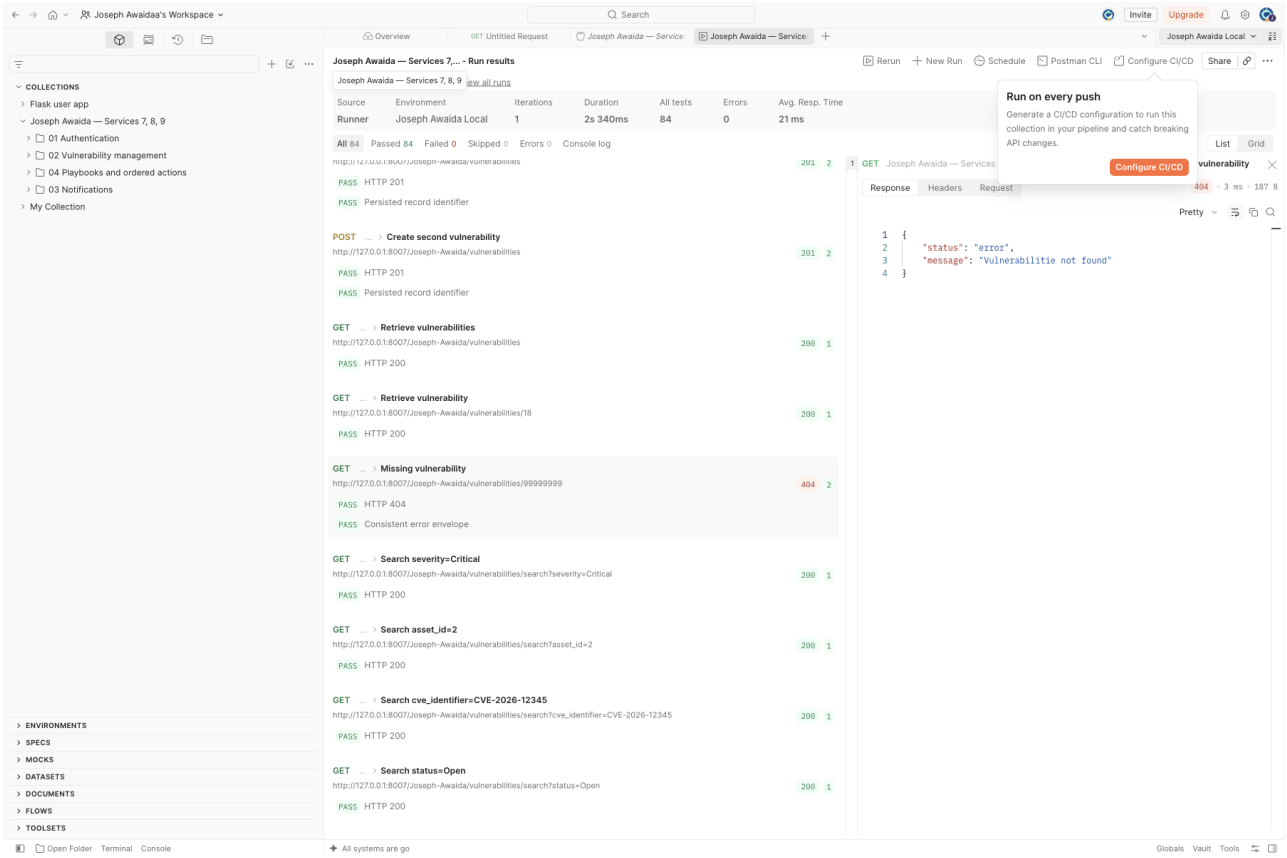
Joseph Awaida — Create second vulnerability: actual request body shown in the Postman runner.



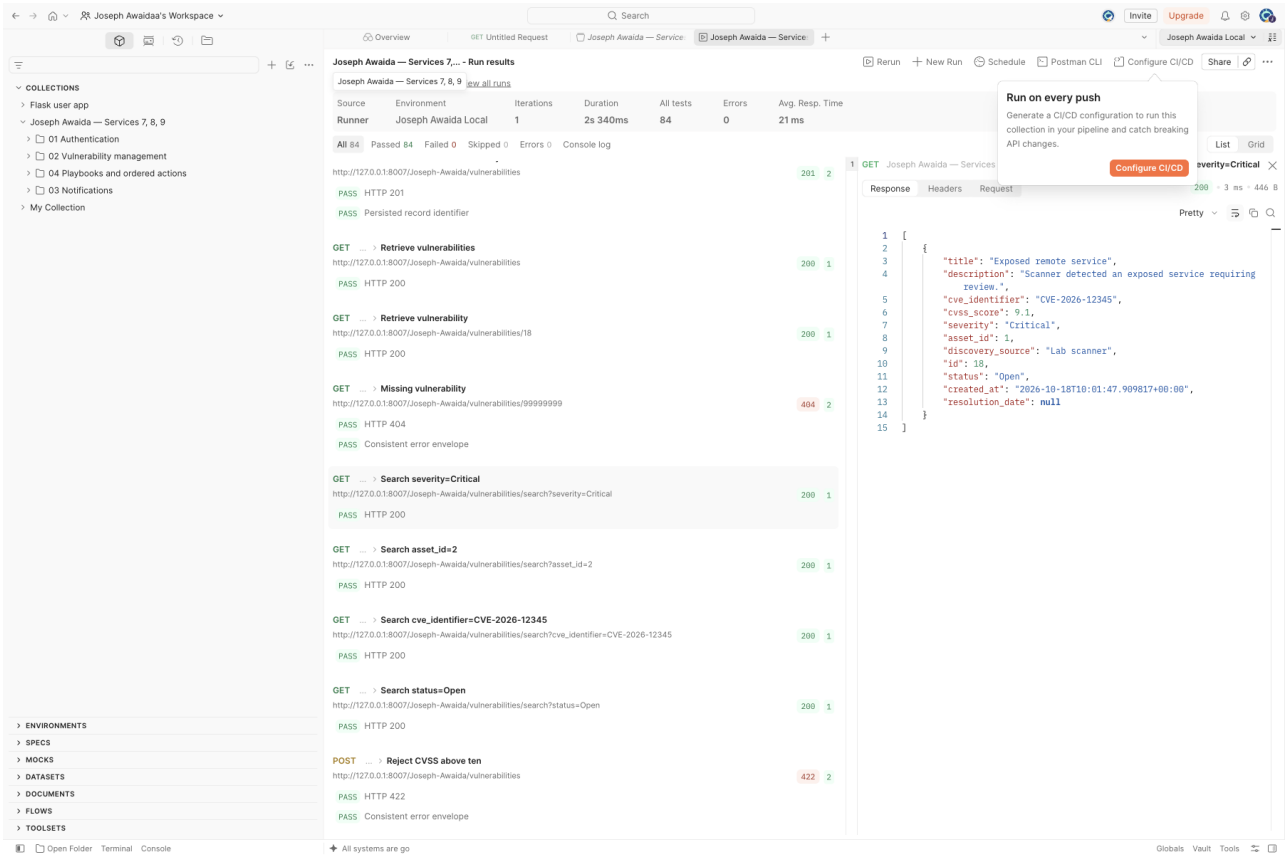
Joseph Awaida — Retrieve vulnerabilities: actual GET result and HTTP status.

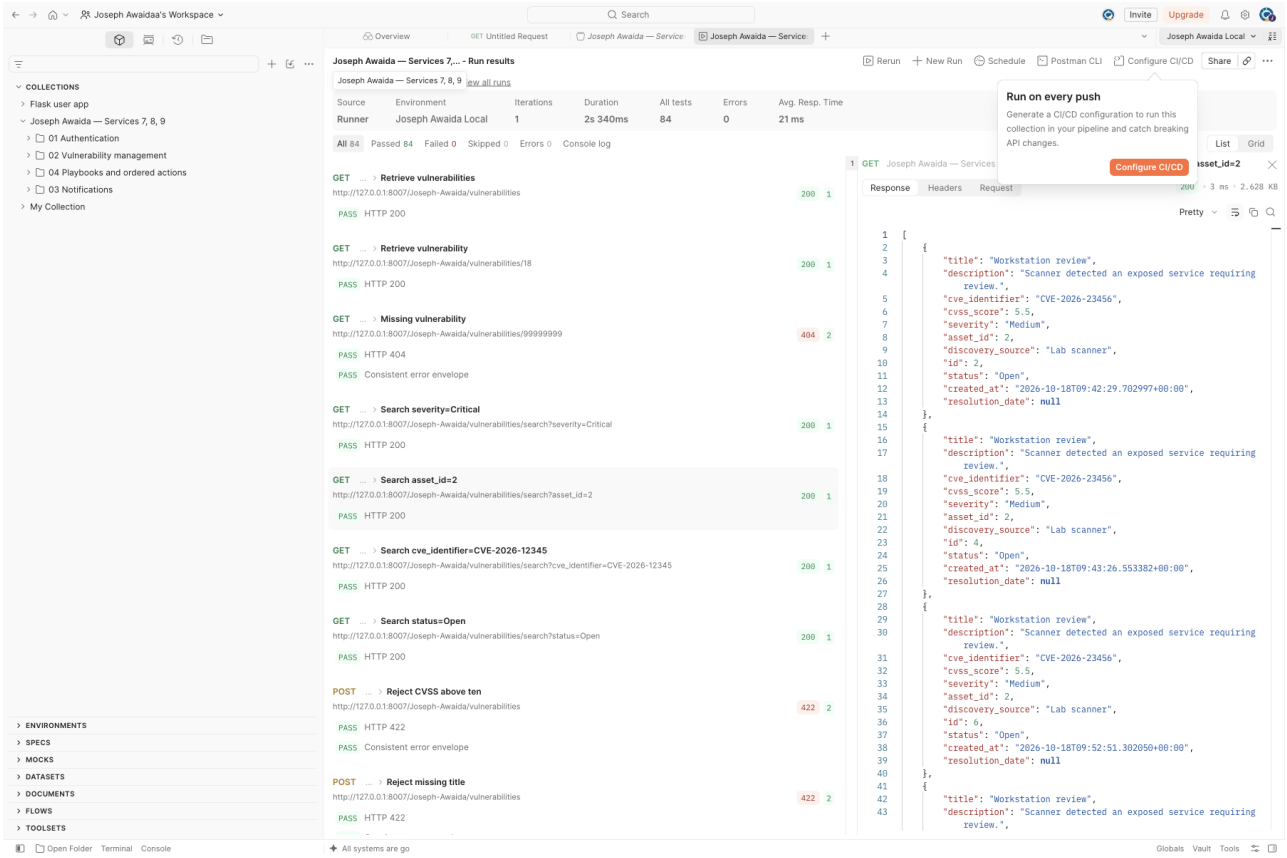


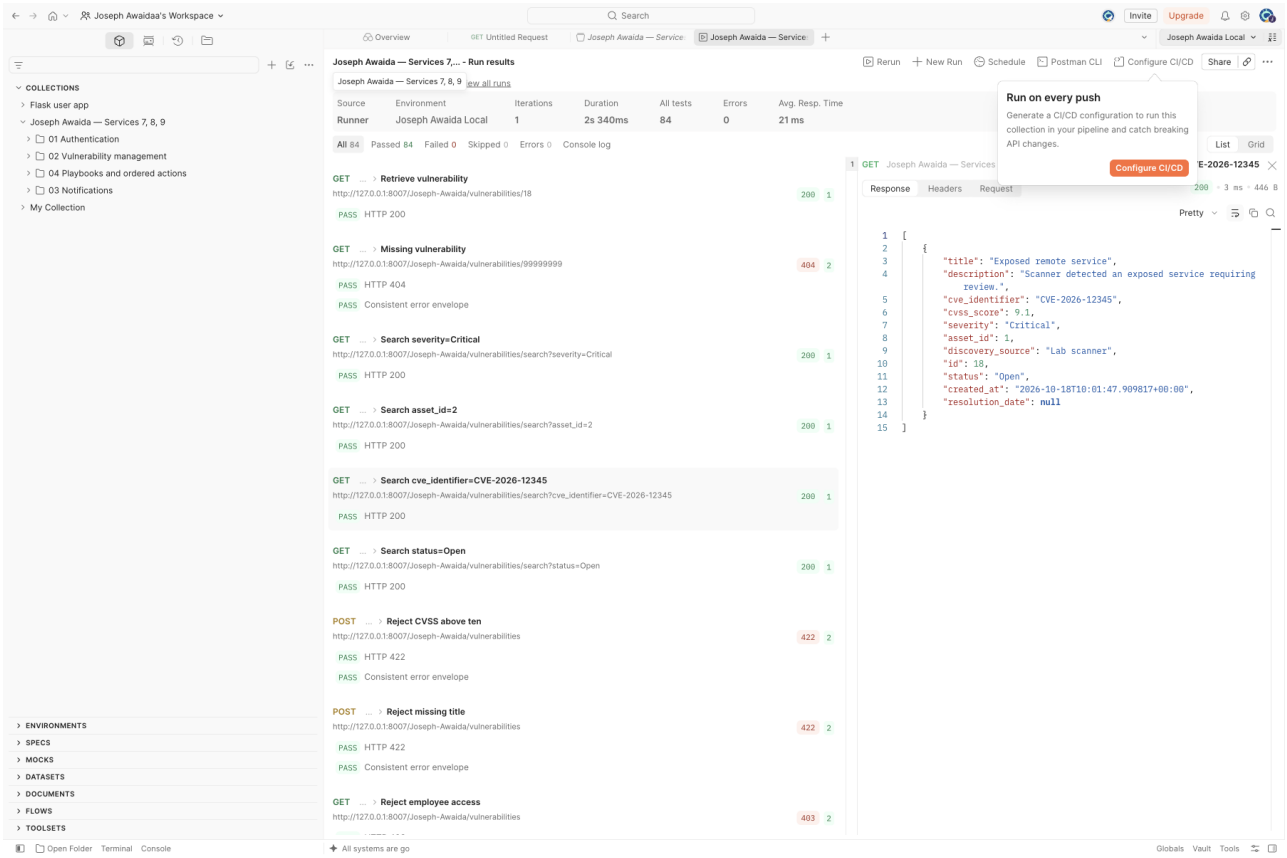
Joseph Awaida — Retrieve vulnerability: actual GET result and HTTP status.



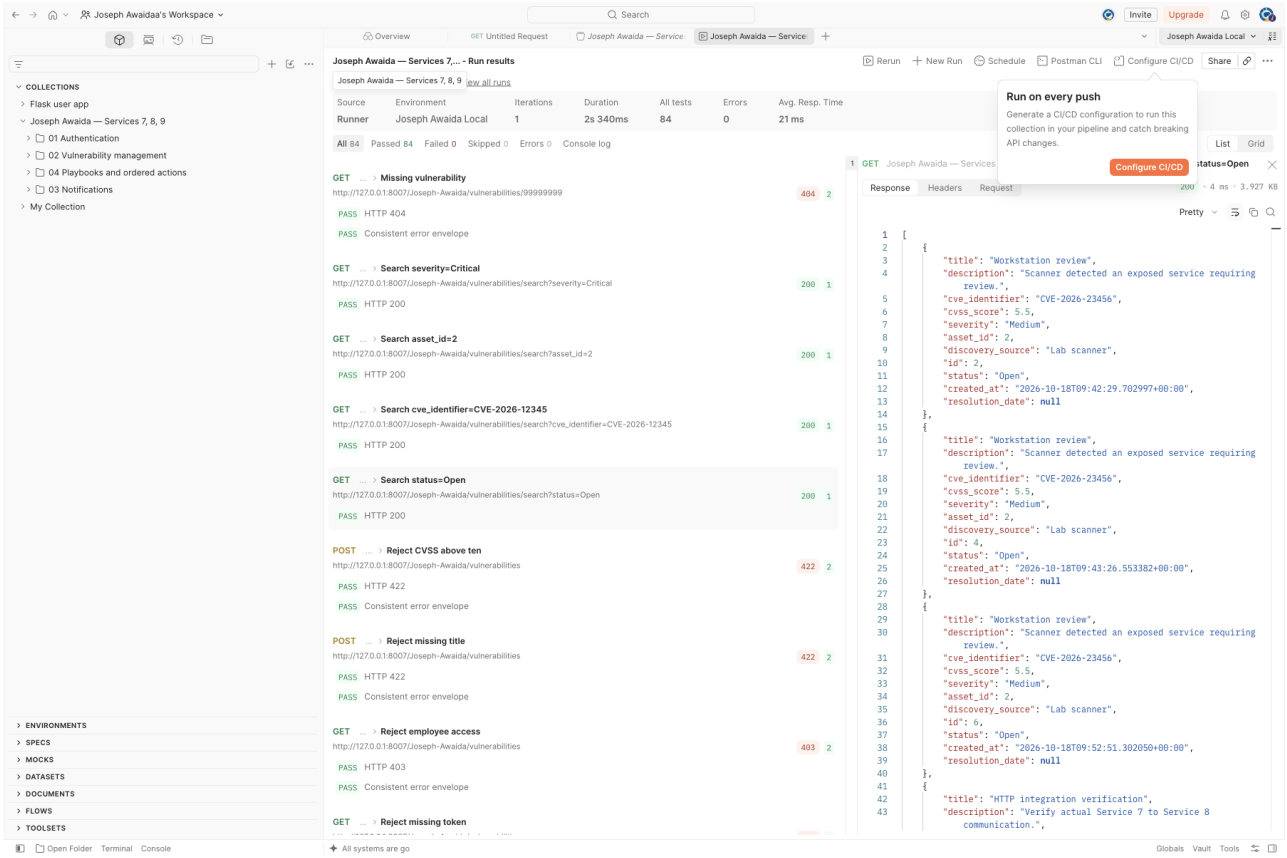
Joseph Awaida — Missing vulnerability: actual GET result and HTTP status.


Joseph Awaida — Search severity=Critical: actual GET result and HTTP status.

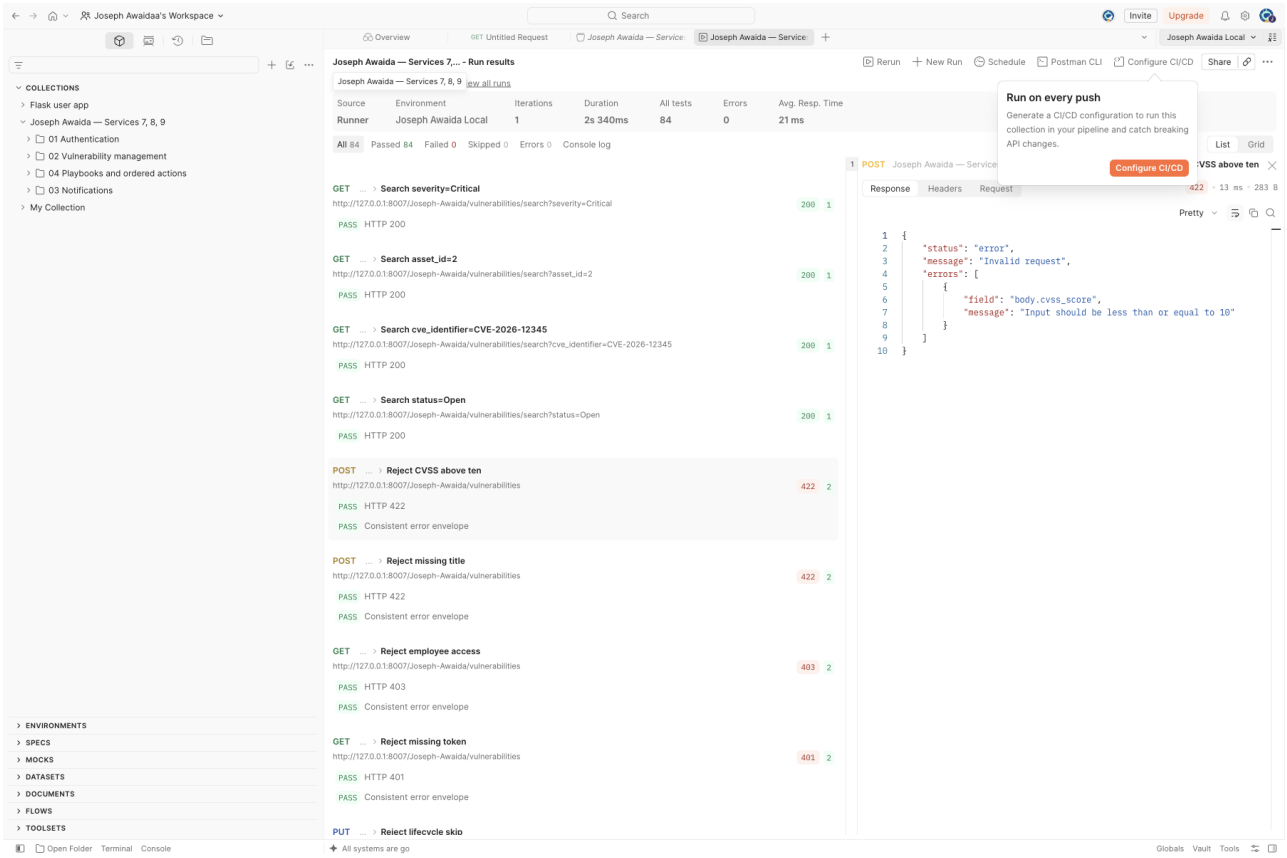

Joseph Awaida — Search asset_id=2: actual GET result and HTTP status.



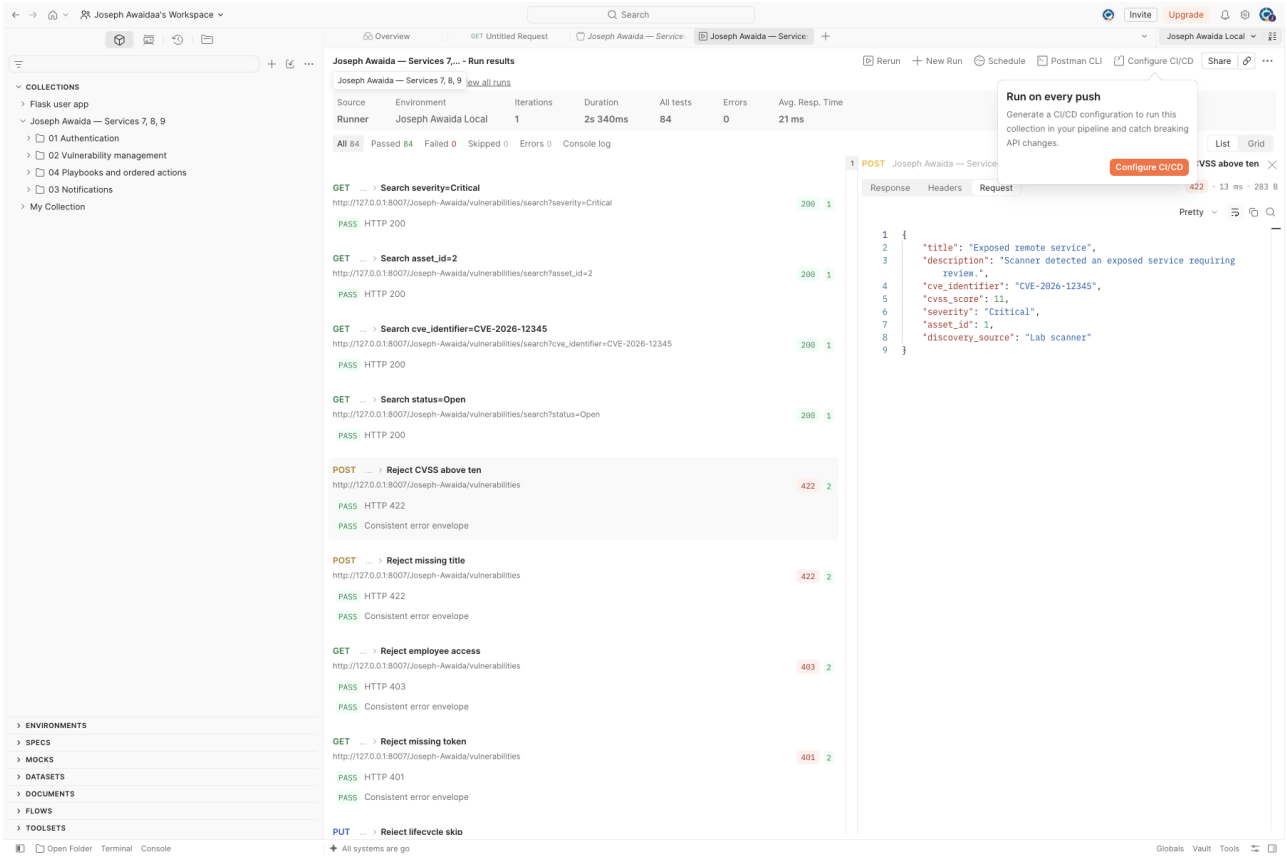
Joseph Awaida — Search cve_identifier=CVE-2026-12345: actual GET result and HTTP status.



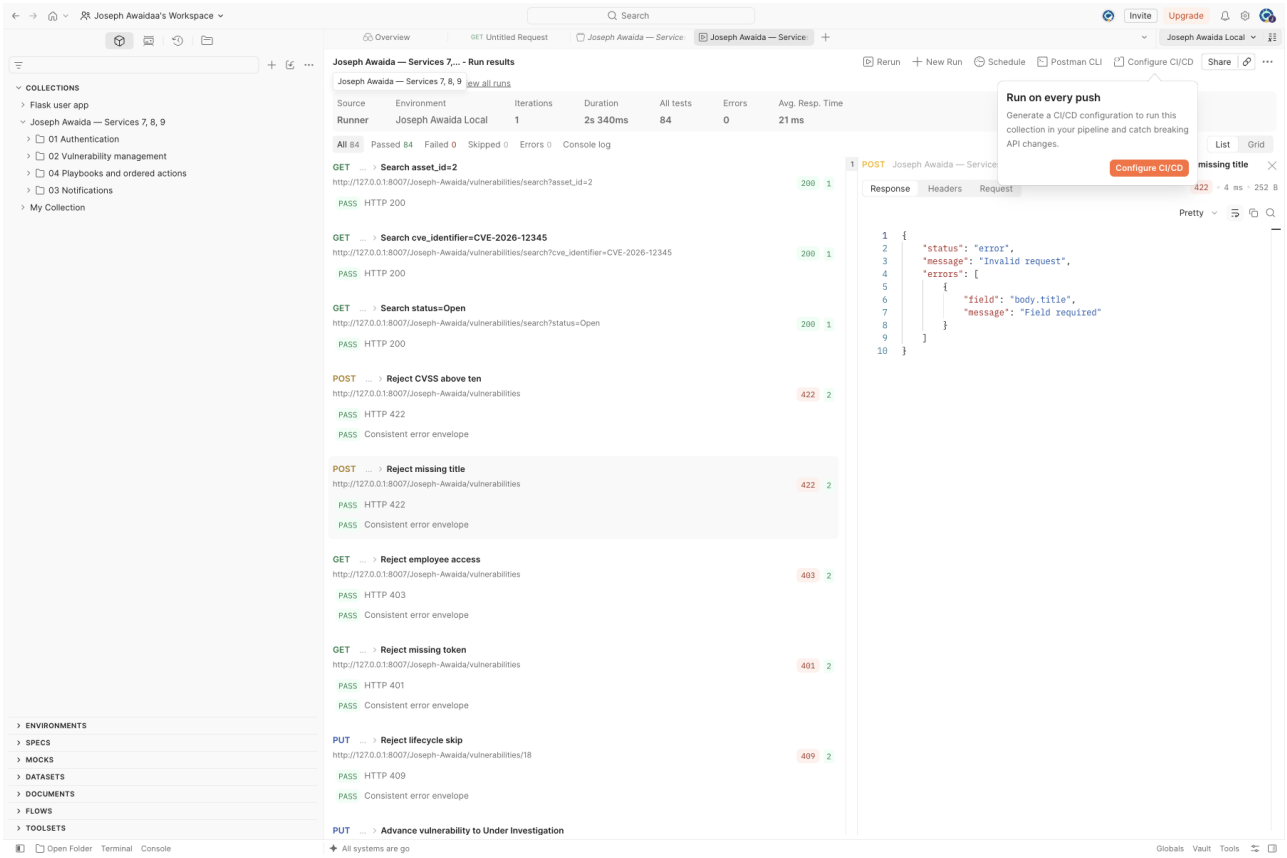
Joseph Awaida — Search status=Open: actual GET result and HTTP status.



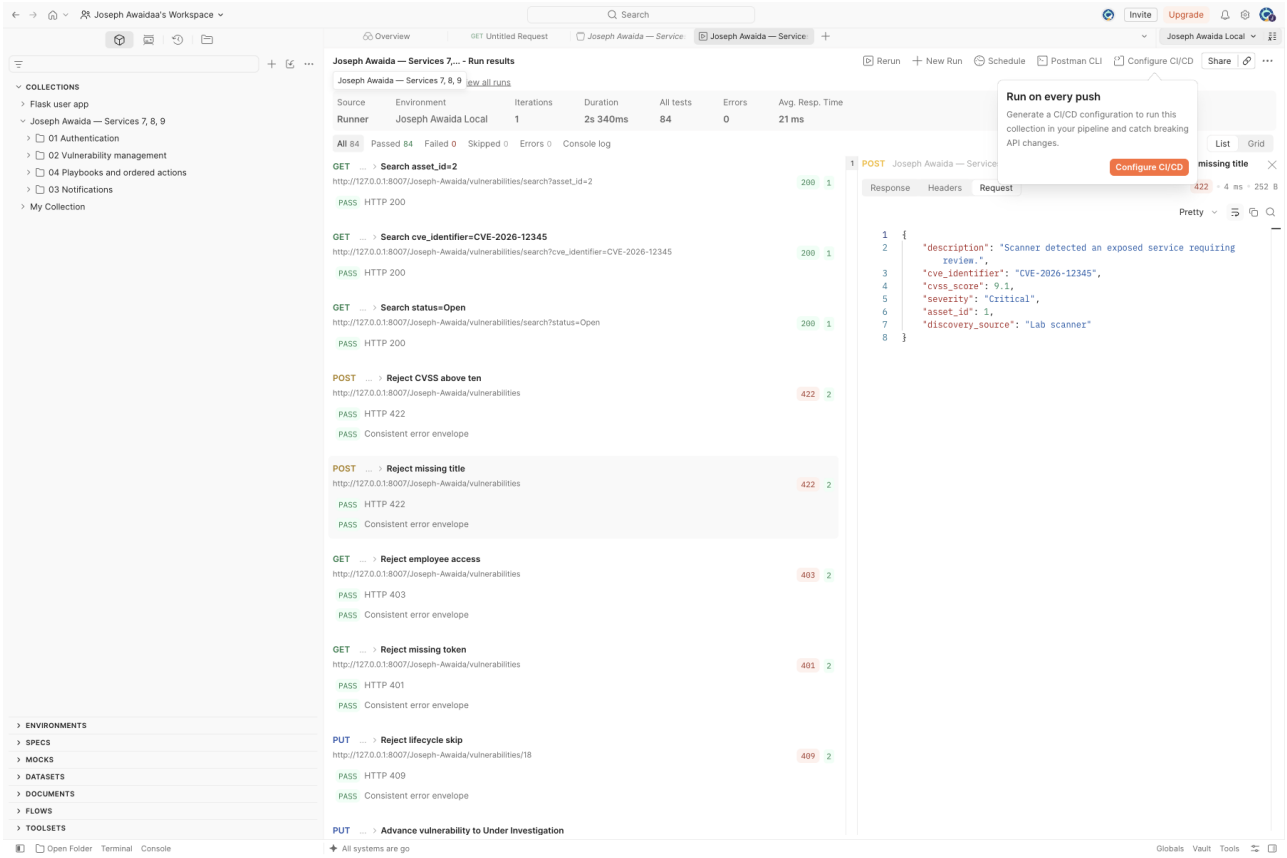
Joseph Awaida — Reject CVSS above ten: actual POST result and HTTP status.



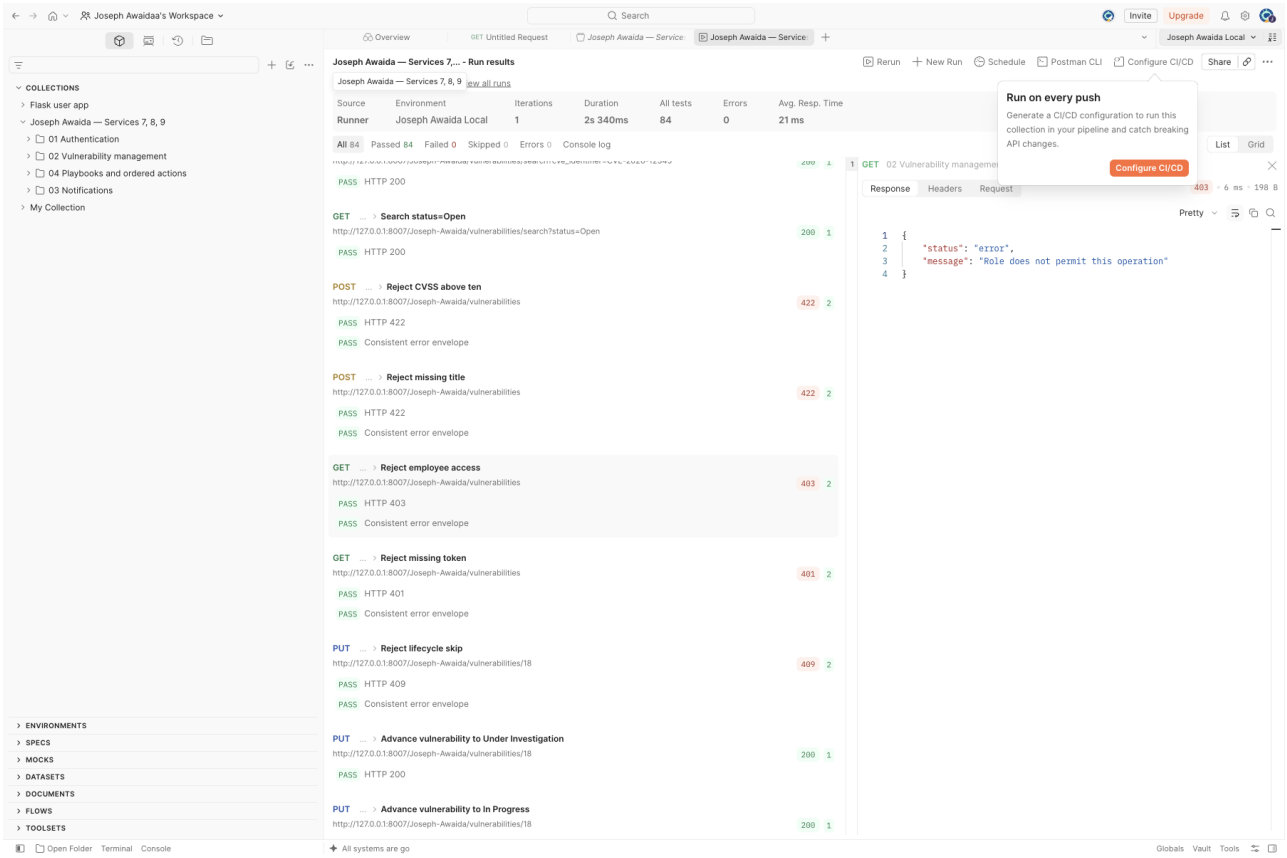
Joseph Awaida — Reject CVSS above ten: actual request body shown in the Postman runner.



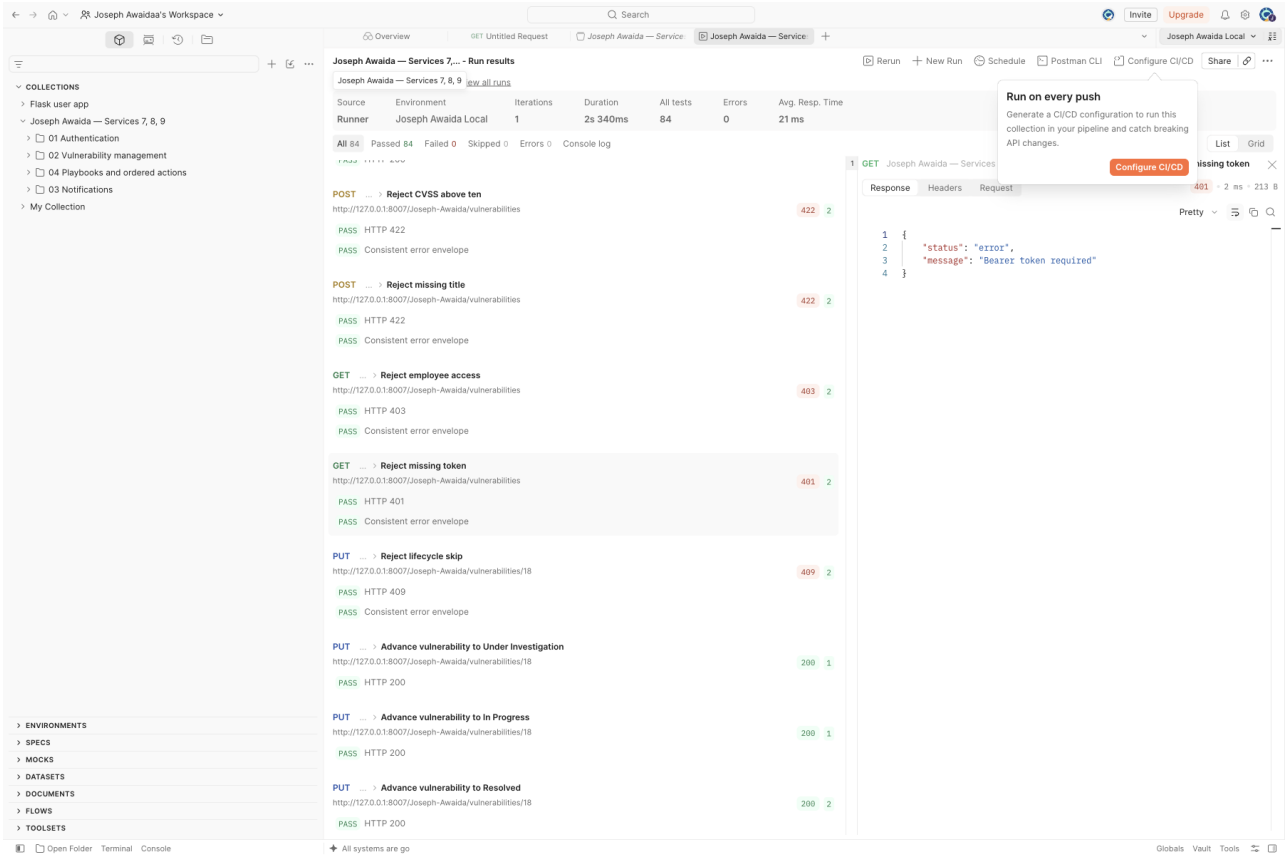
Joseph Awaida — Reject missing title: actual POST result and HTTP status.



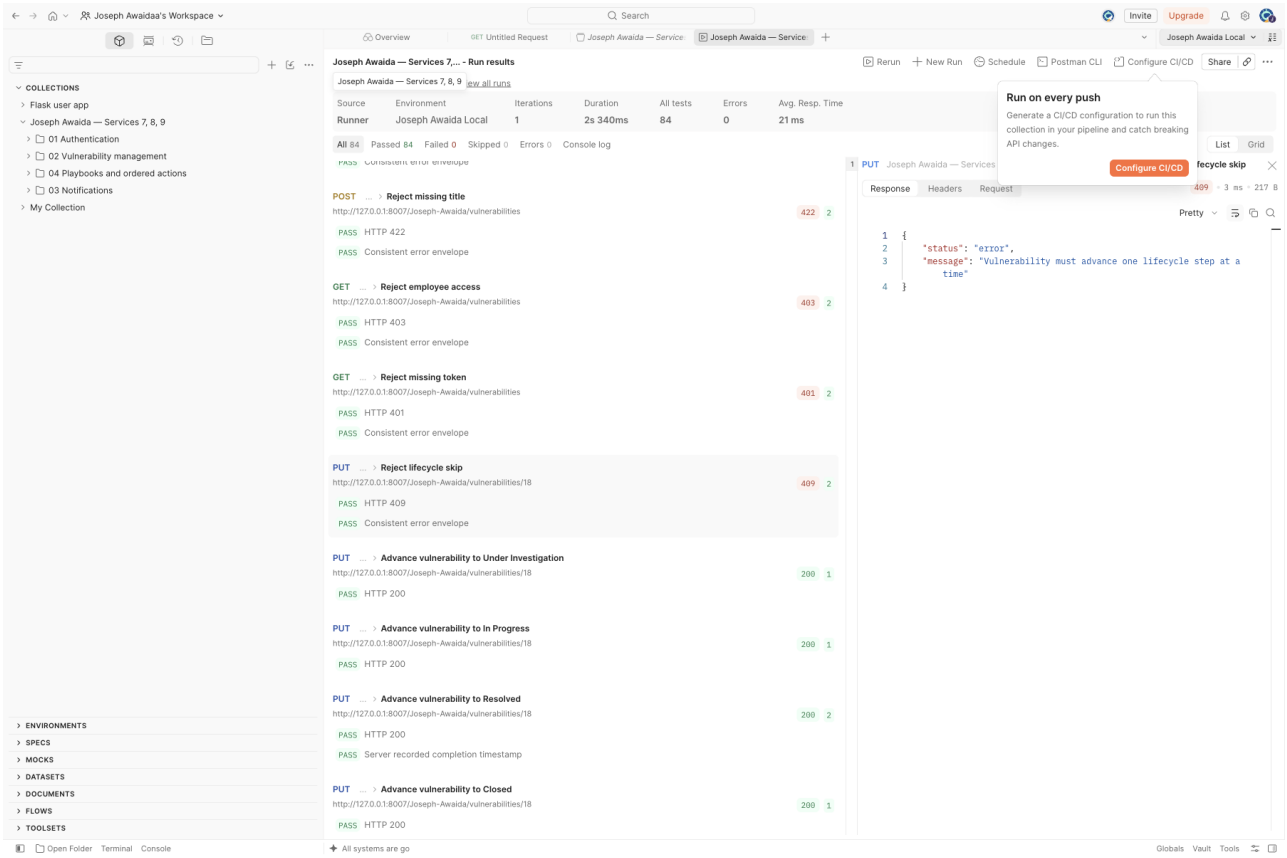
Joseph Awaida — Reject missing title: actual request body shown in the Postman runner.



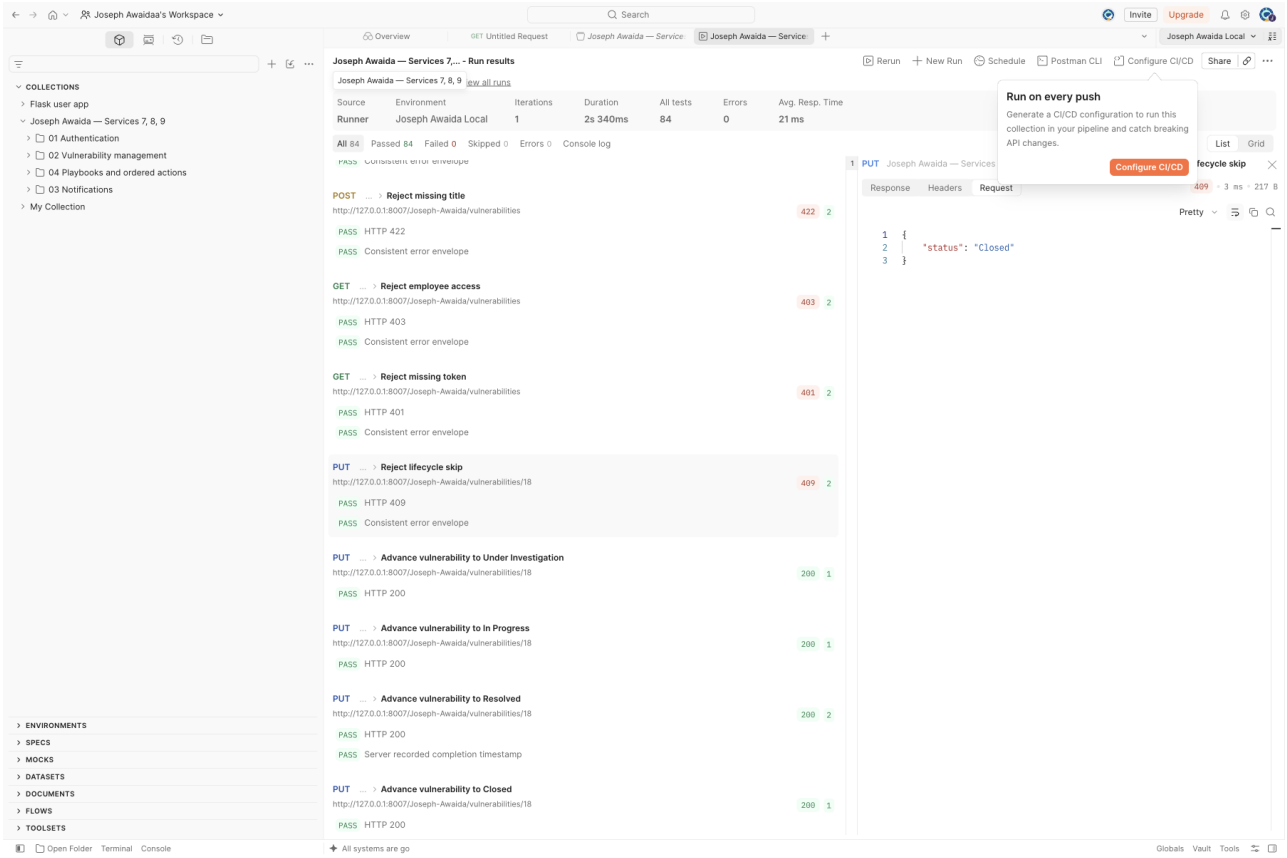
Joseph Awaida — Reject employee access: actual GET result and HTTP status.



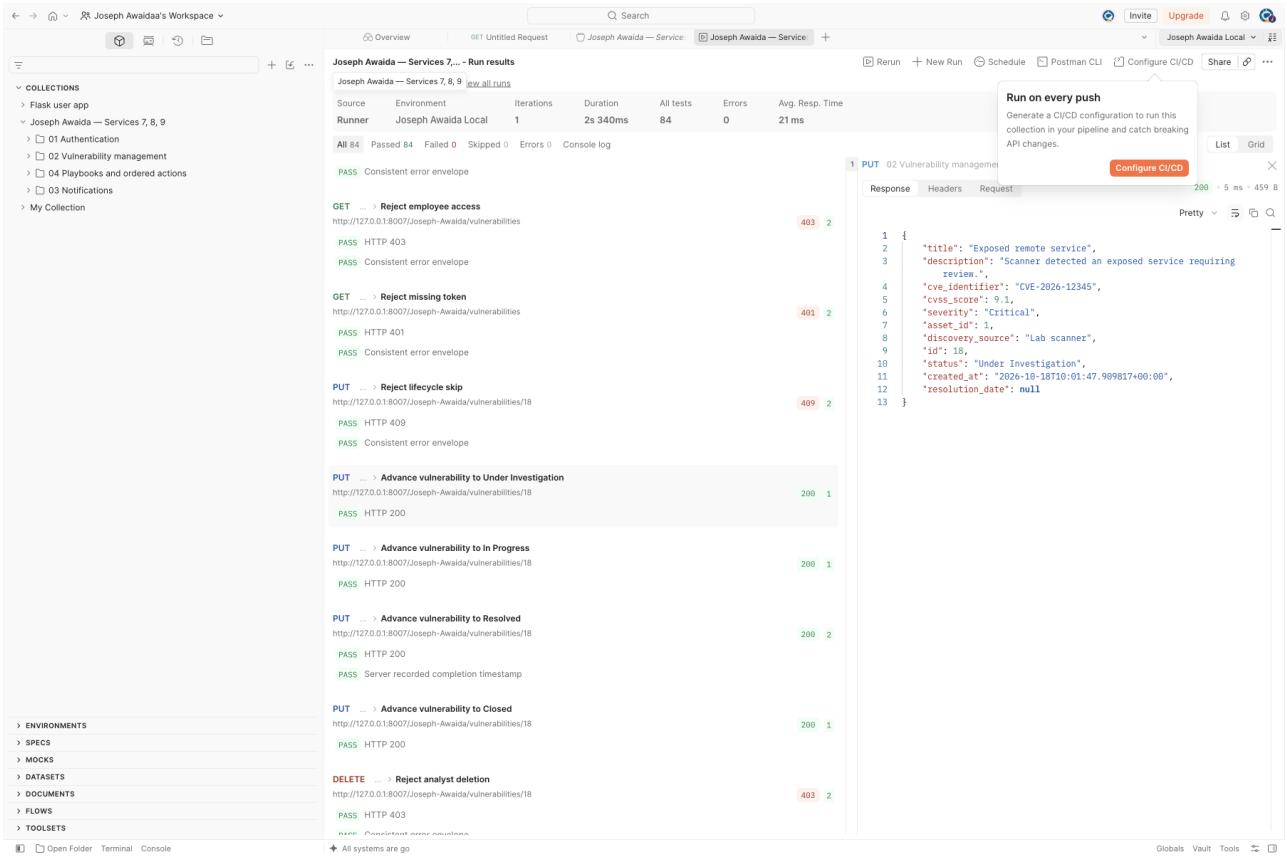
Joseph Awaida — Reject missing token: actual GET result and HTTP status.



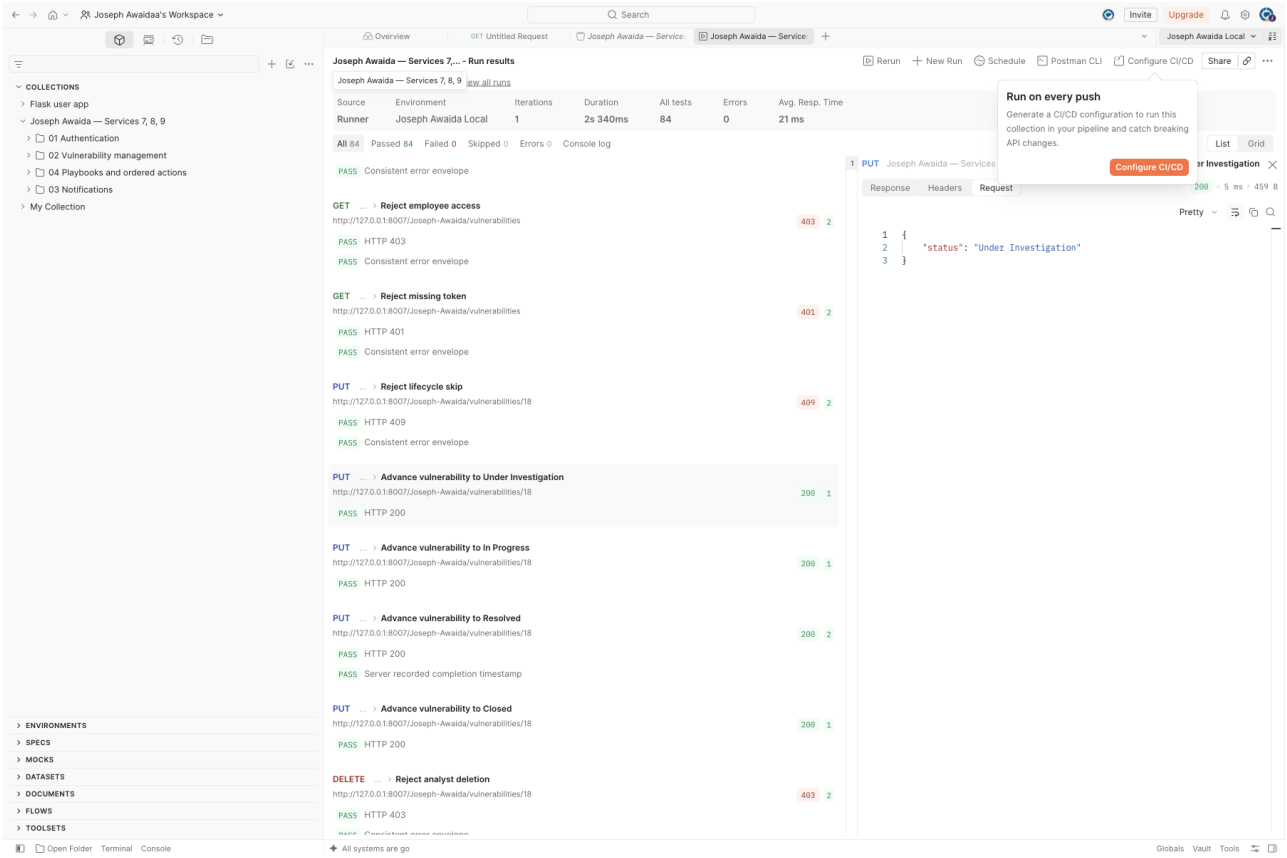
Joseph Awaida — Reject lifecycle skip: actual PUT result and HTTP status.



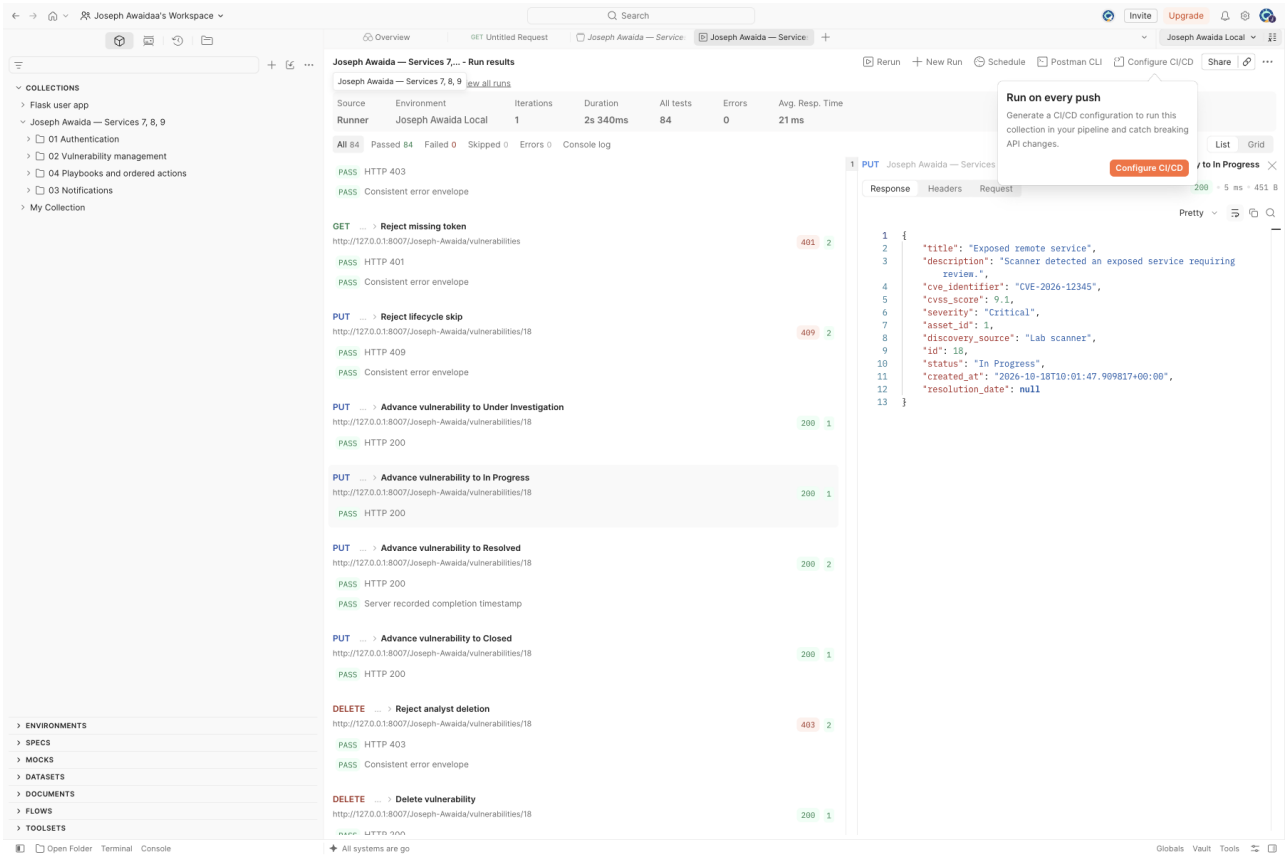
Joseph Awaida — Reject lifecycle skip: actual request body shown in the Postman runner.



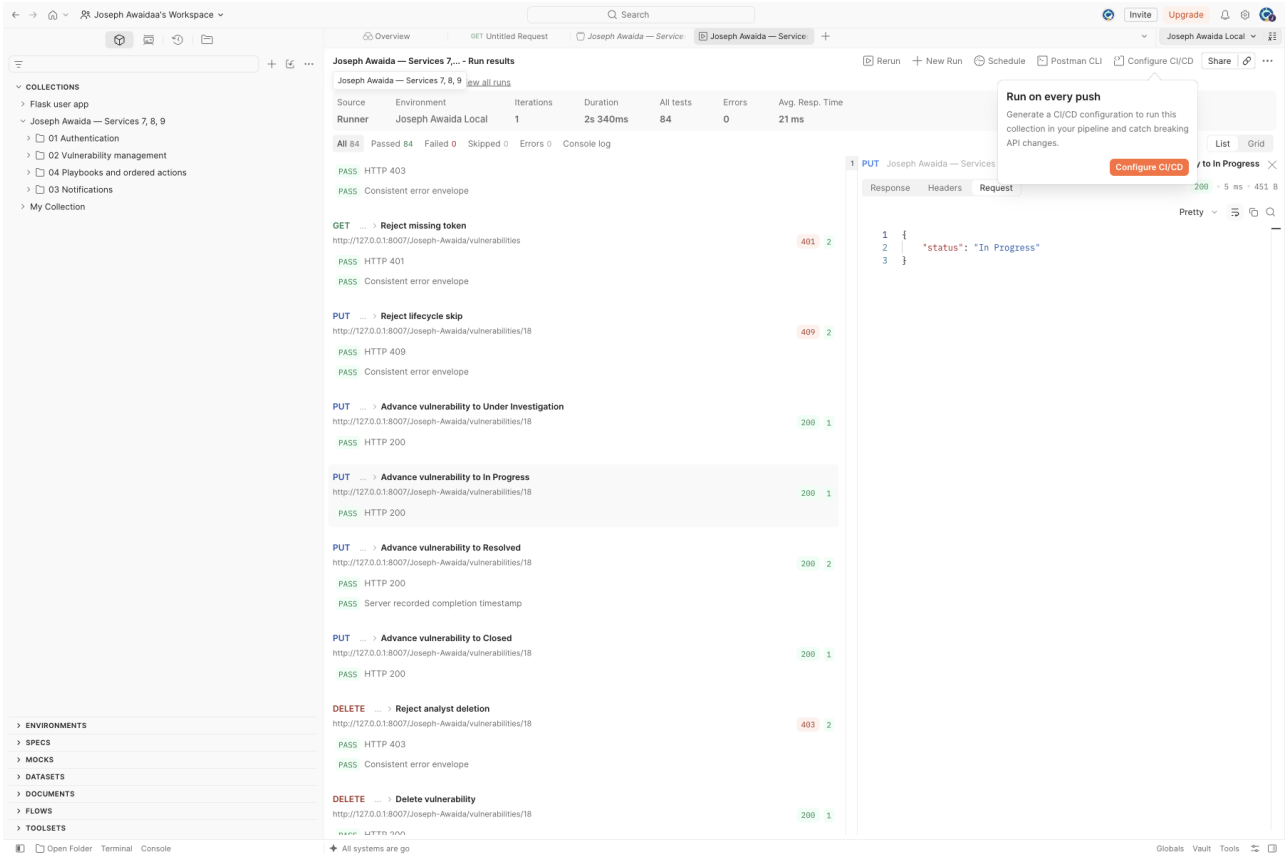
Joseph Awaida — Advance vulnerability to Under Investigation: actual PUT result and HTTP status.



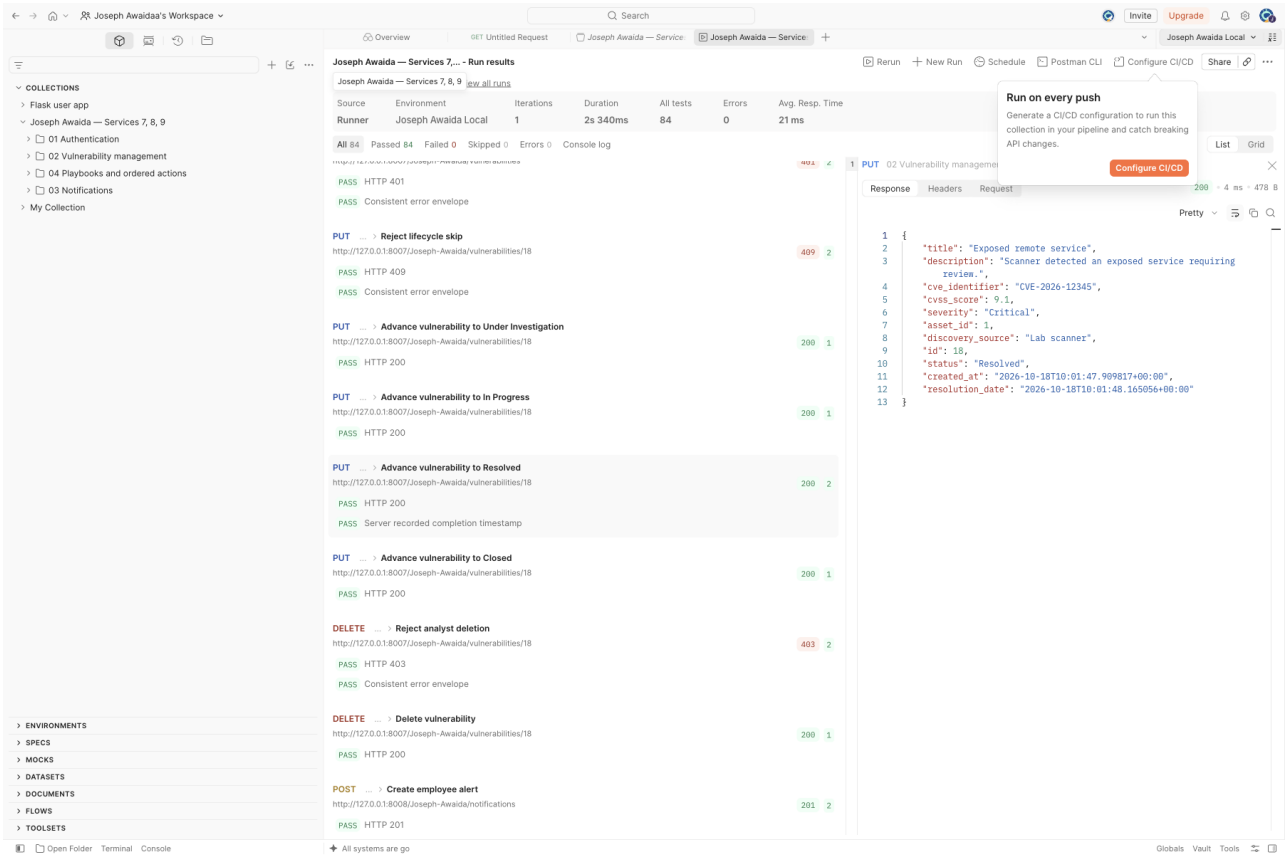
Joseph Awaida — Advance vulnerability to Under Investigation: actual request body shown in the Postman runner.



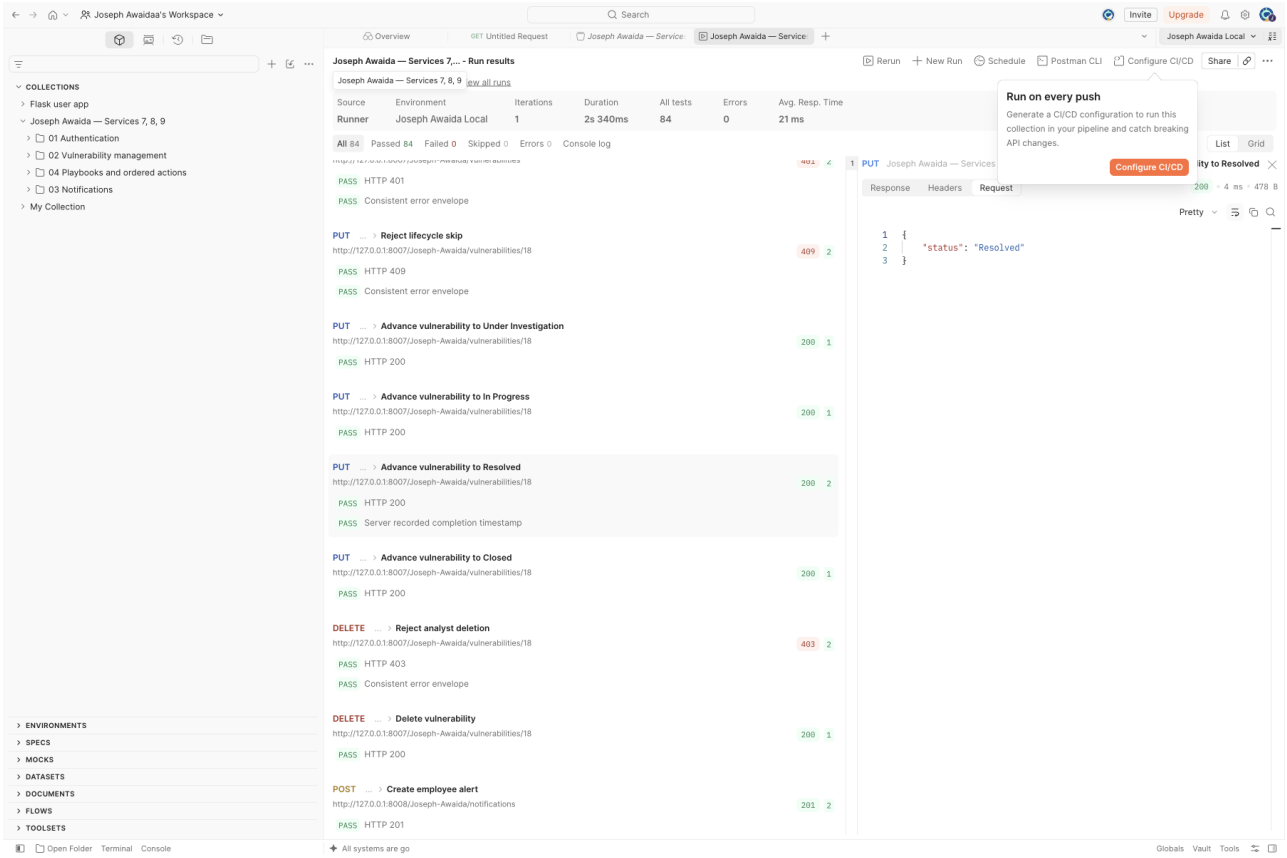
Joseph Awaida — Advance vulnerability to In Progress: actual PUT result and HTTP status.



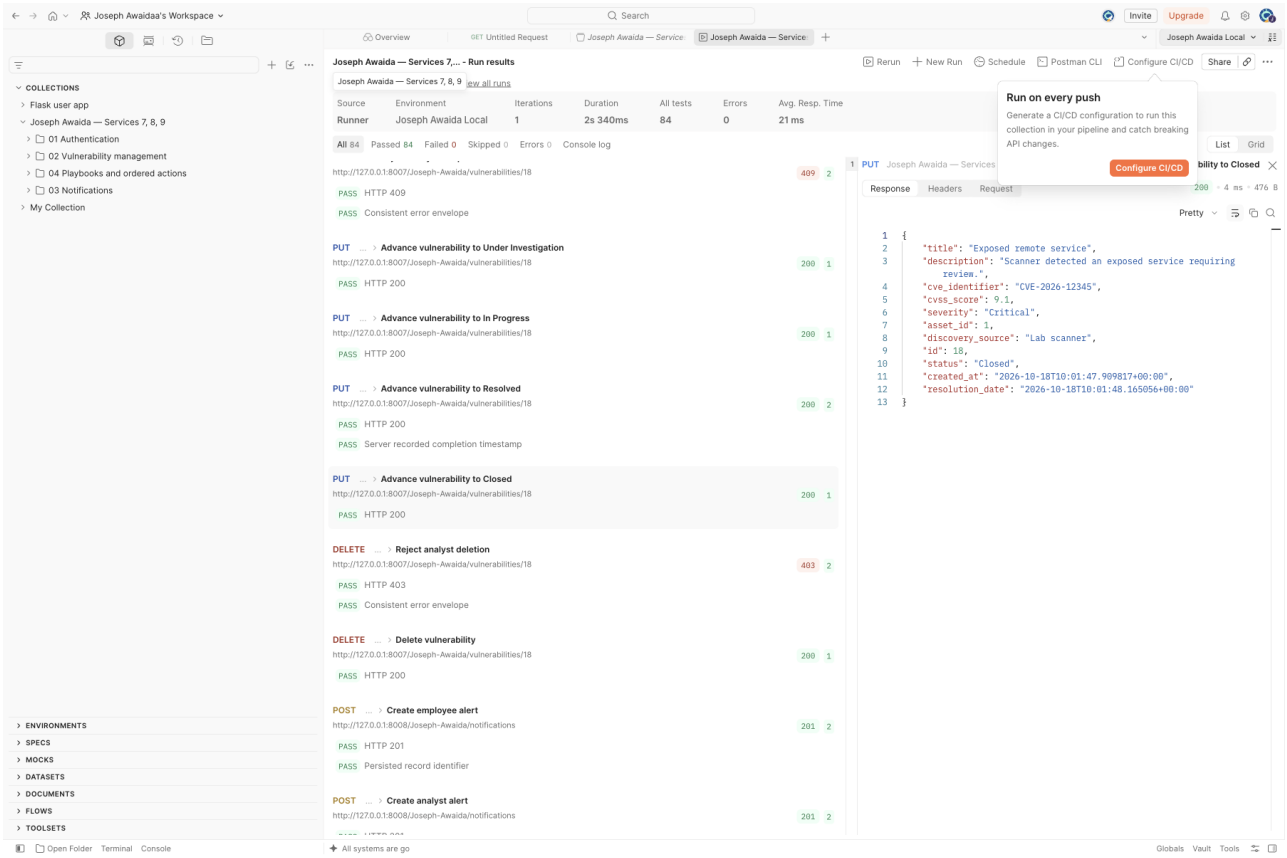
Joseph Awaida — Advance vulnerability to In Progress: actual request body shown in the Postman runner.



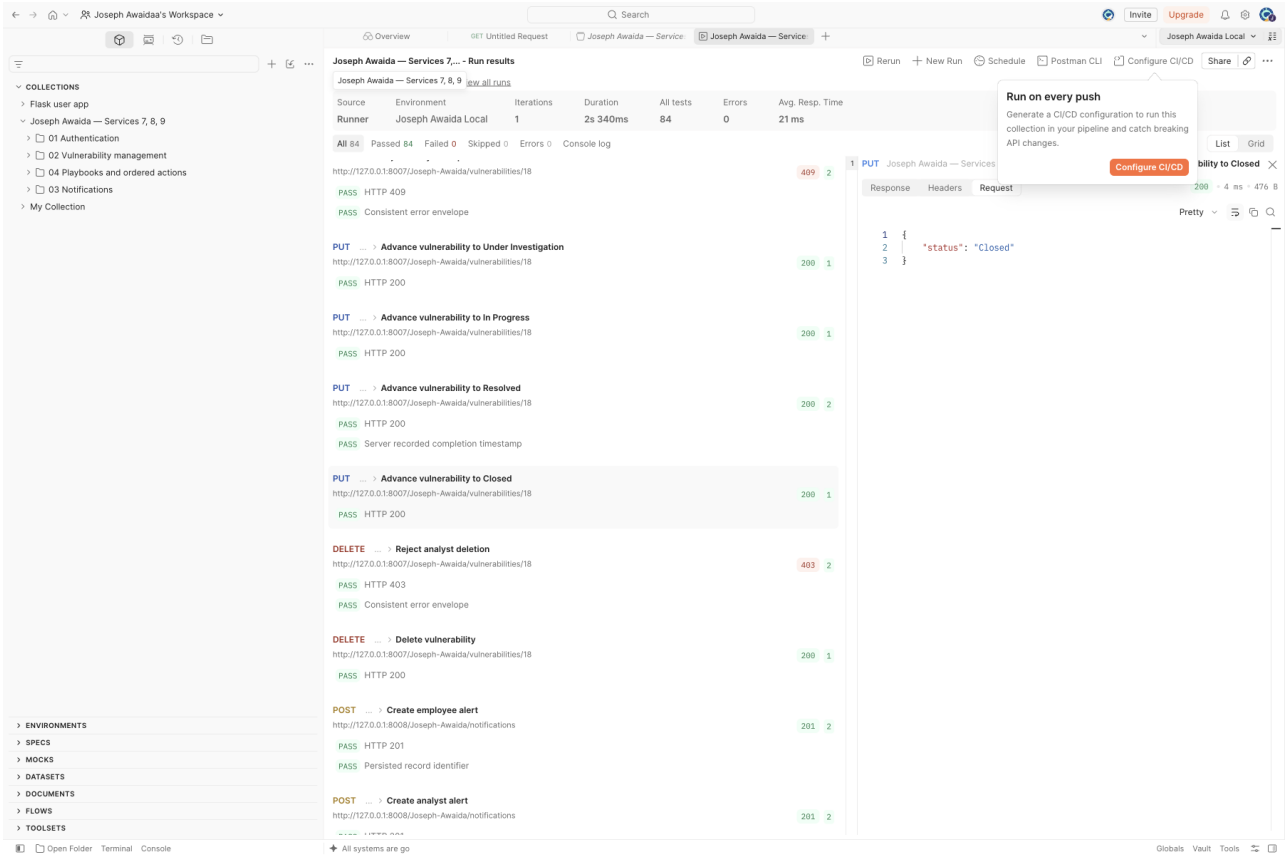
Joseph Awaida — Advance vulnerability to Resolved: actual PUT result and HTTP status.



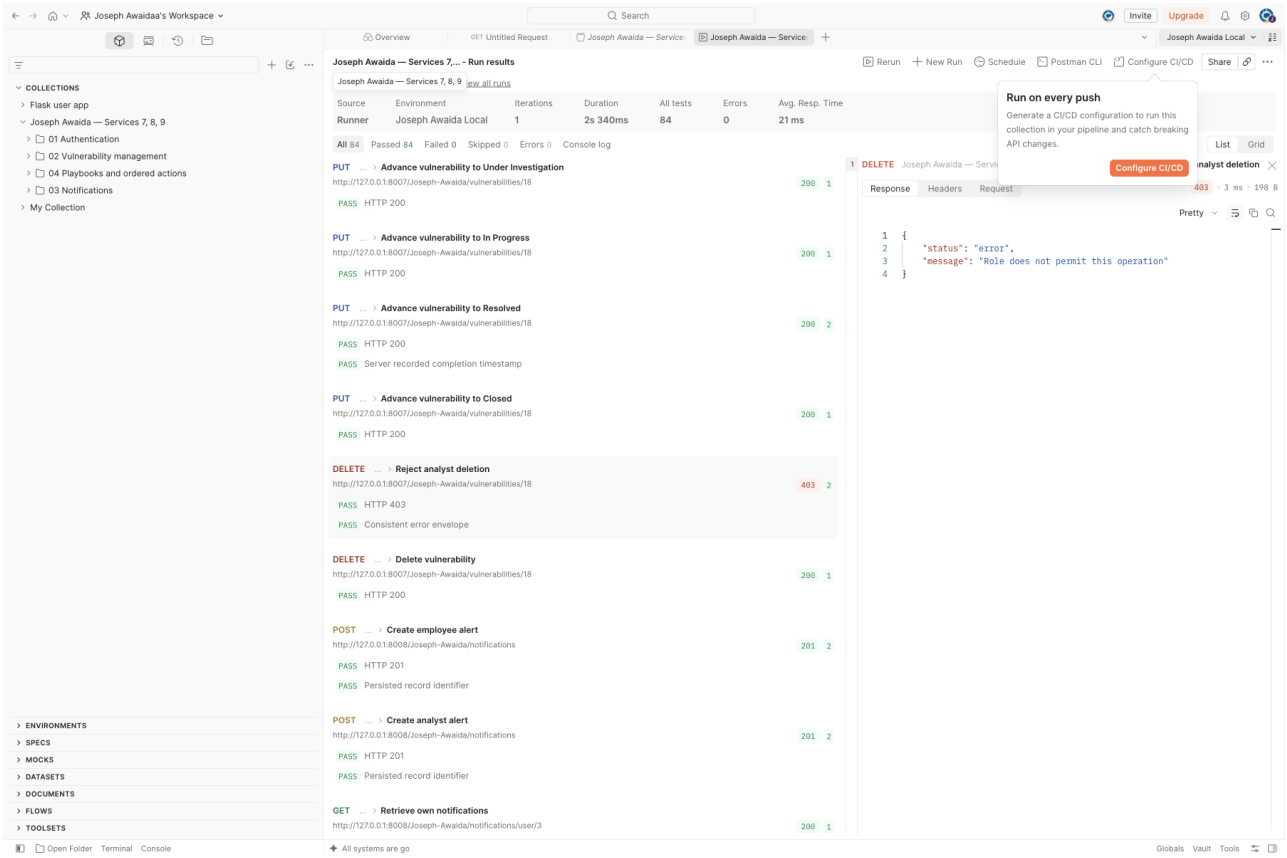
Joseph Awaida — Advance vulnerability to Resolved: actual request body shown in the Postman runner.



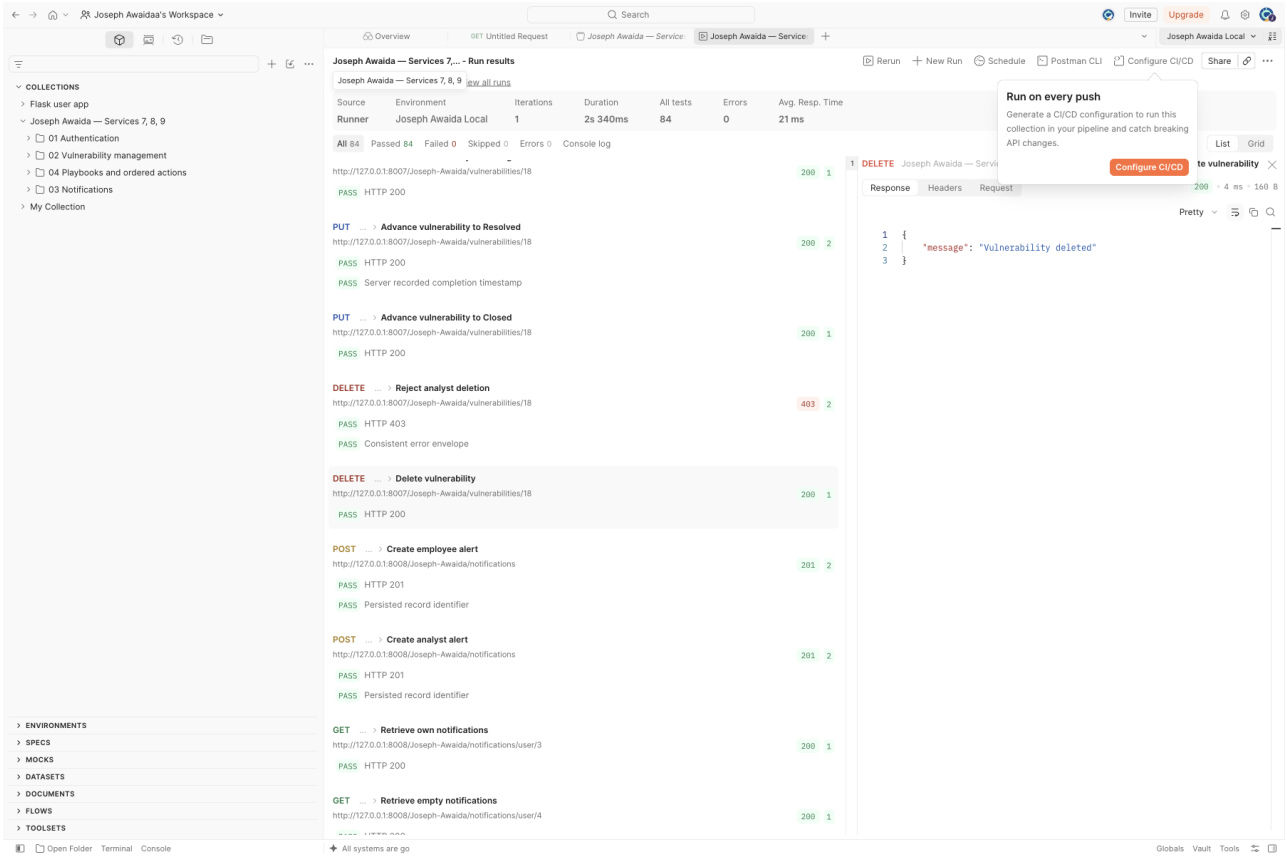
Joseph Awaida — Advance vulnerability to Closed: actual PUT result and HTTP status.



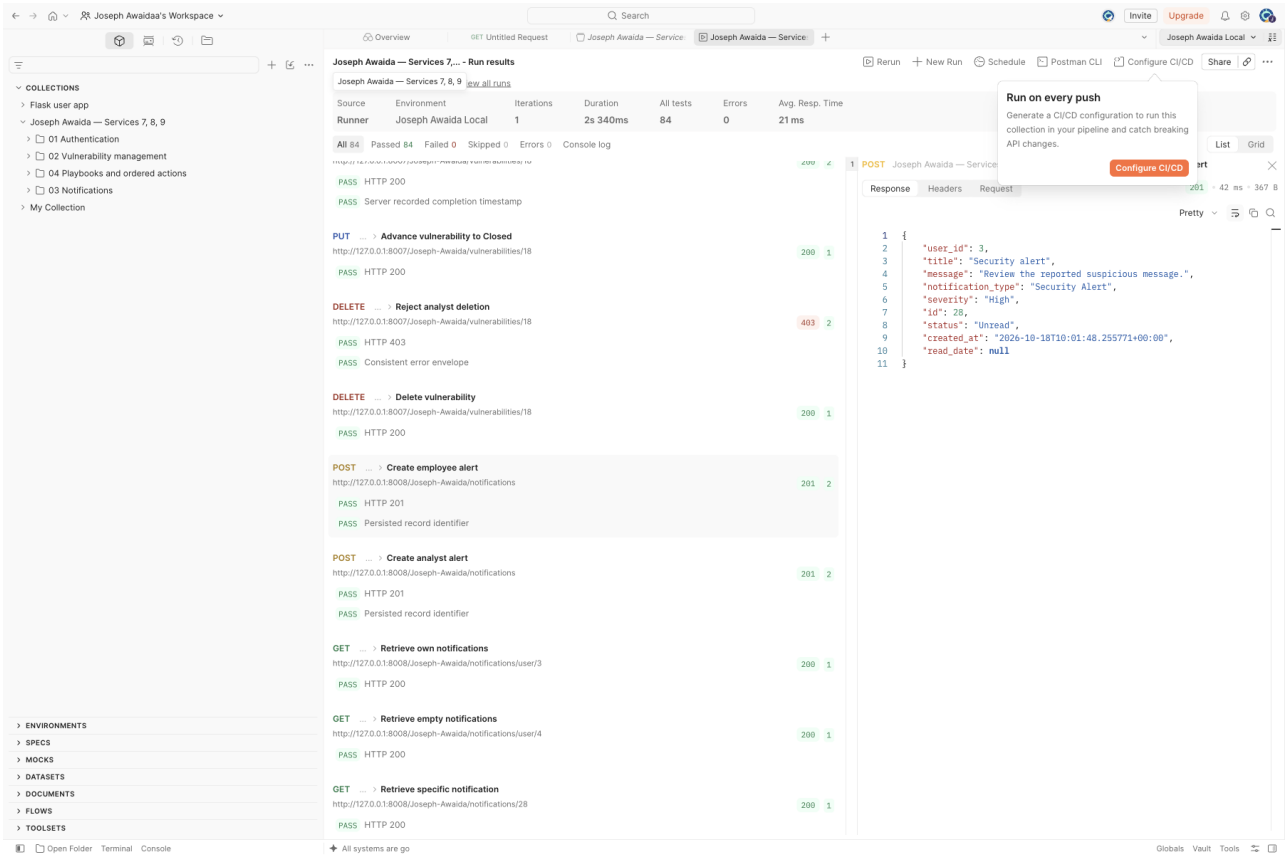
Joseph Awaida — Advance vulnerability to Closed: actual request body shown in the Postman runner.



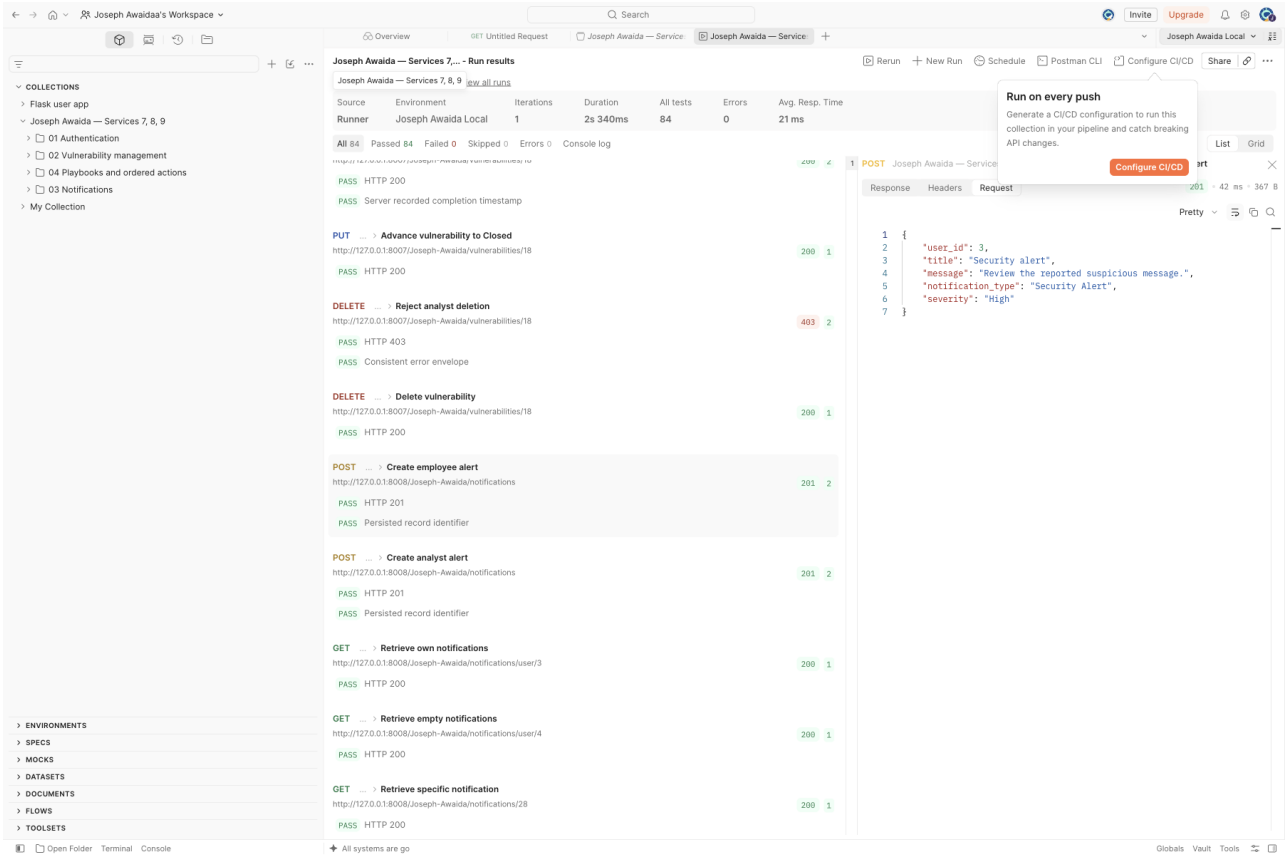
Joseph Awaida — Reject analyst deletion: actual DELETE result and HTTP status.



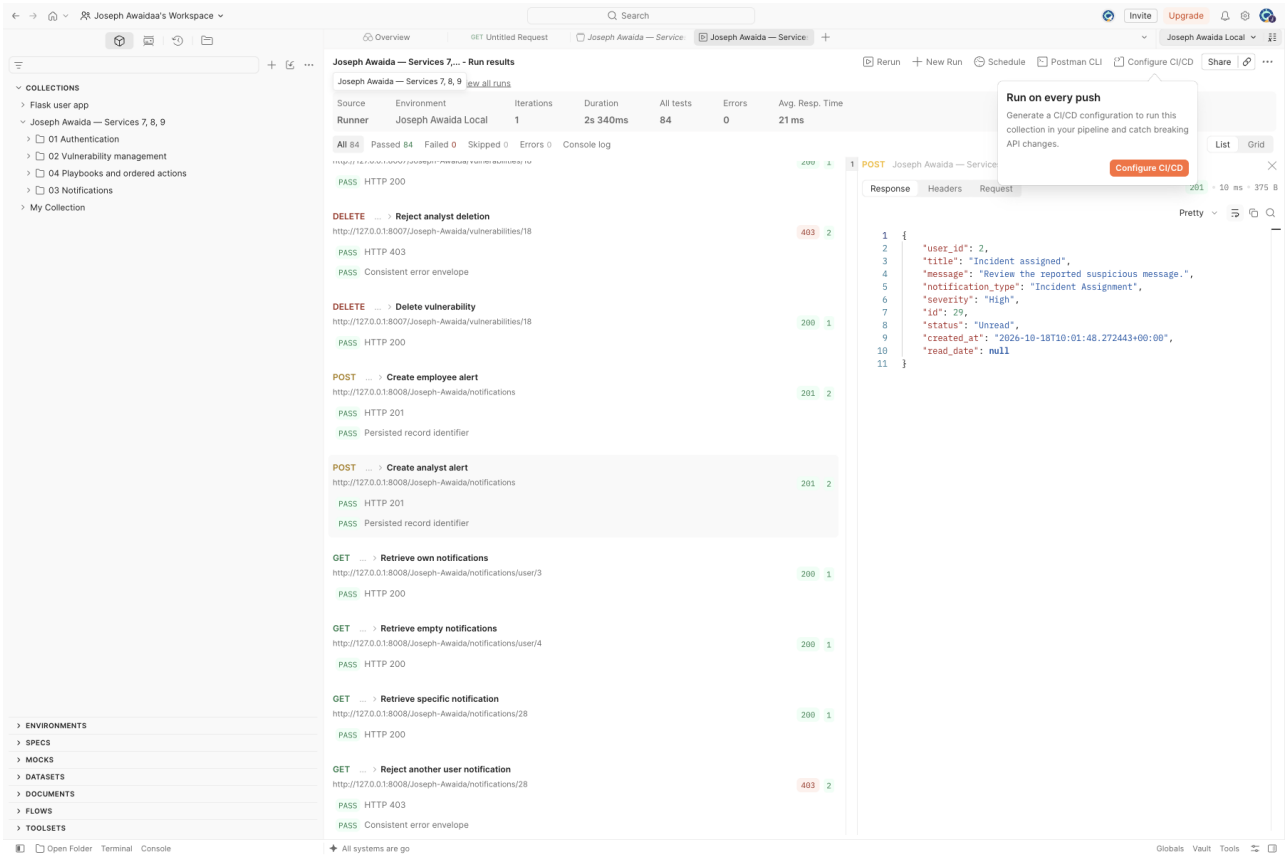
Joseph Awaida — Delete vulnerability: actual DELETE result and HTTP status.



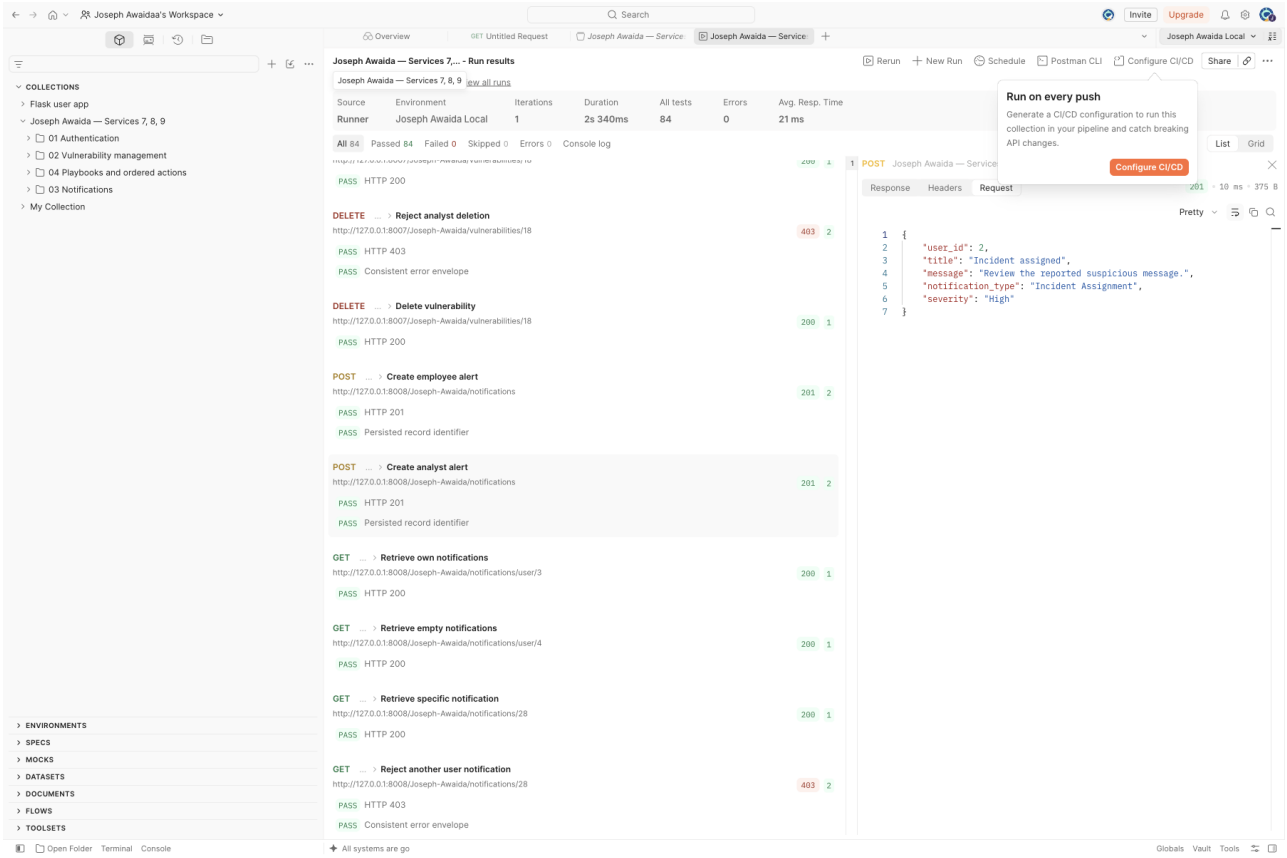
Joseph Awaida — Create employee alert: actual POST result and HTTP status.



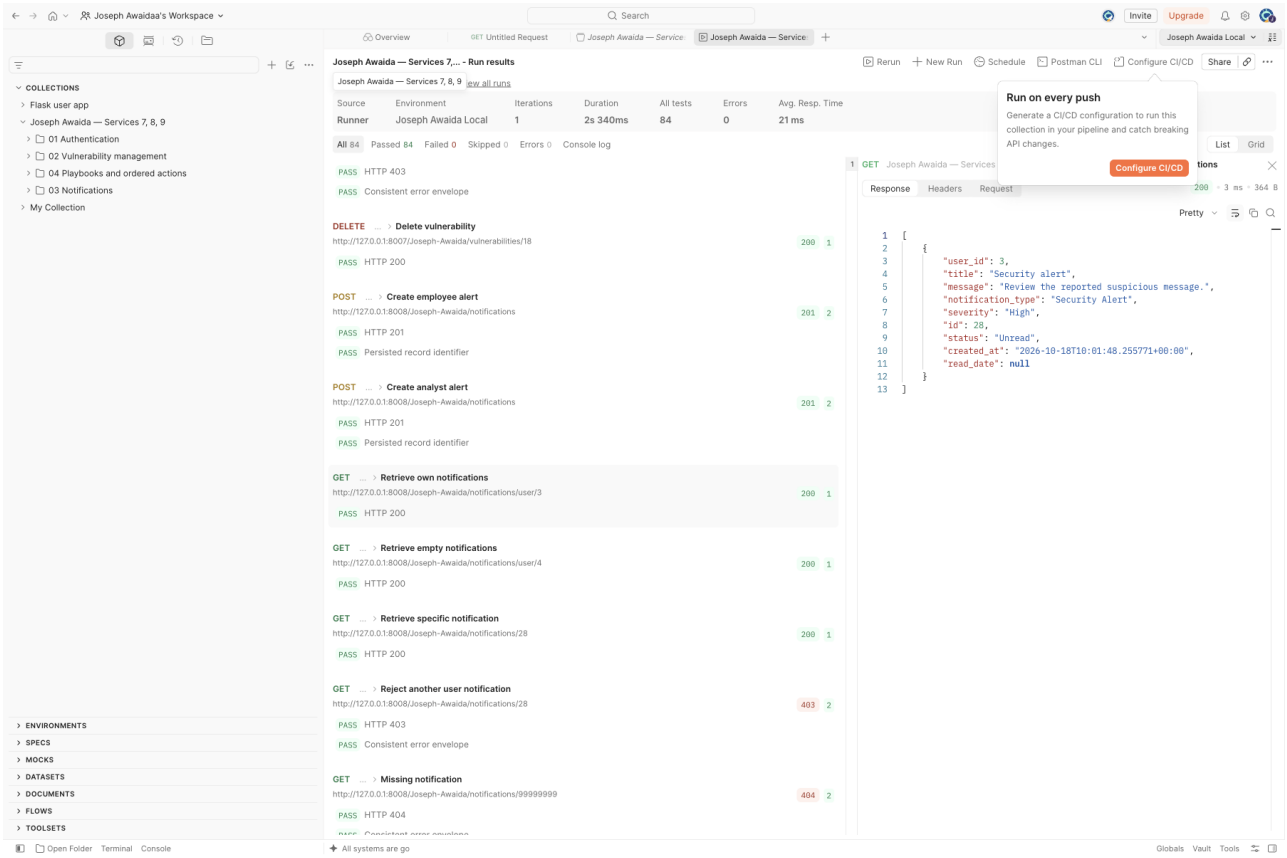
Joseph Awaida — Create employee alert: actual request body shown in the Postman runner.



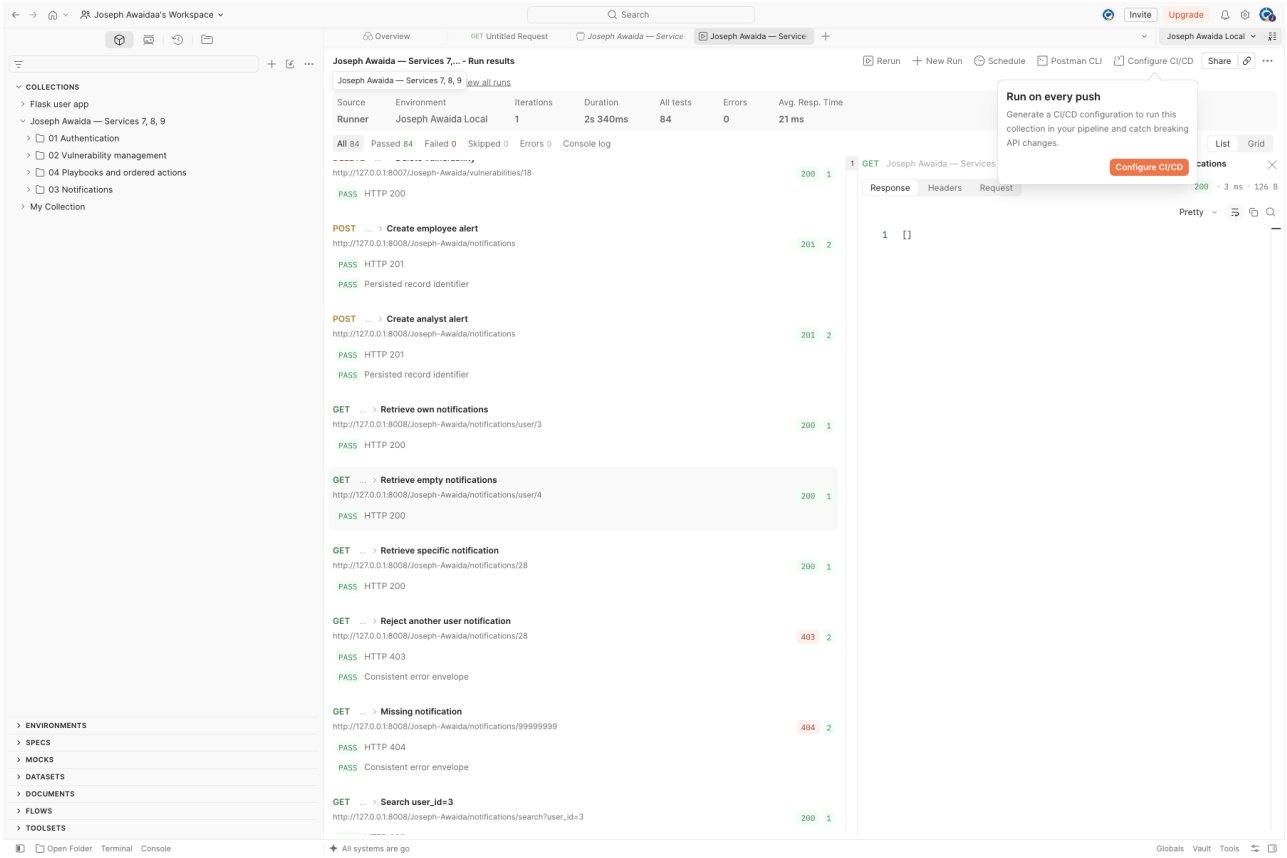
Joseph Awaida — Create analyst alert: actual POST result and HTTP status.



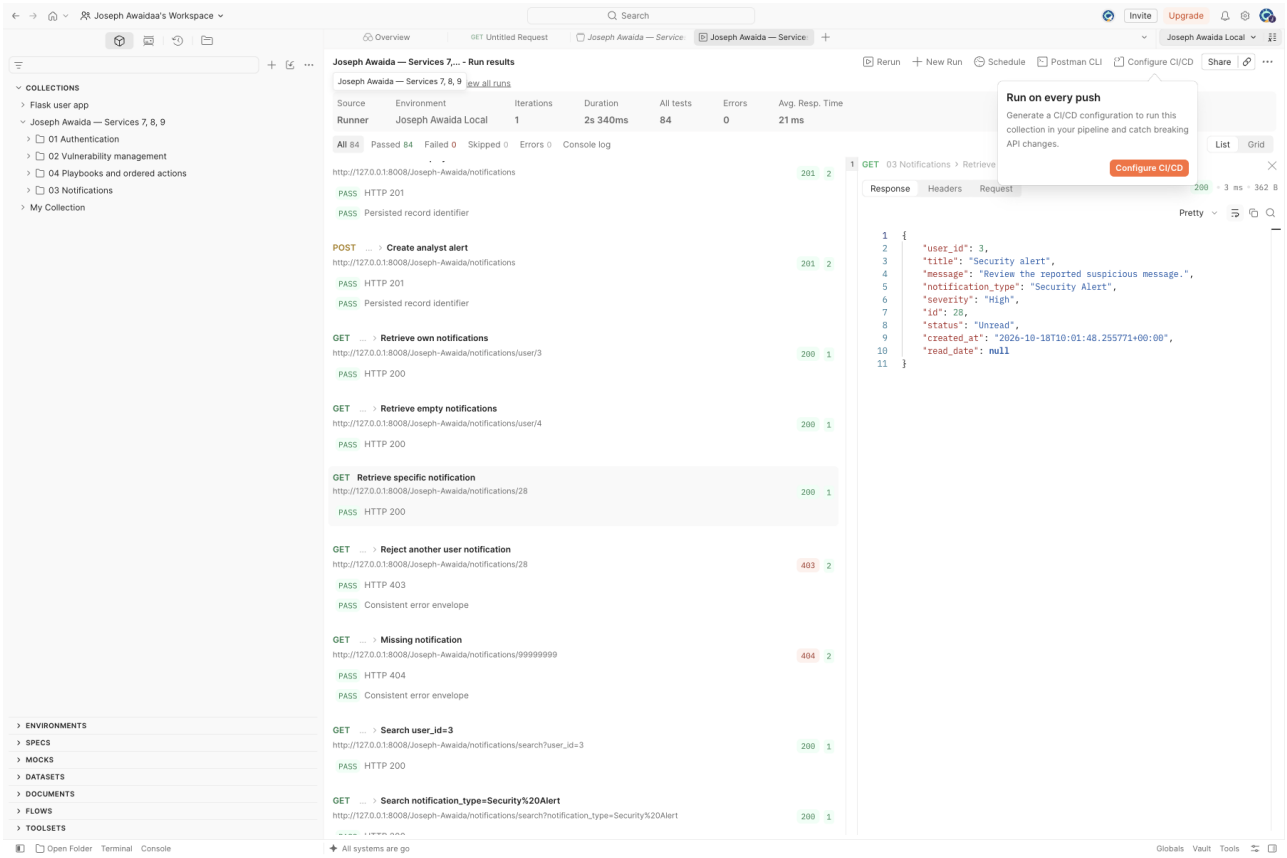
Joseph Awaida — Create analyst alert: actual request body shown in the Postman runner.

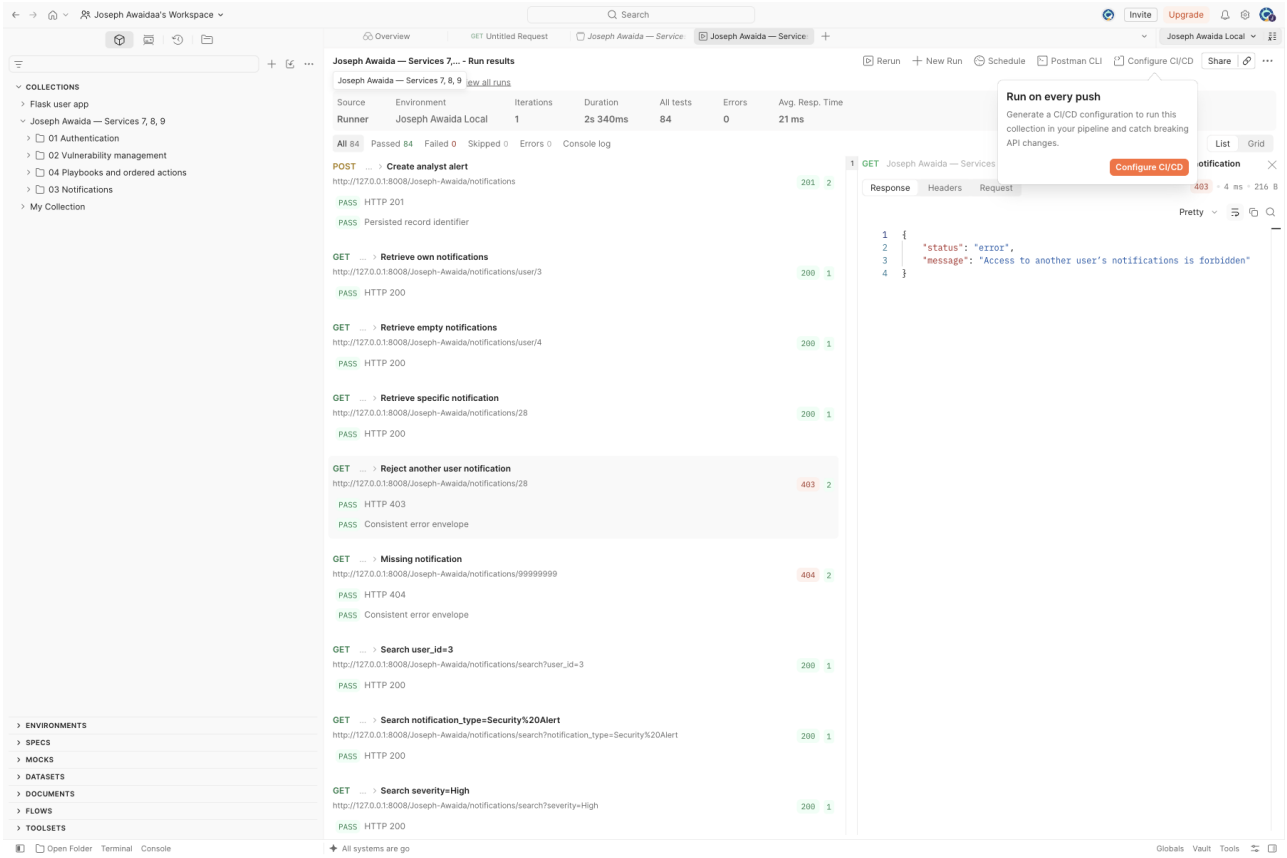


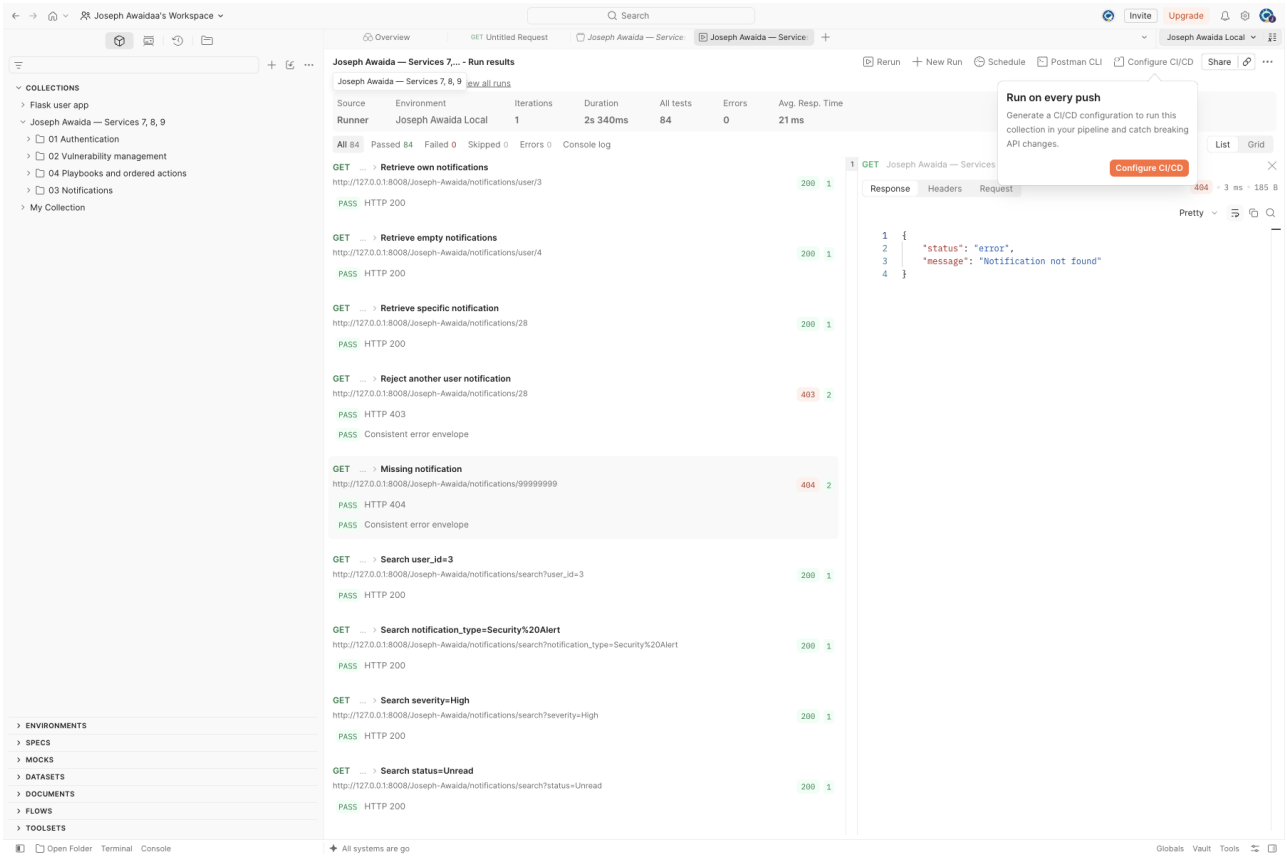
Joseph Awaida — Retrieve own notifications: actual GET result and HTTP status.



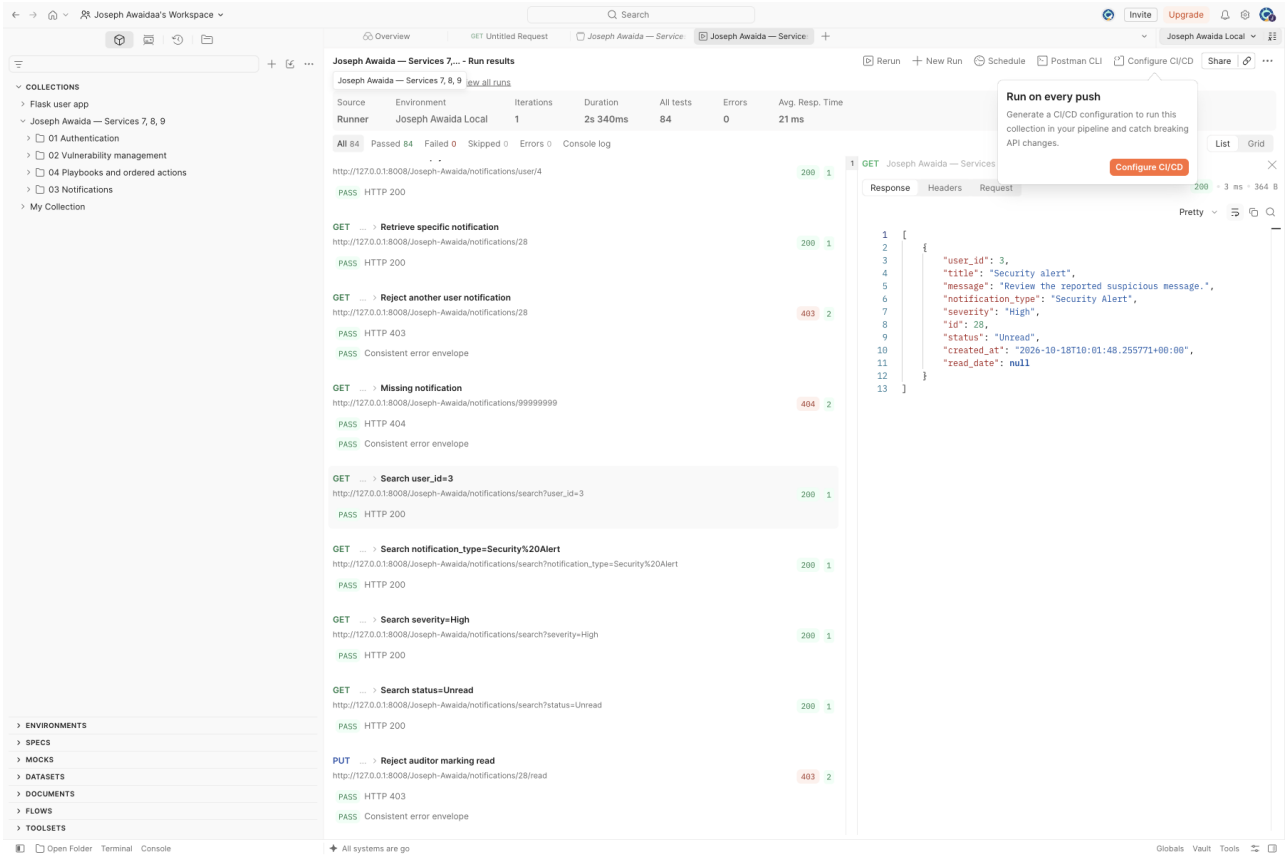
Joseph Awaida — Retrieve empty notifications: actual GET result and HTTP status.


Joseph Awaida — Retrieve specific notification: actual GET result and HTTP status.

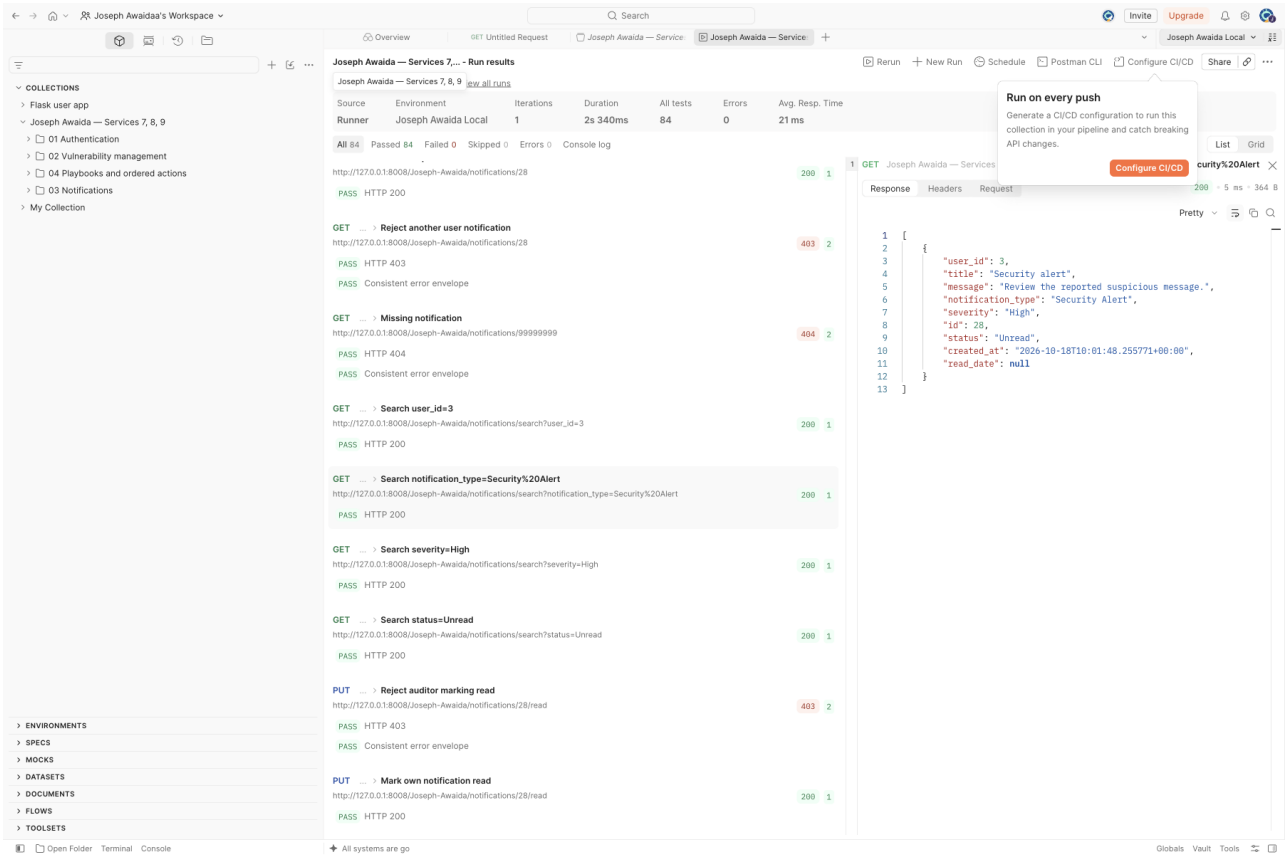

Joseph Awaida — Reject another user notification: actual GET result and HTTP status.



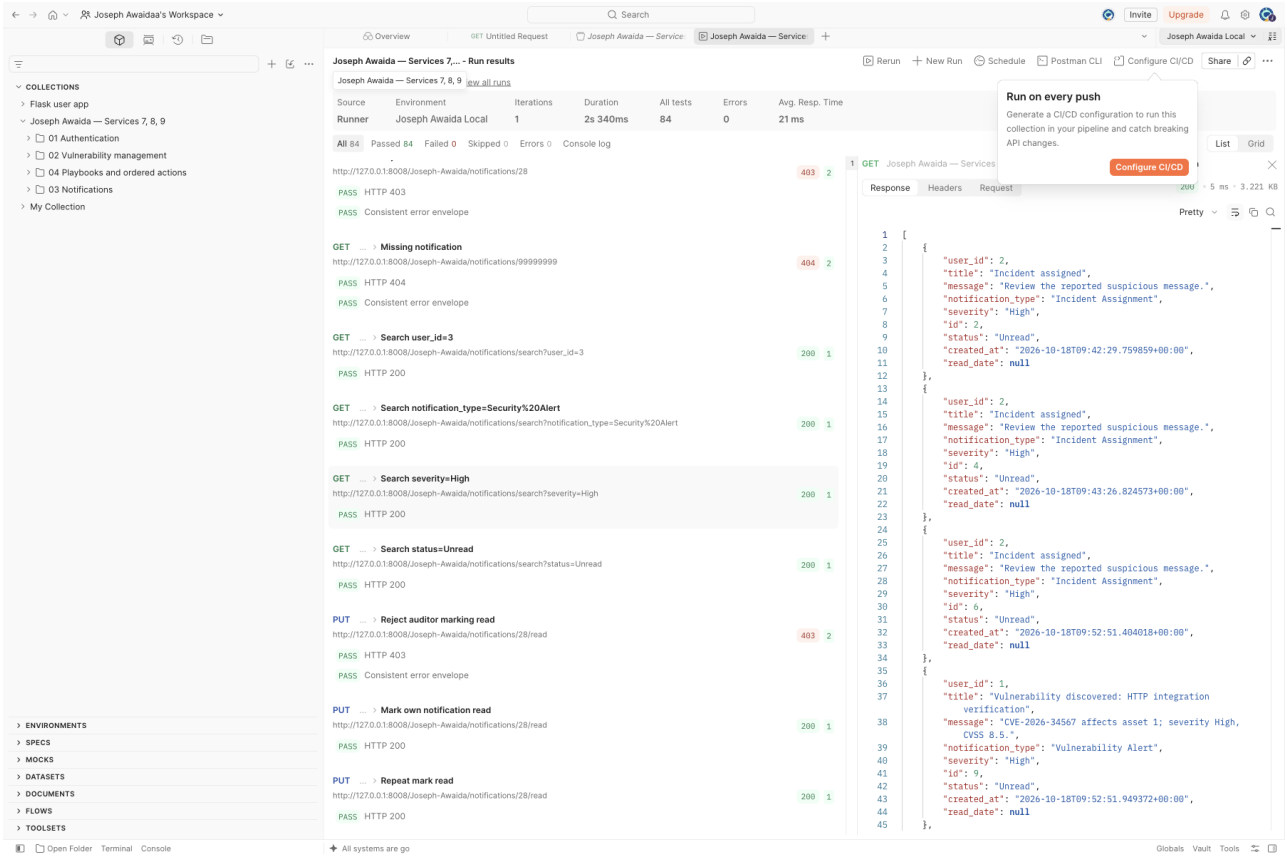
Joseph Awaida — Missing notification: actual GET result and HTTP status.



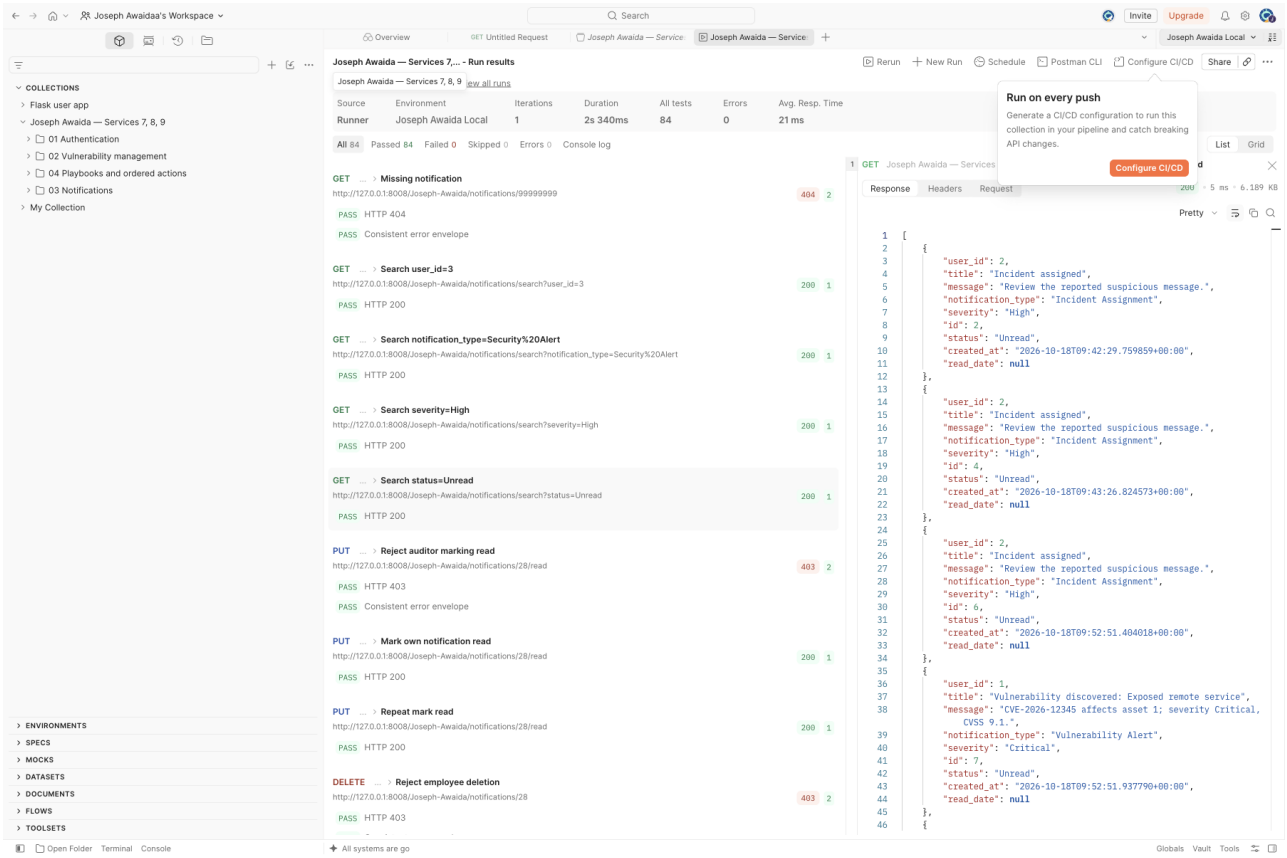
Joseph Awaida — Search user_id=3: actual GET result and HTTP status.



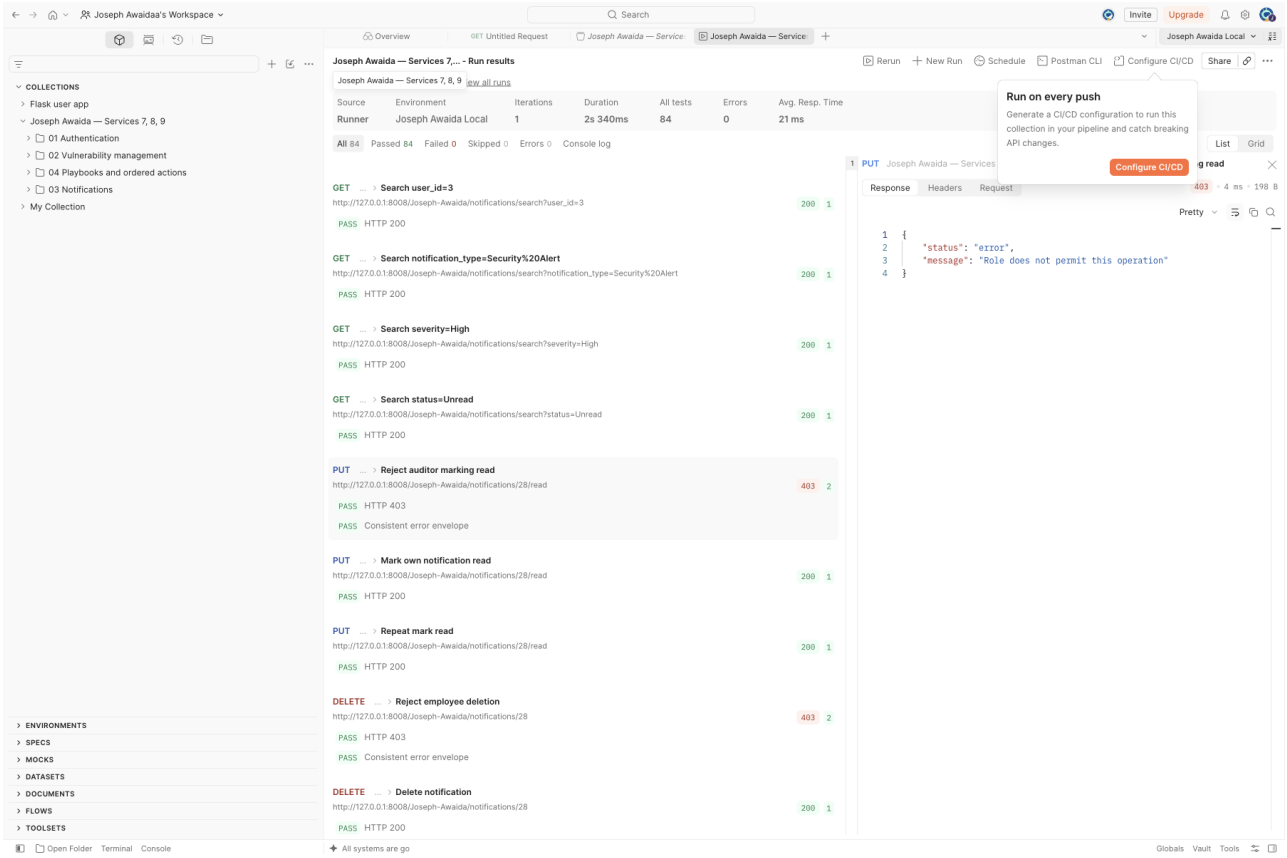
Joseph Awaida — Search notification_type=Security%20Alert: actual GET result and HTTP status.



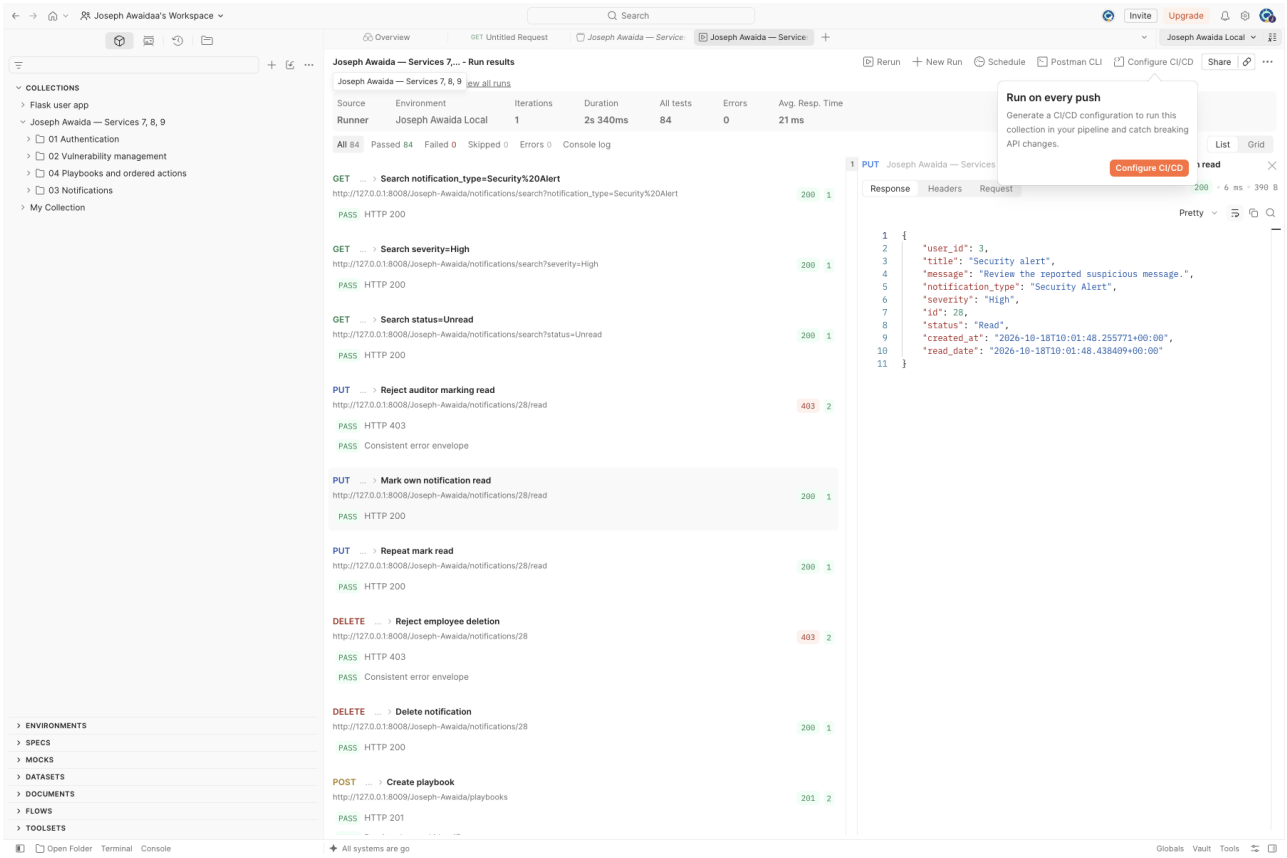
Joseph Awaida — Search severity=High: actual GET result and HTTP status.



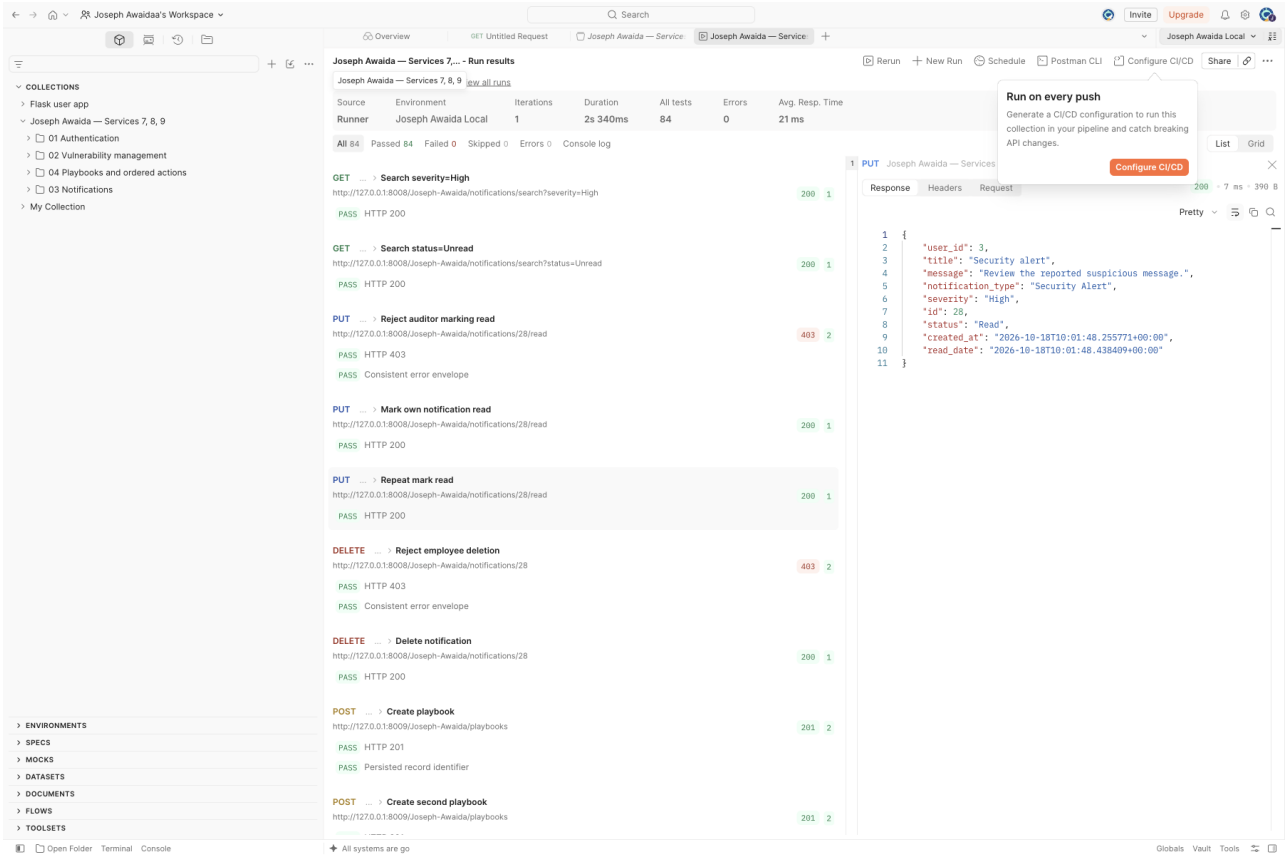
Joseph Awaida — Search status=Unread: actual GET result and HTTP status.



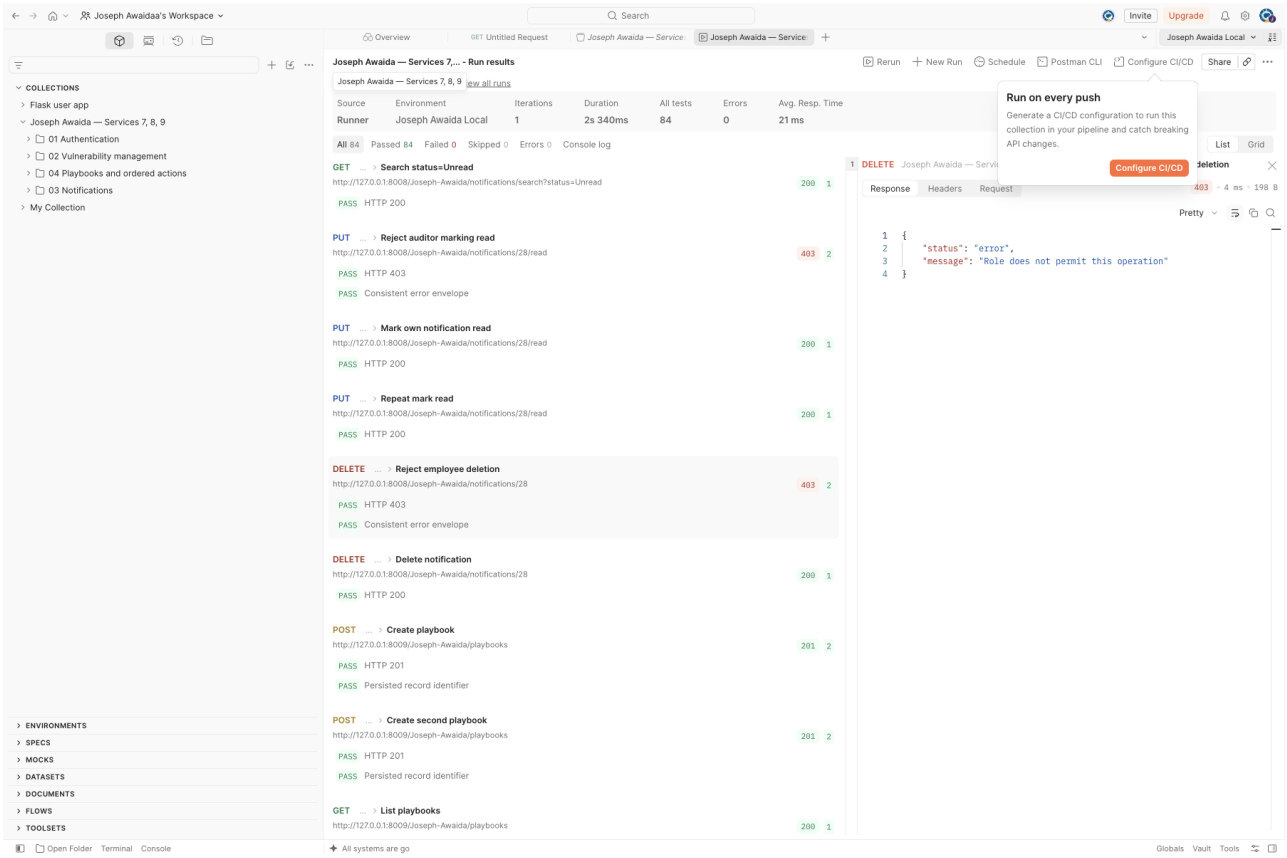
Joseph Awaida — Reject auditor marking read: actual PUT result and HTTP status.



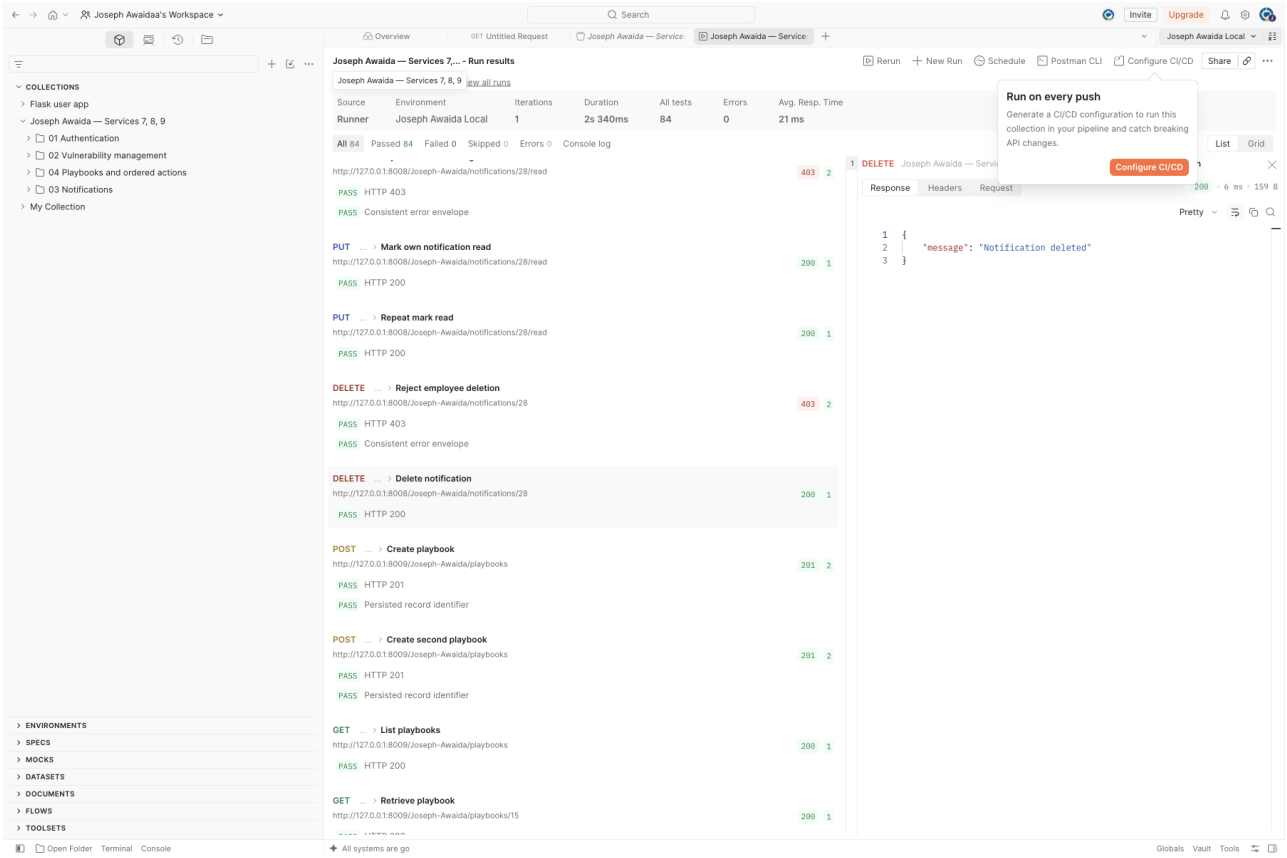
Joseph Awaida — Mark own notification read: actual PUT result and HTTP status.



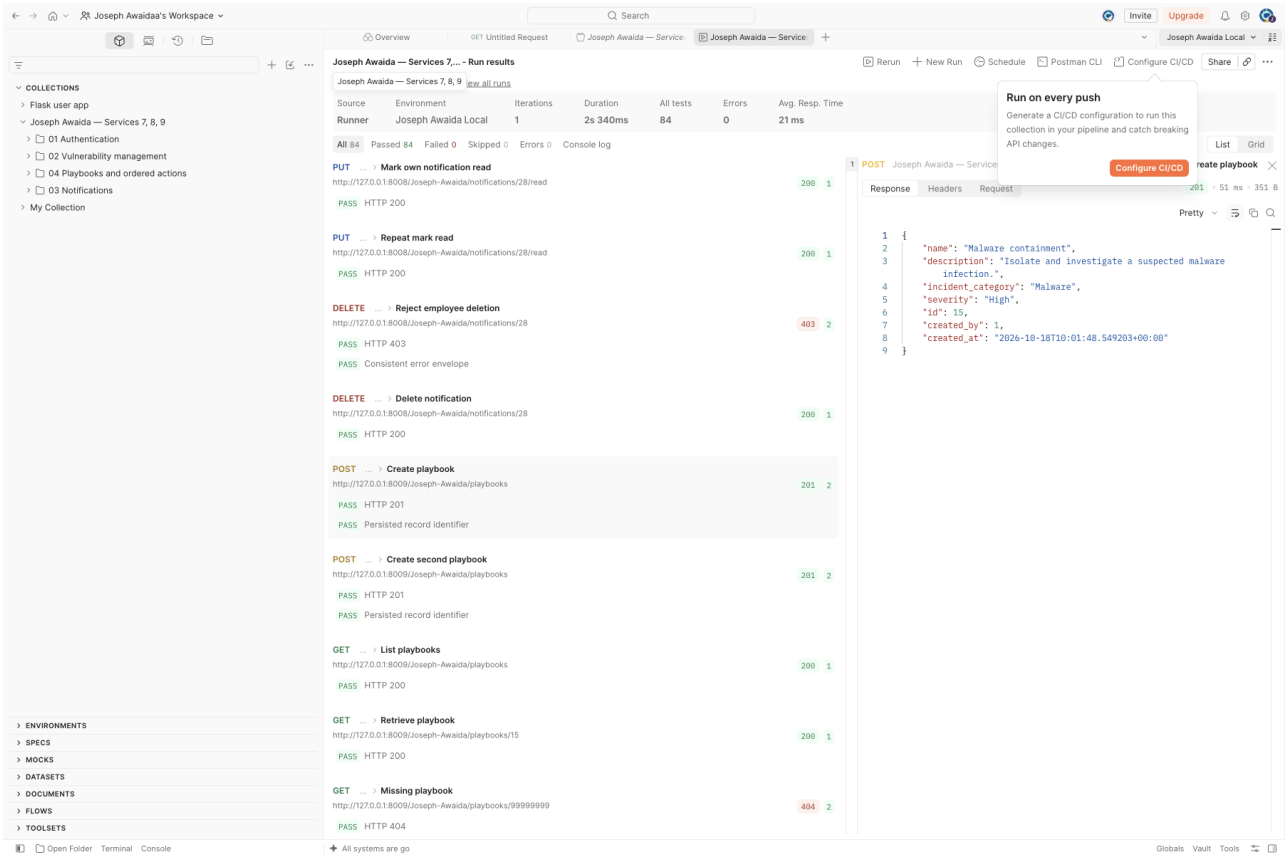
Joseph Awaida — Repeat mark read: actual PUT result and HTTP status.



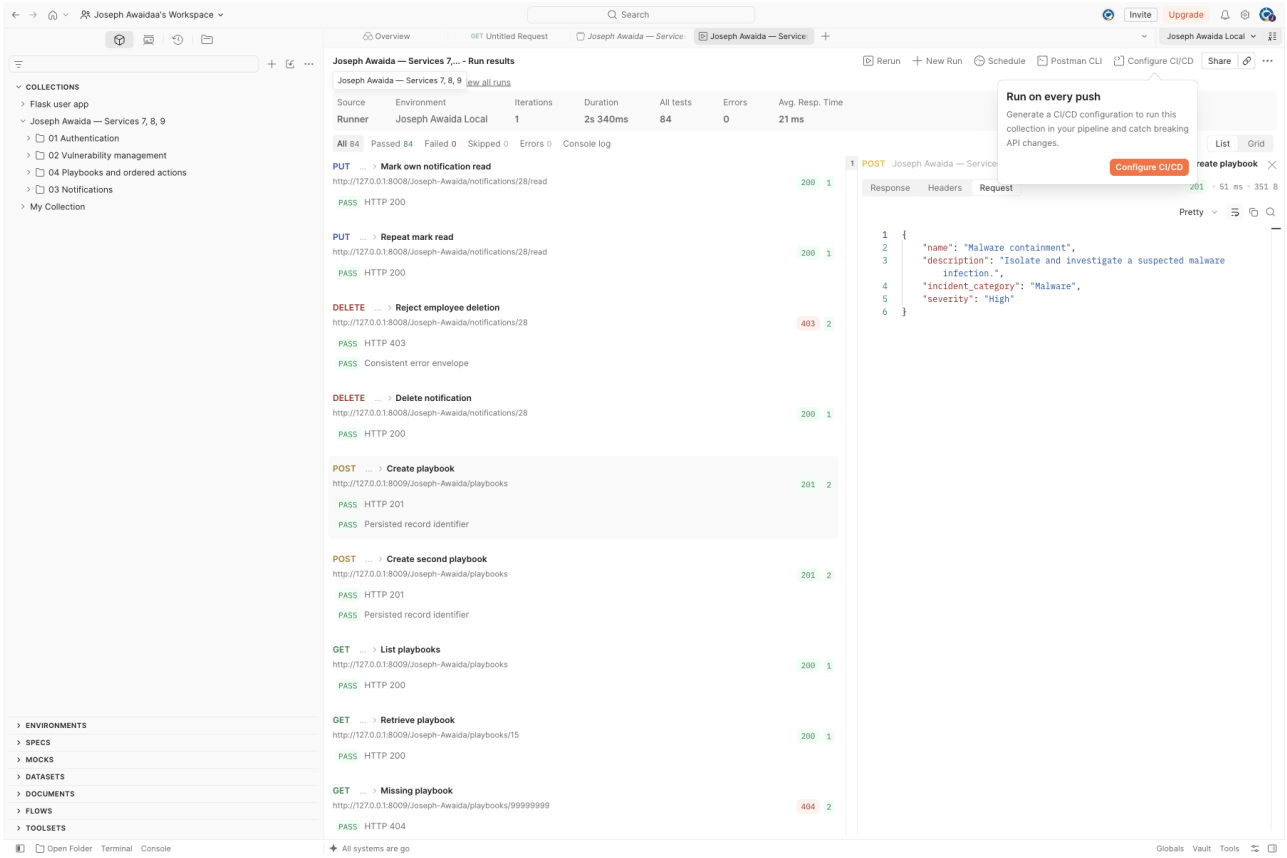
Joseph Awaida — Reject employee deletion: actual DELETE result and HTTP status.



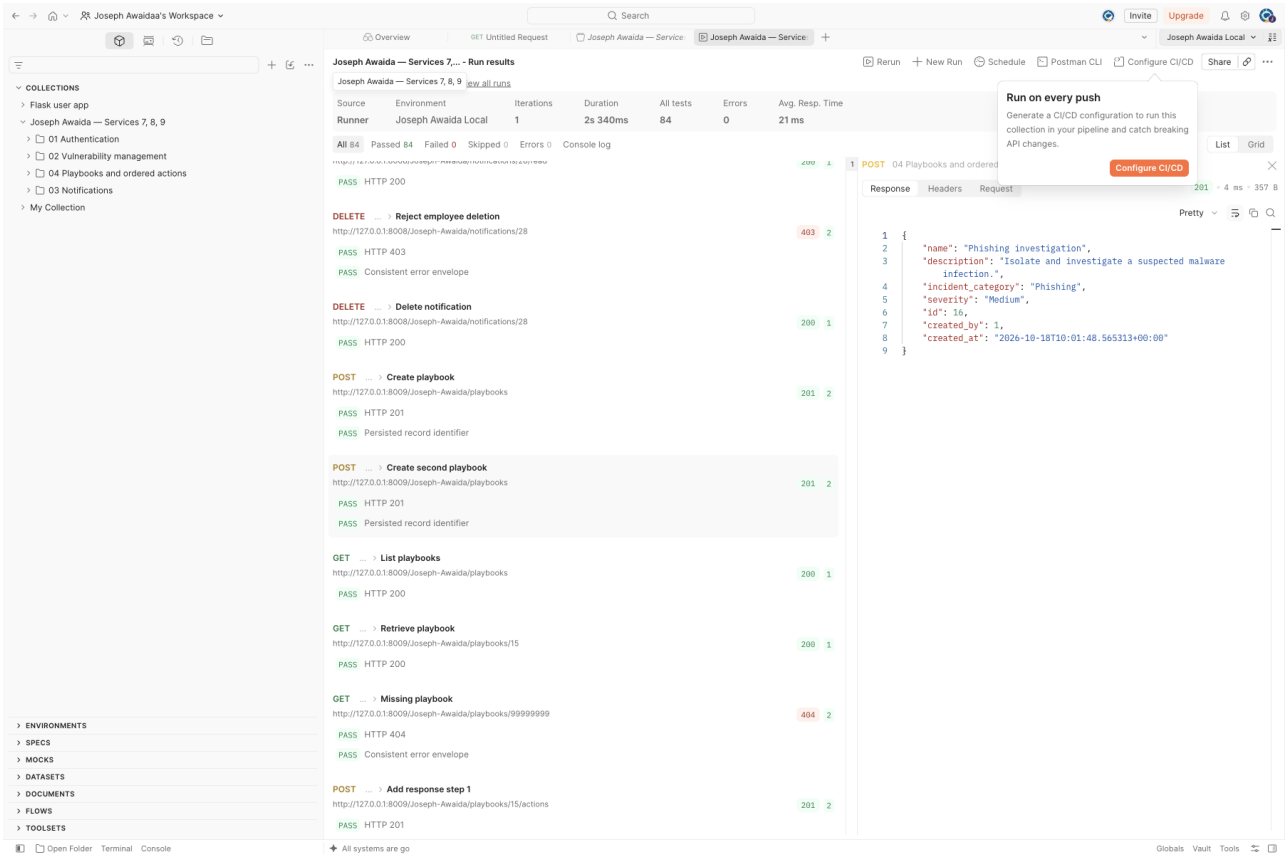
Joseph Awaida — Delete notification: actual DELETE result and HTTP status.



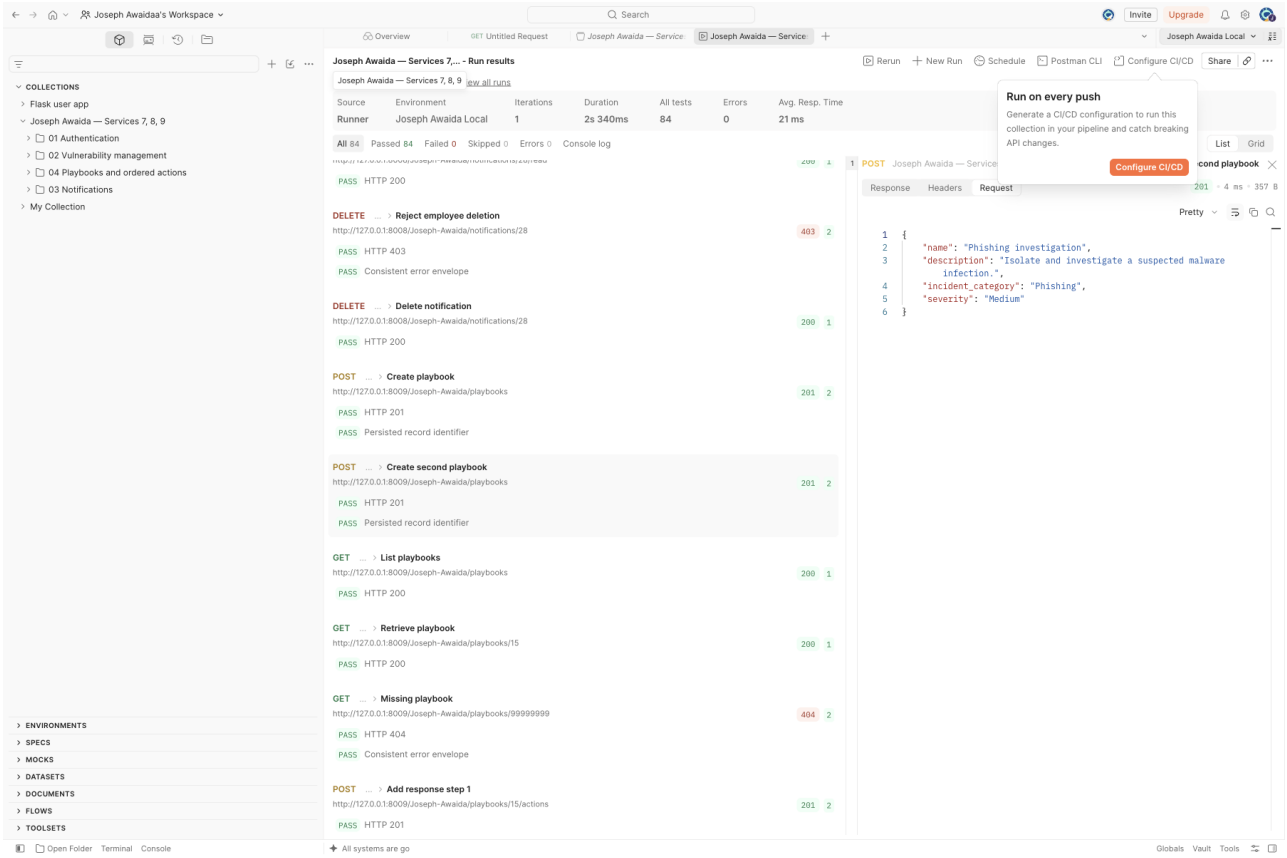
Joseph Awaida — Create playbook: actual POST result and HTTP status.



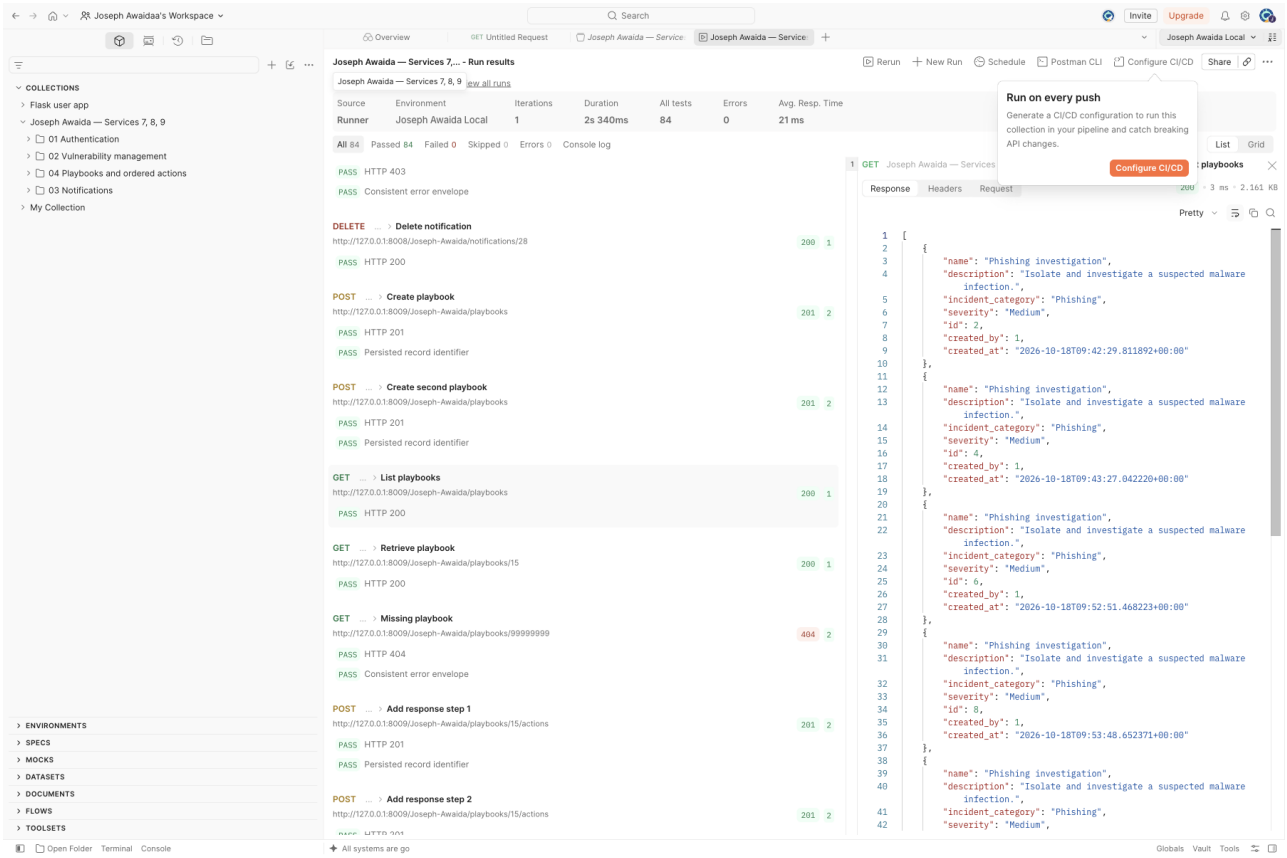
Joseph Awaida — Create playbook: actual request body shown in the Postman runner.



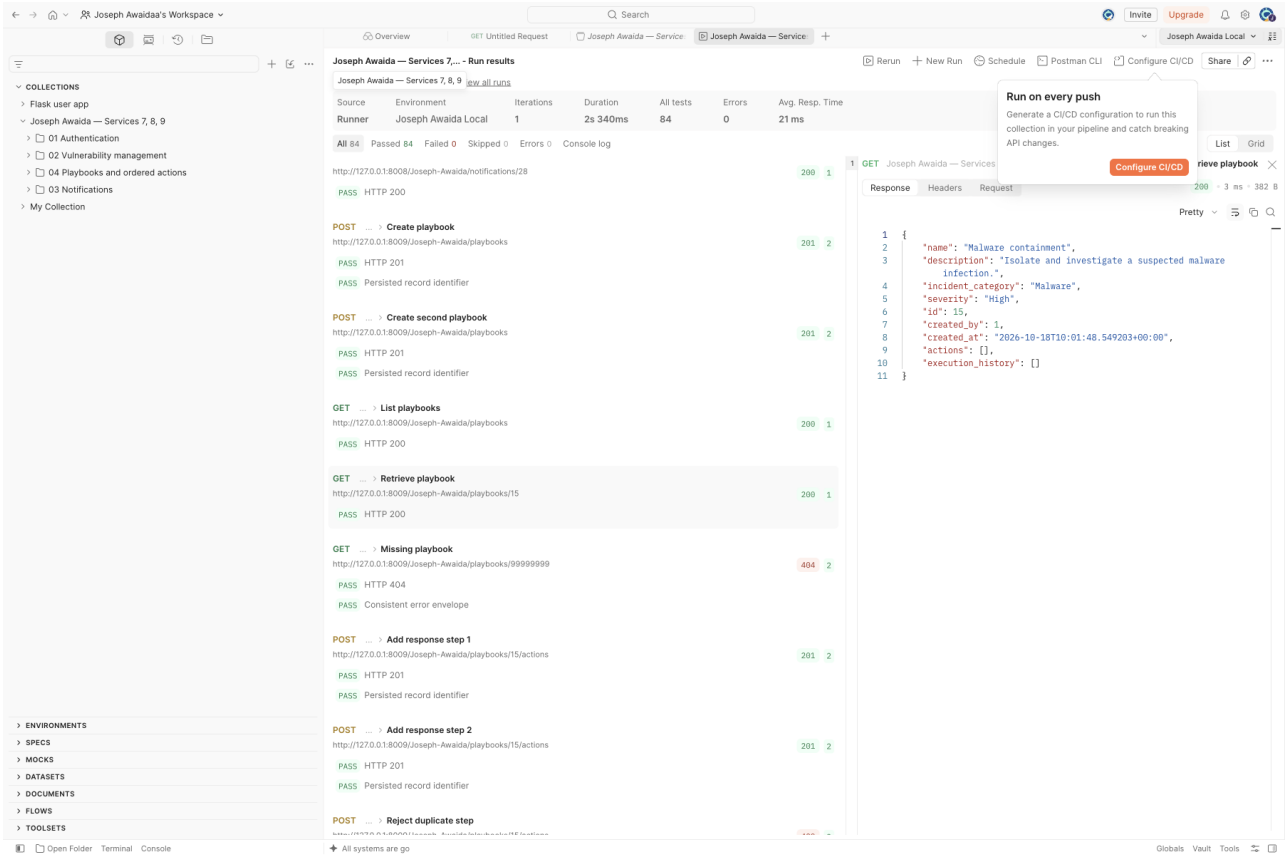
Joseph Awaida — Create second playbook: actual POST result and HTTP status.



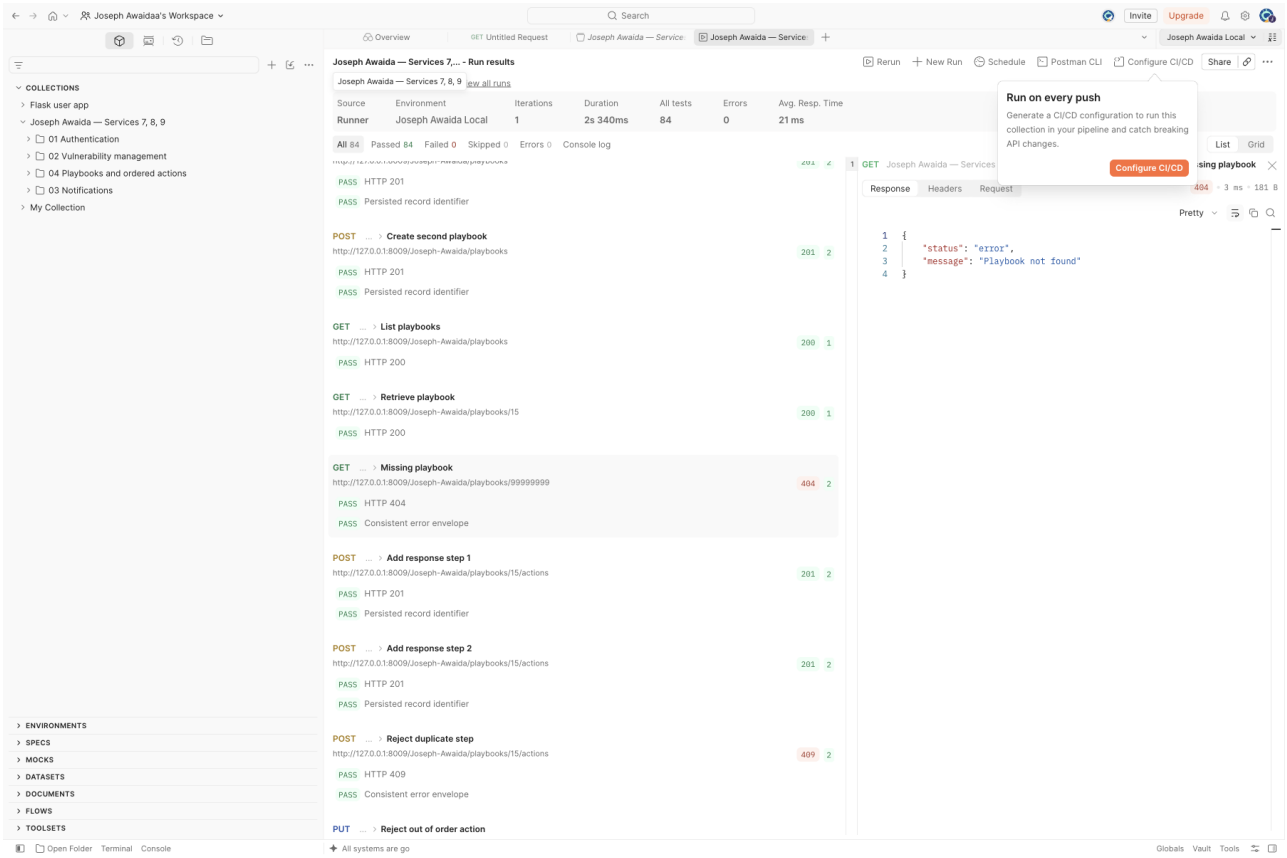
Joseph Awaida — Create second playbook: actual request body shown in the Postman runner.



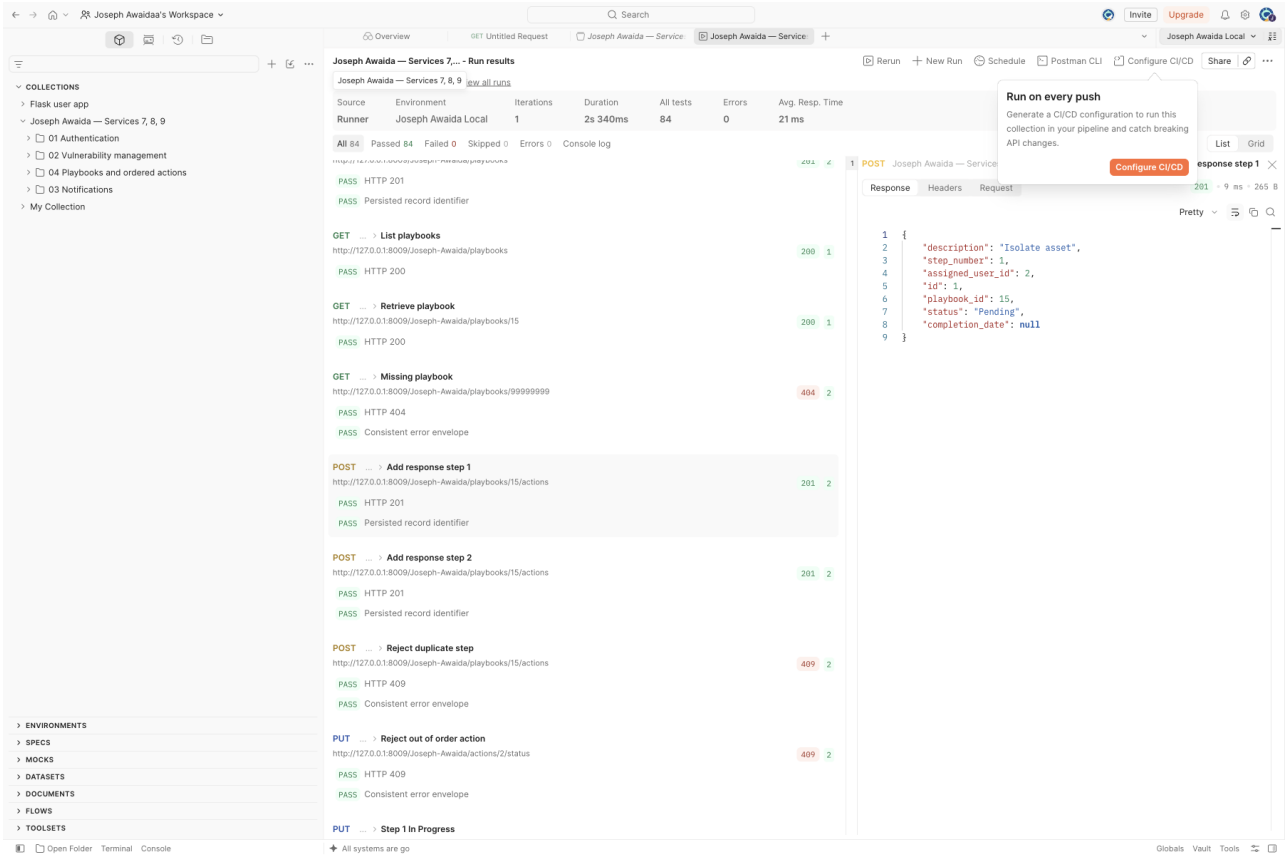
Joseph Awaida — List playbooks: actual GET result and HTTP status.



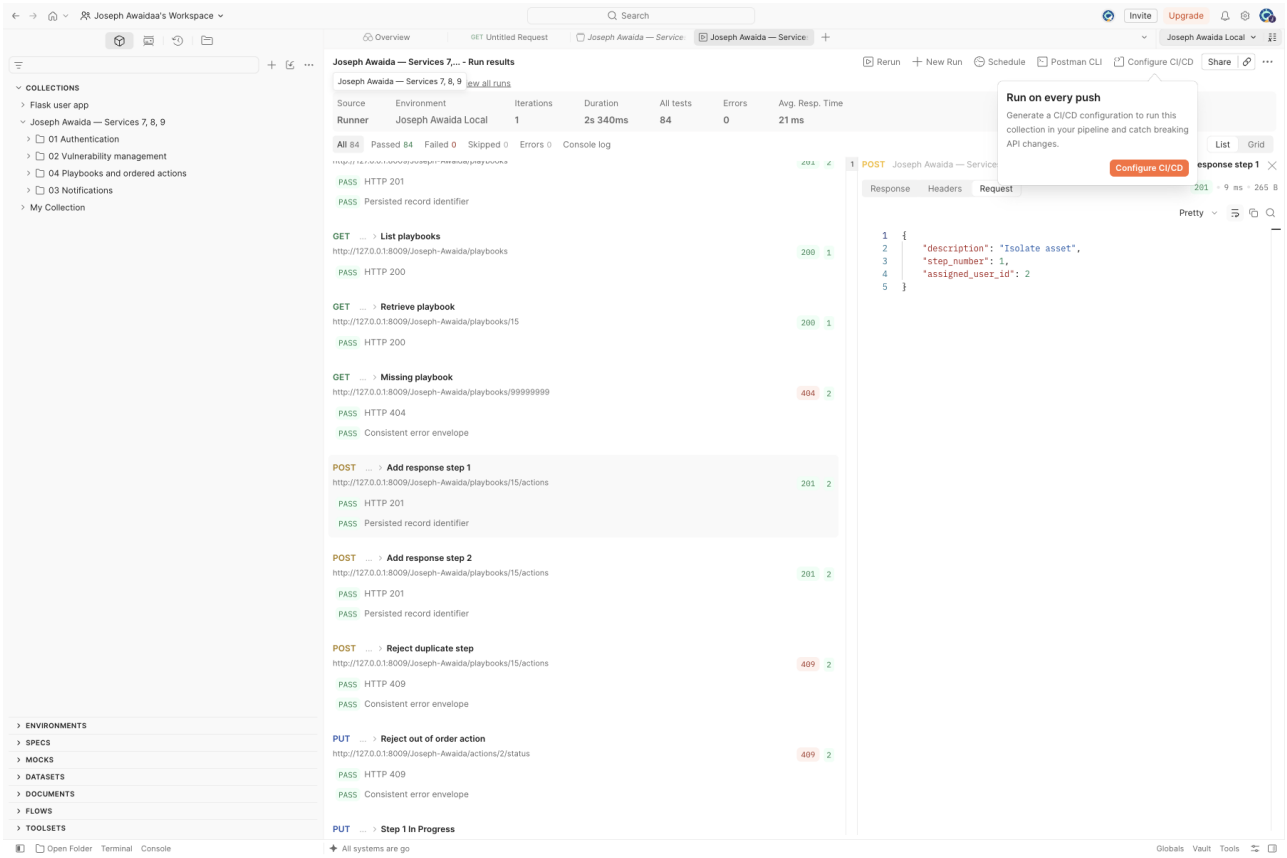
Joseph Awaida — Retrieve playbook: actual GET result and HTTP status.



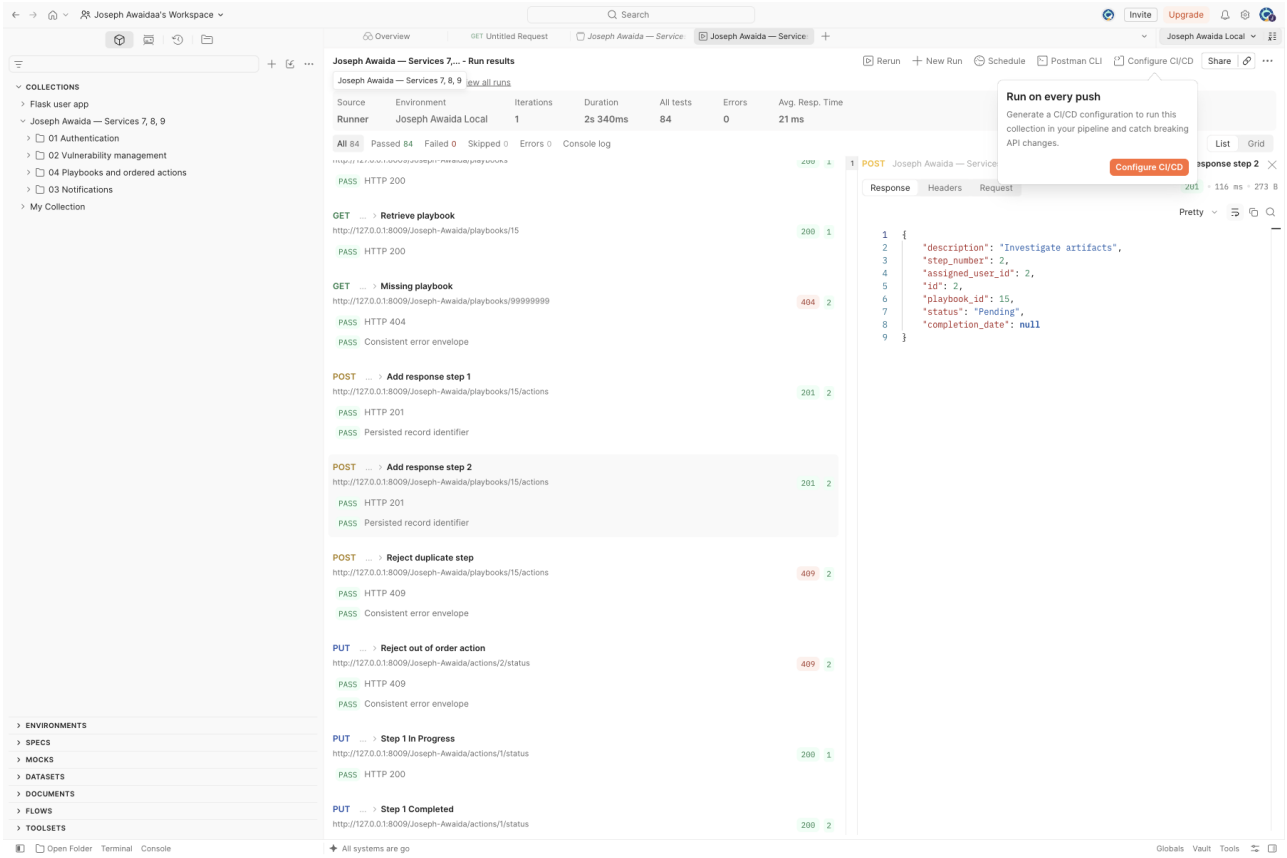
Joseph Awaida — Missing playbook: actual GET result and HTTP status.



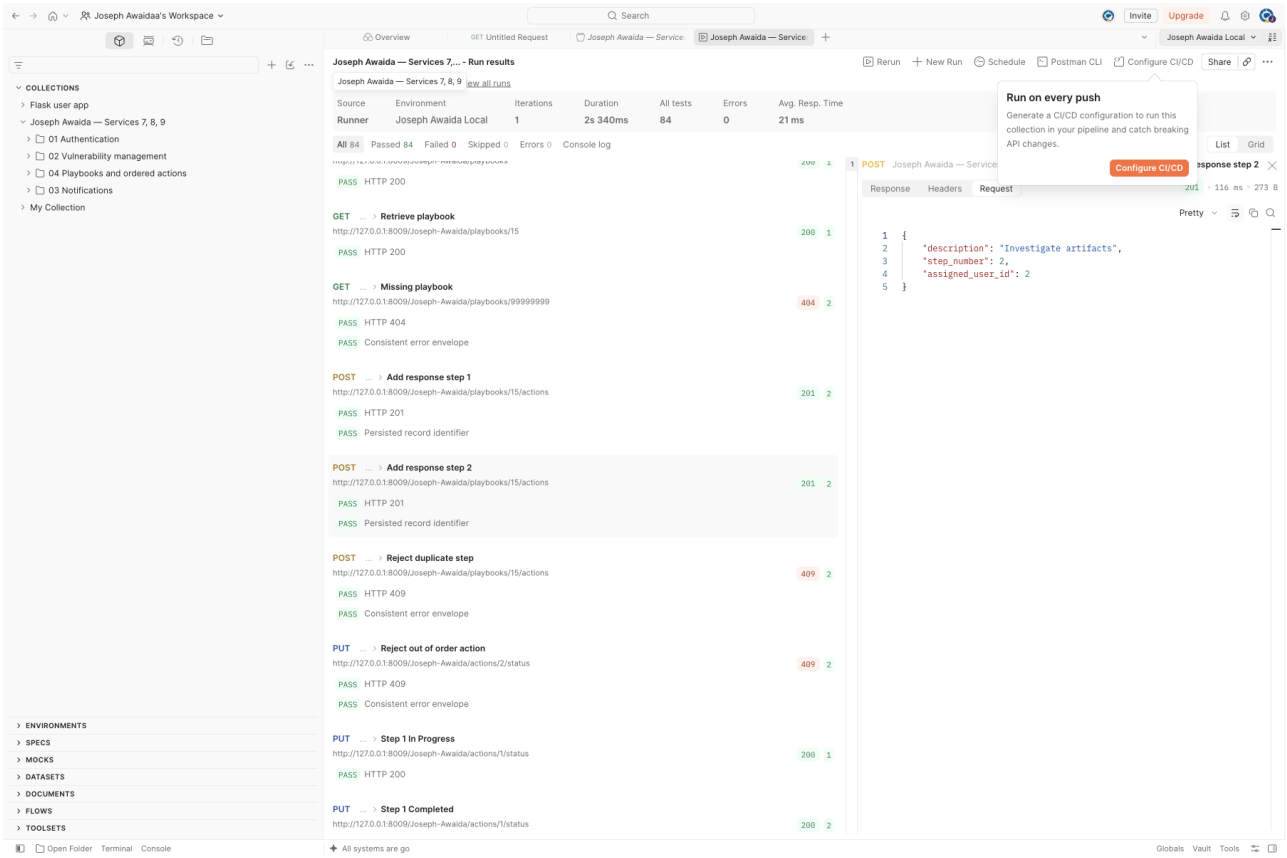
Joseph Awaida — Add response step 1: actual POST result and HTTP status.



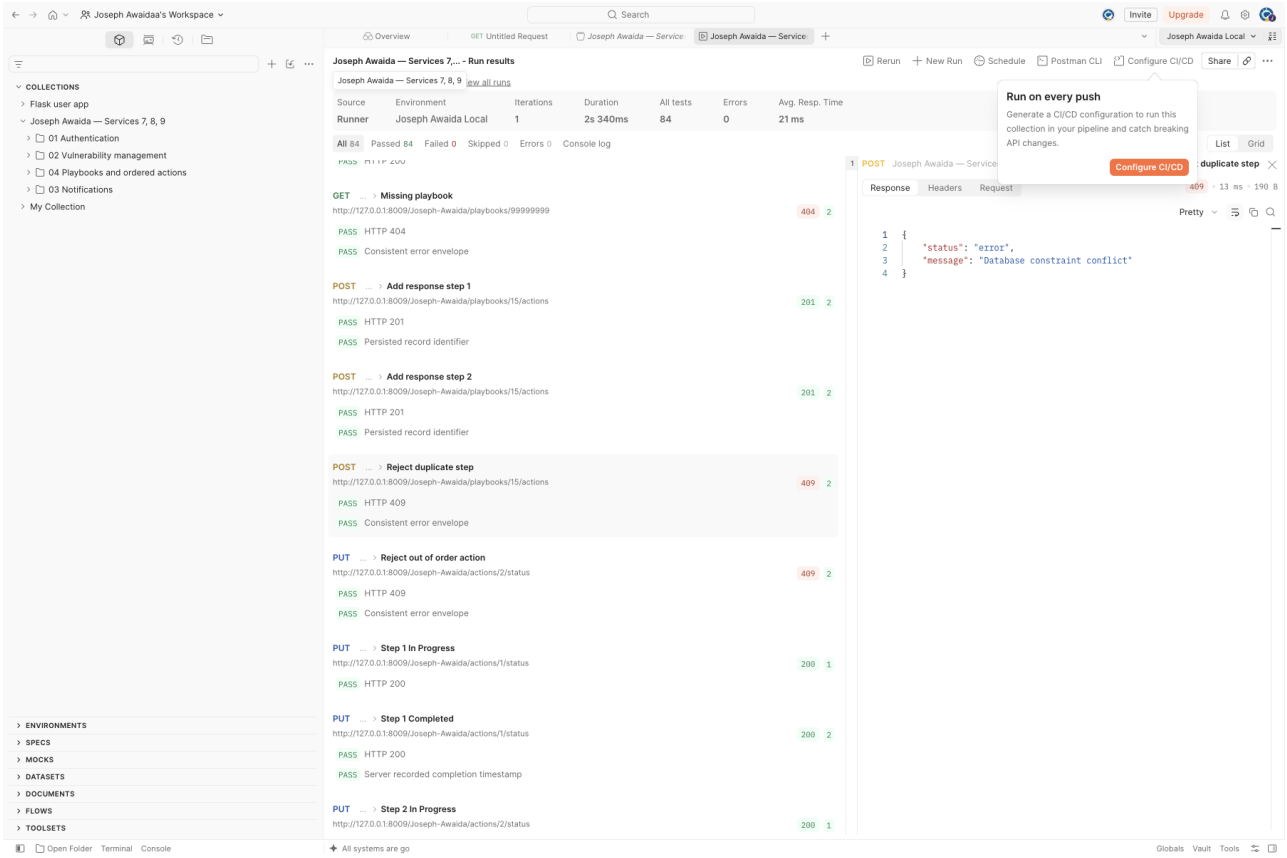
Joseph Awaida — Add response step 1: actual request body shown in the Postman runner.



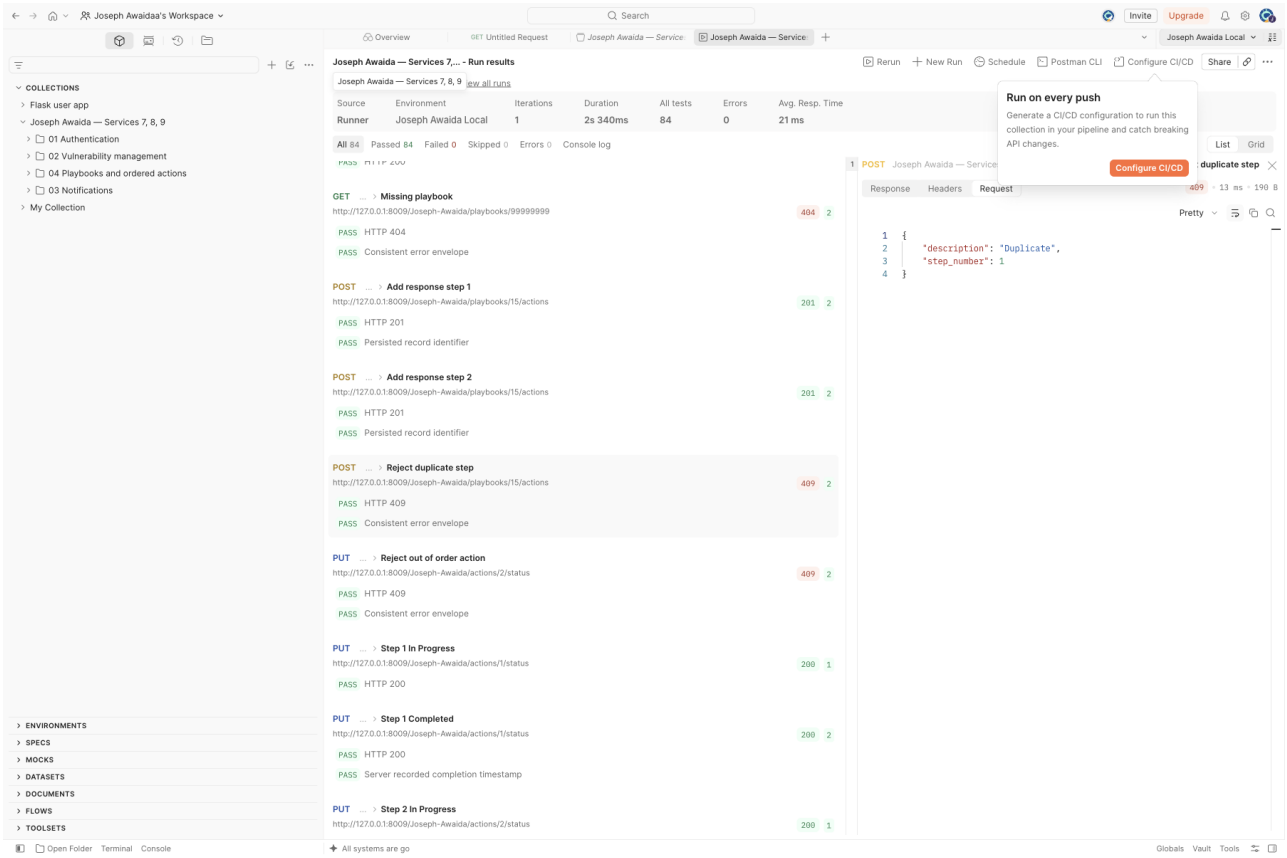
Joseph Awaida — Add response step 2: actual POST result and HTTP status.



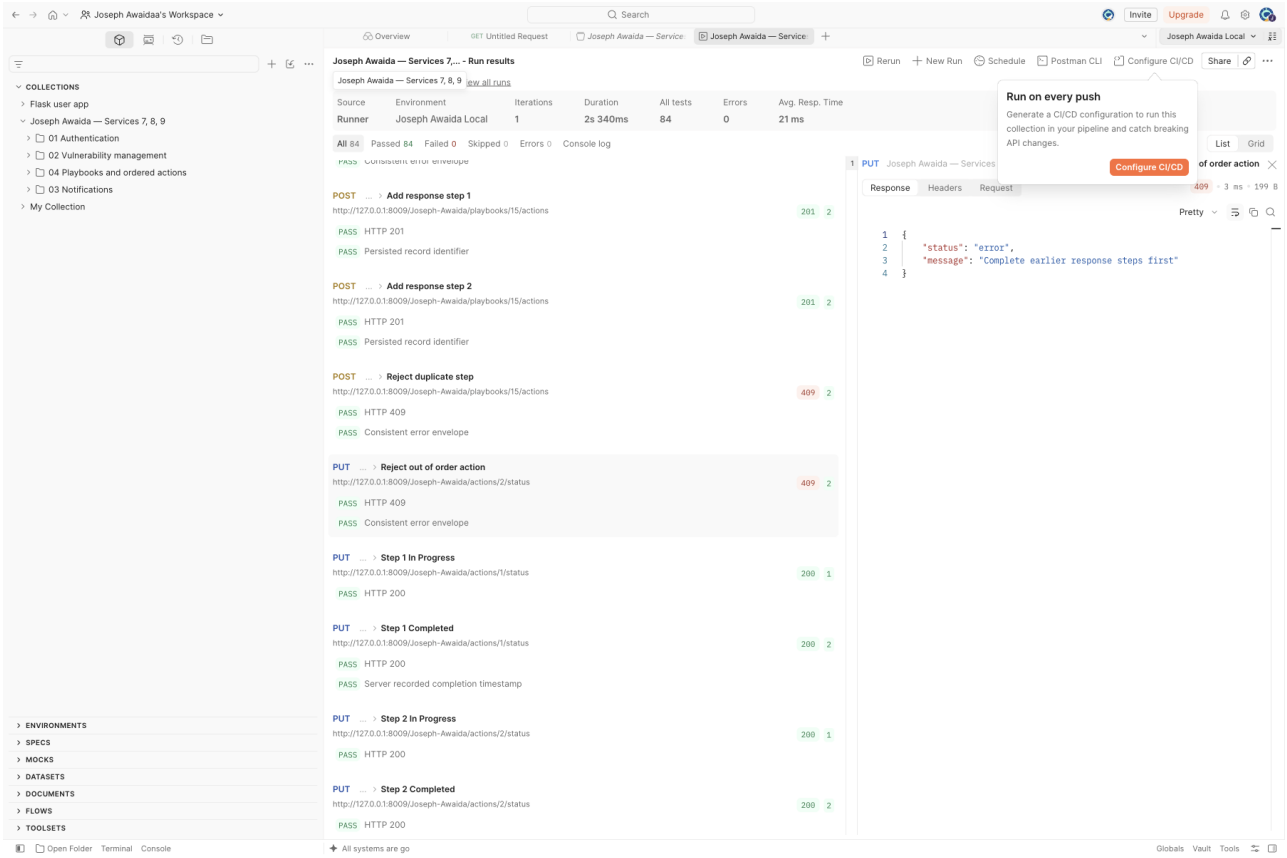
Joseph Awaida — Add response step 2: actual request body shown in the Postman runner.



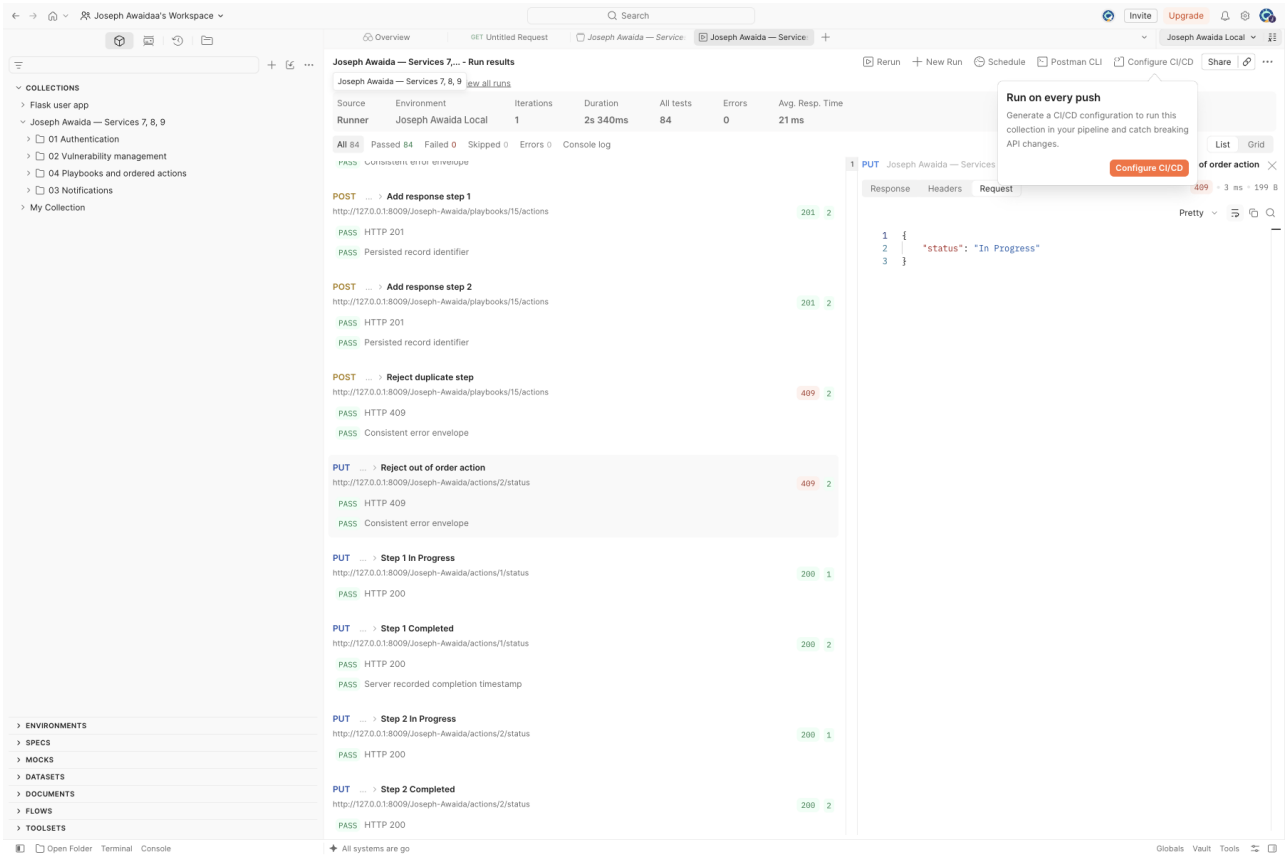
Joseph Awaida — Reject duplicate step: actual POST result and HTTP status.



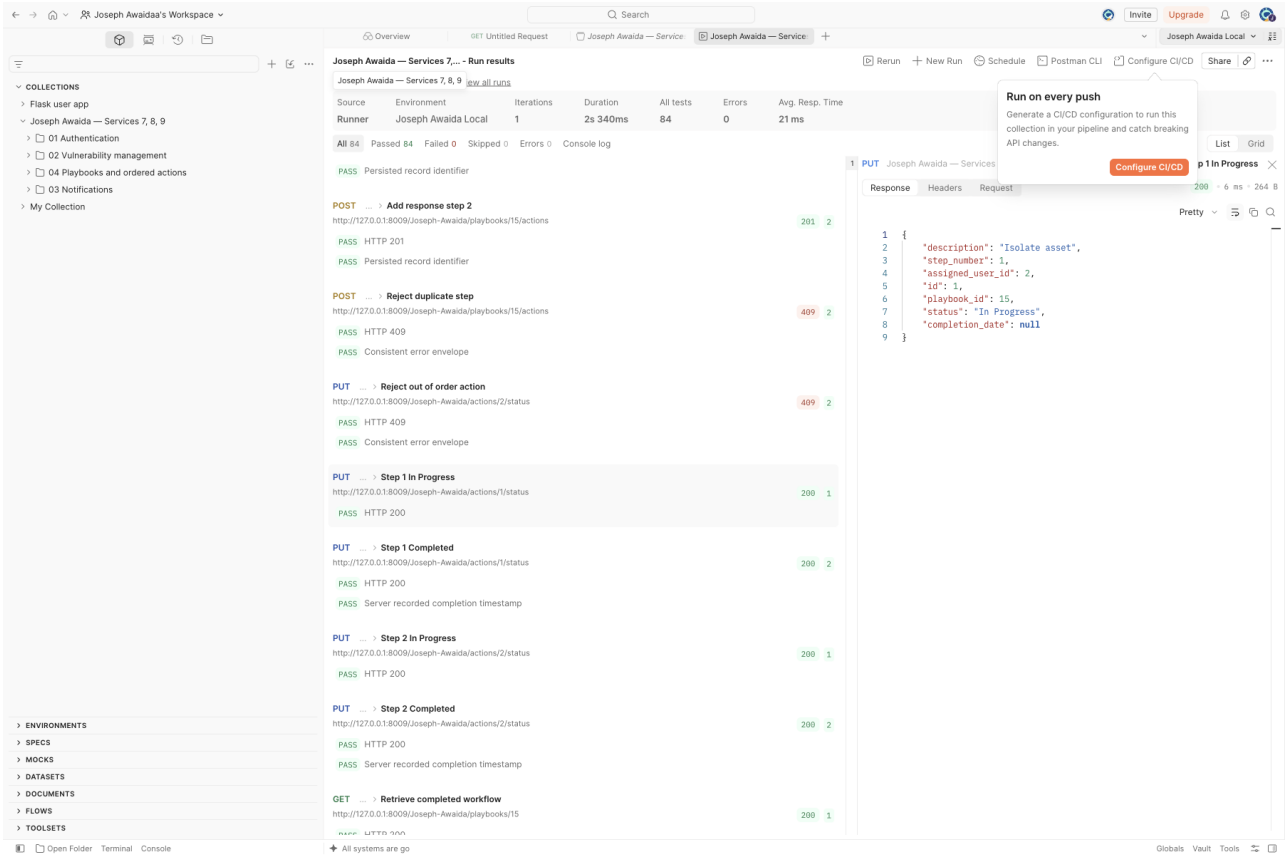
Joseph Awaida — Reject duplicate step: actual request body shown in the Postman runner.



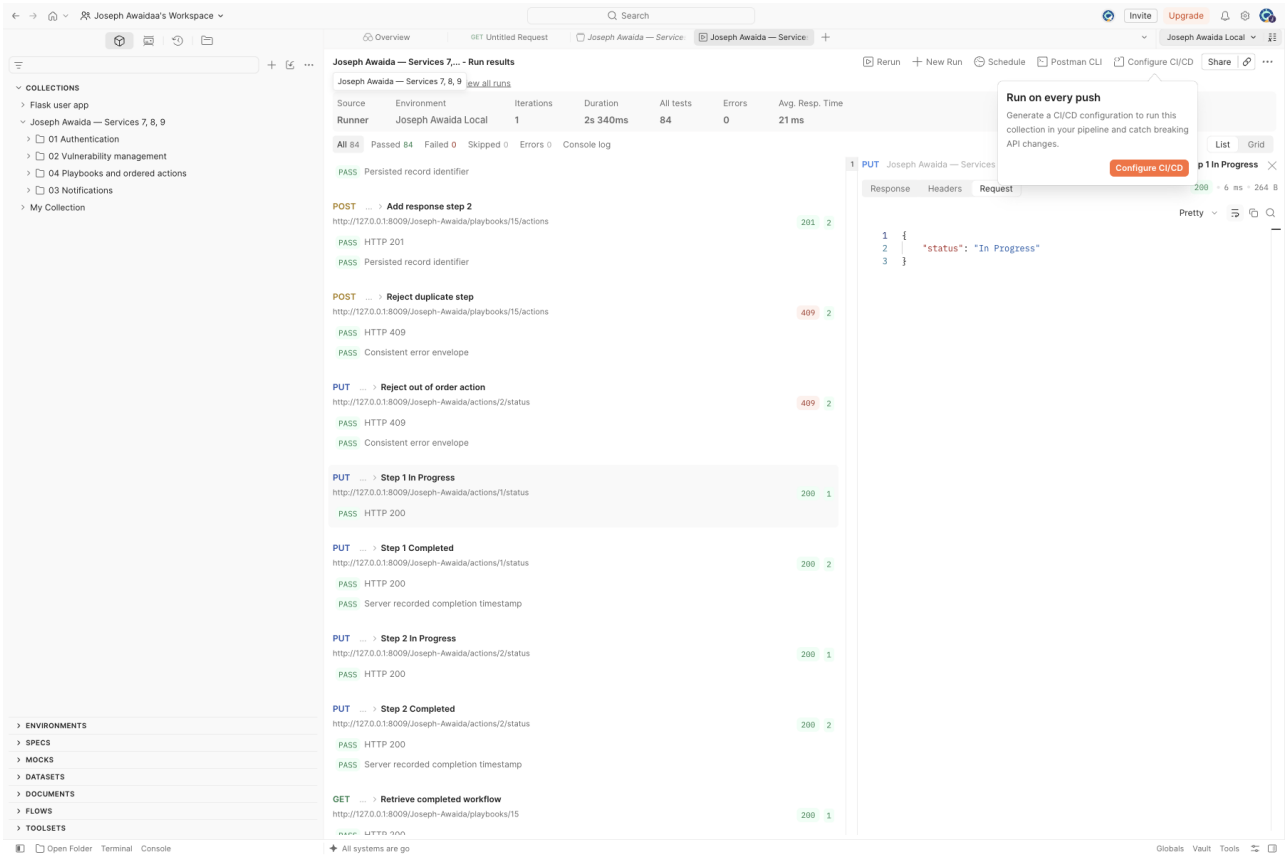
Joseph Awaida — Reject out of order action: actual PUT result and HTTP status.

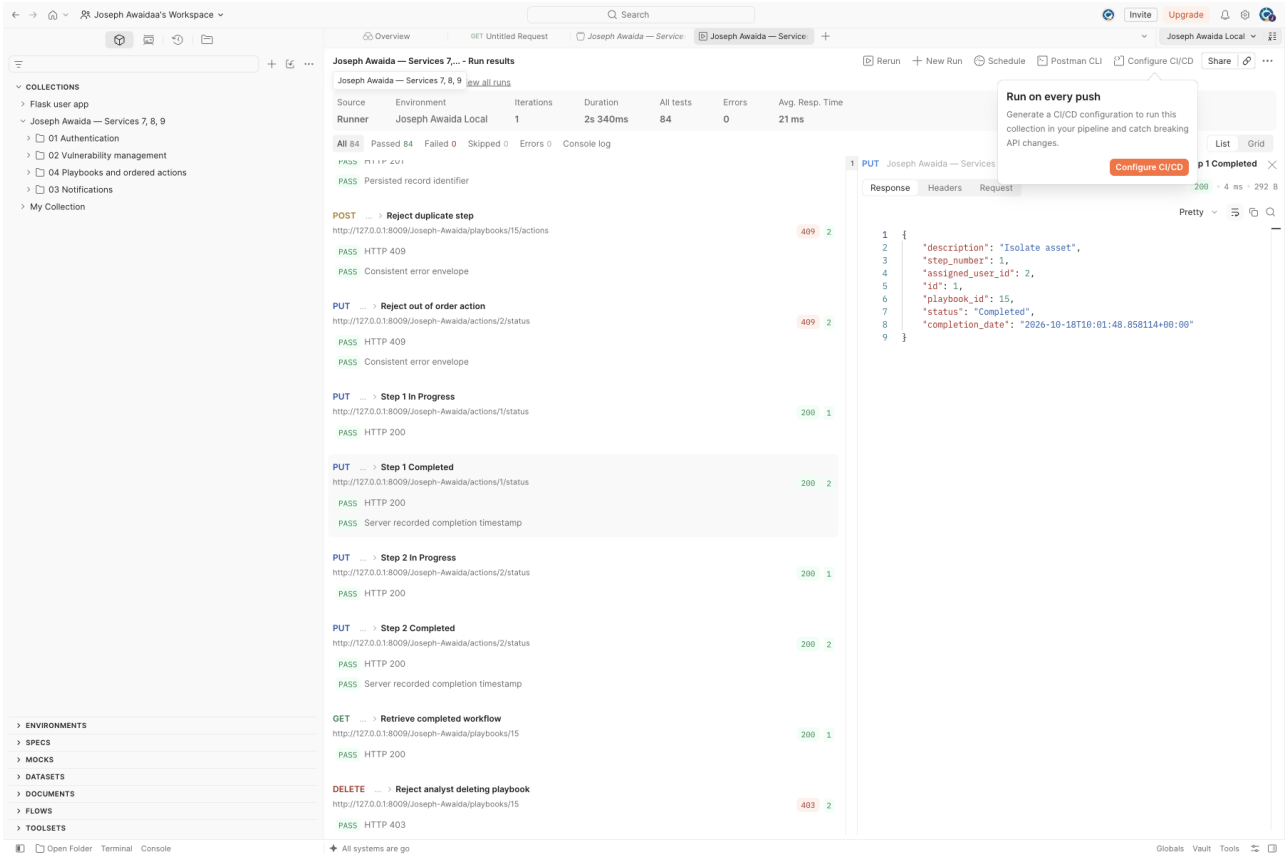


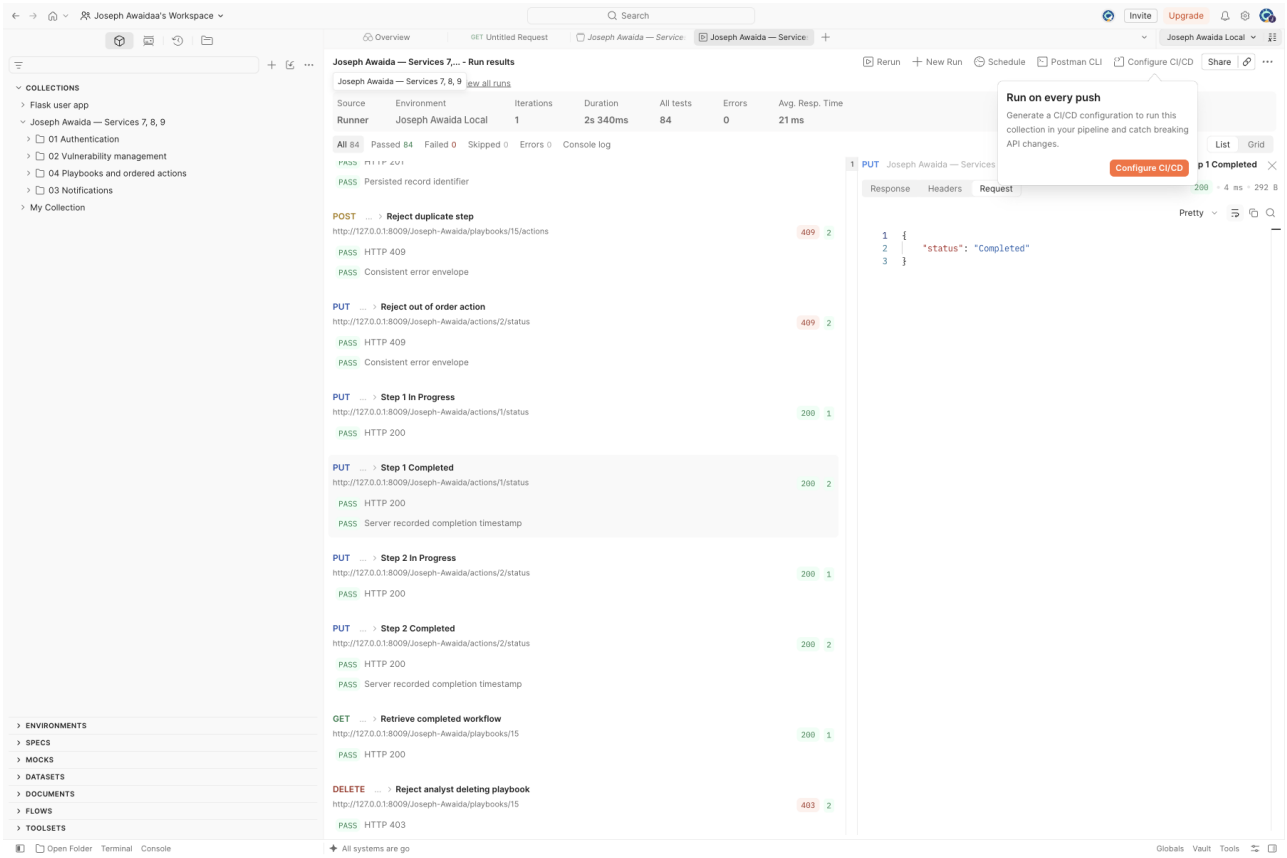
Joseph Awaida — Reject out of order action: actual request body shown in the Postman runner.

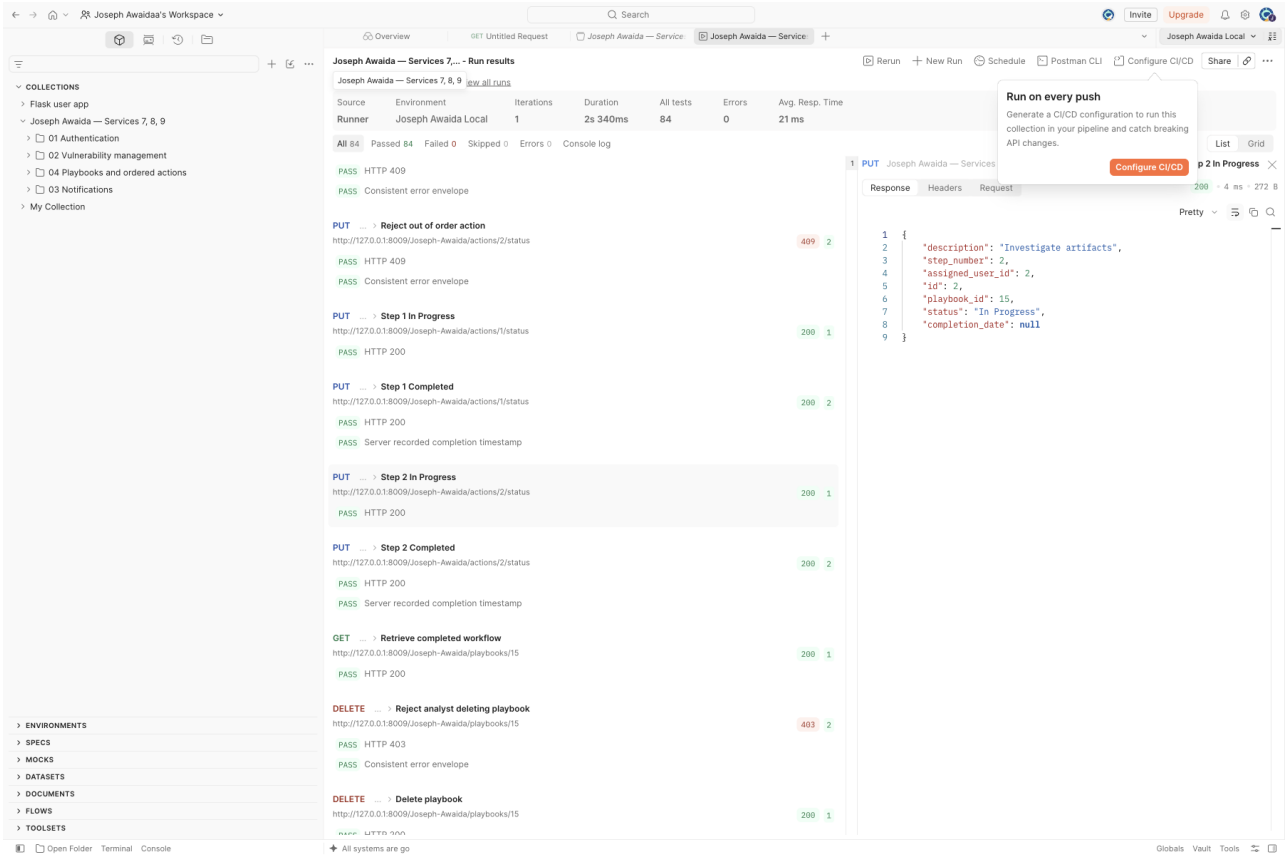


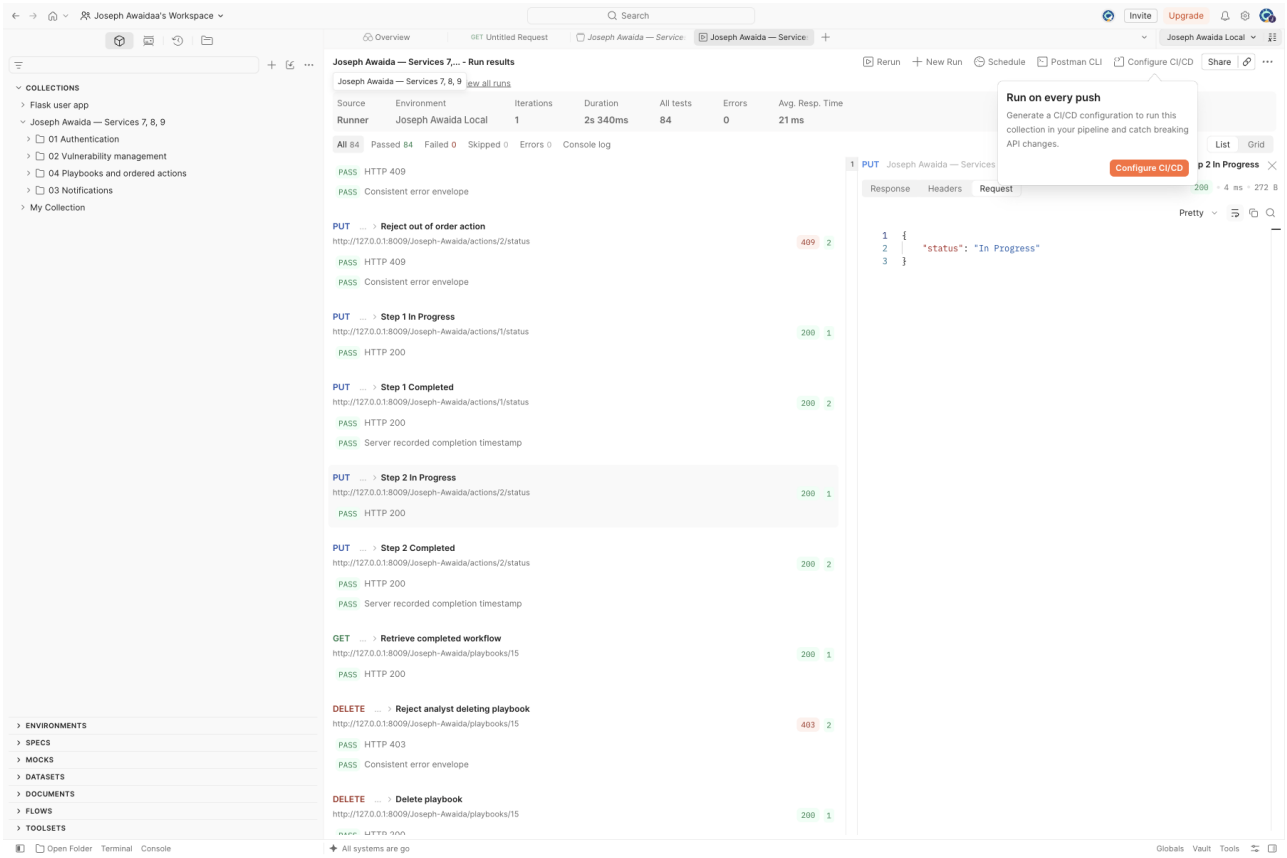
Joseph Awaida — Step 1 In Progress: actual PUT result and HTTP status.

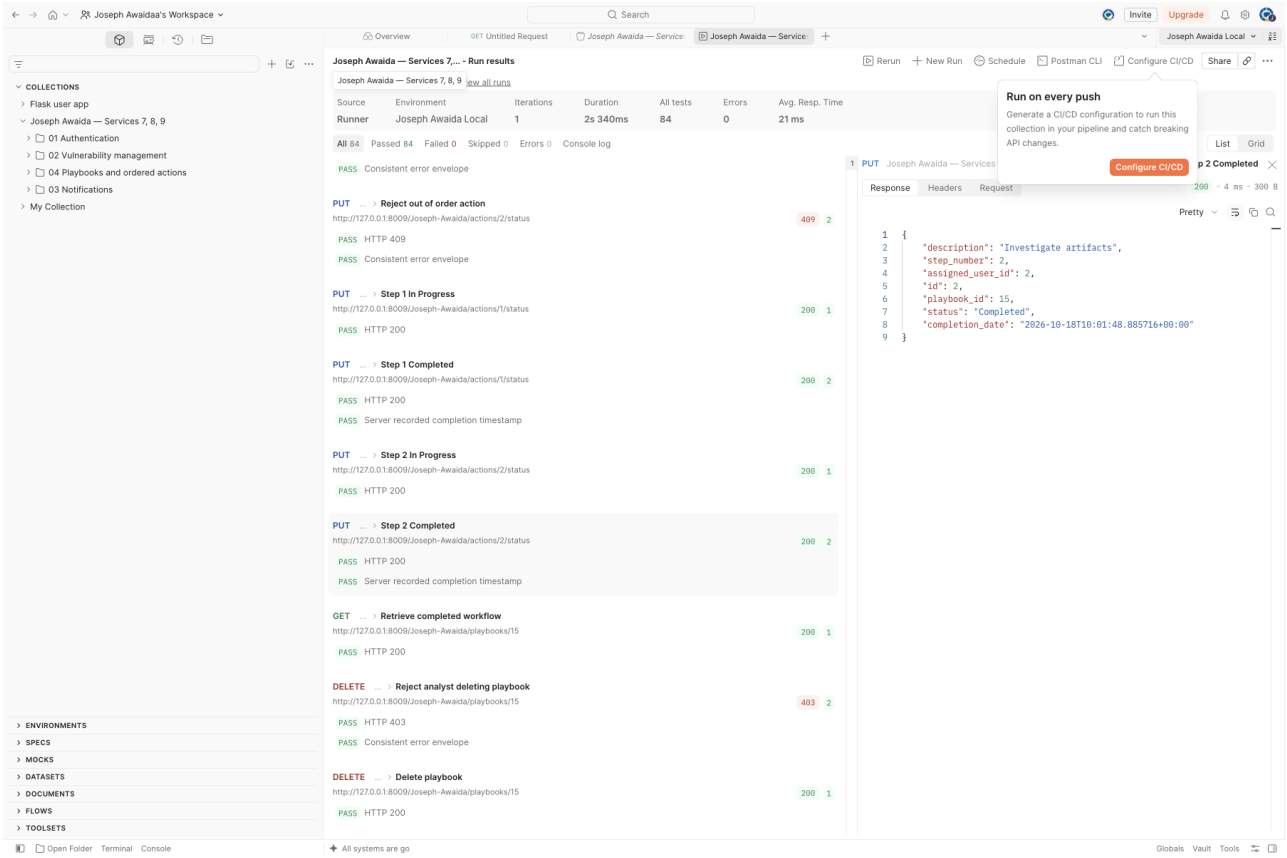

Joseph Awaida — Step 1 In Progress: actual request body shown in the Postman runner.

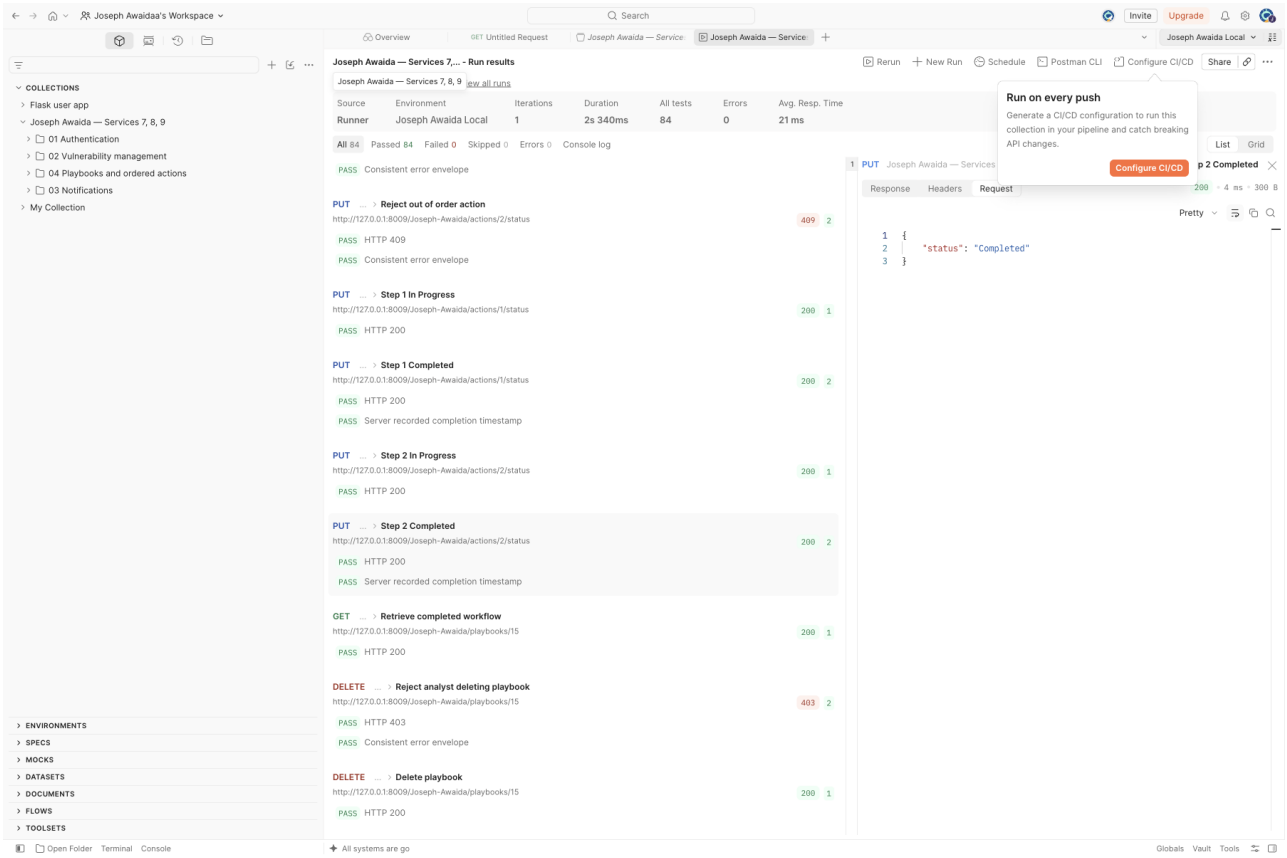

Joseph Awaida — Step 1 Completed: actual PUT result and HTTP status.


Joseph Awaida — Step 1 Completed: actual request body shown in the Postman runner.

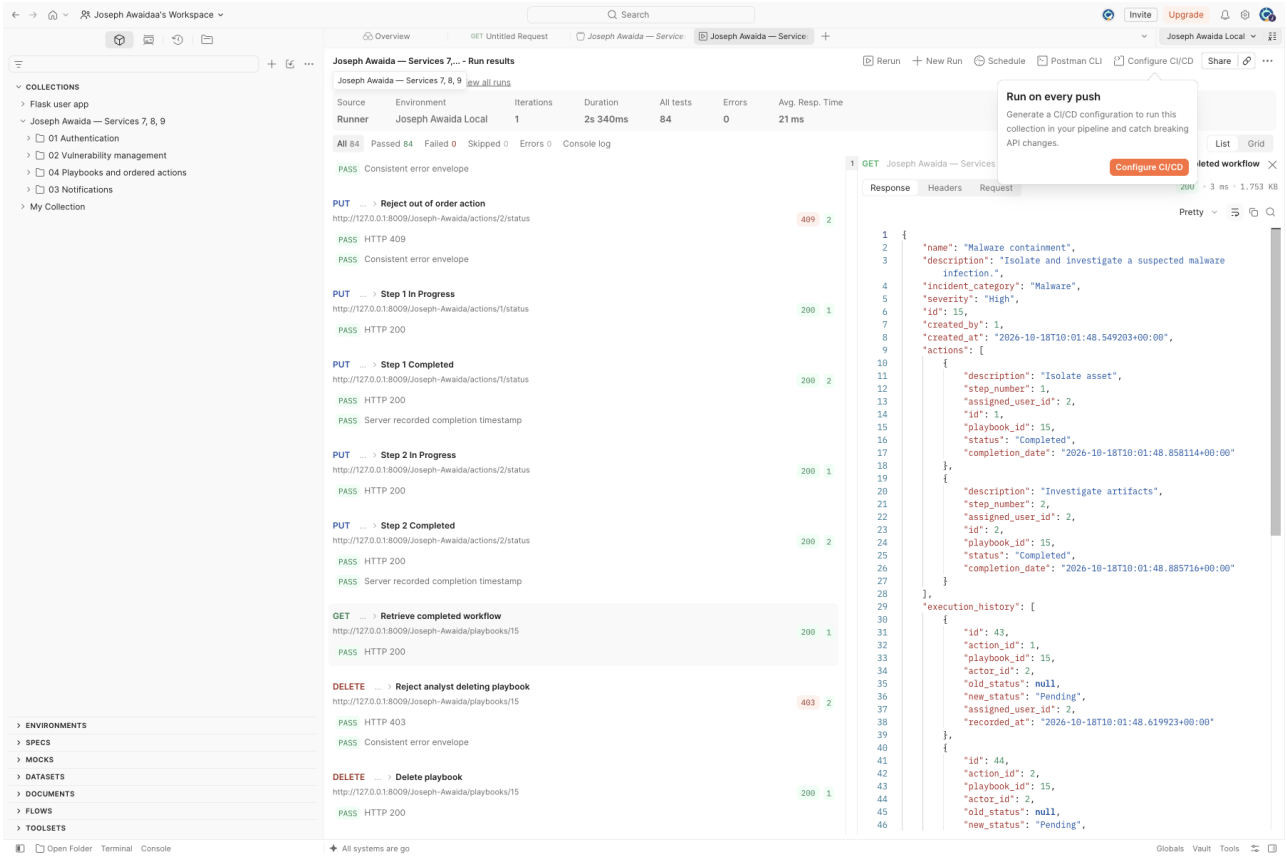

Joseph Awaida — Step 2 In Progress: actual PUT result and HTTP status.


Joseph Awaida — Step 2 In Progress: actual request body shown in the Postman runner.

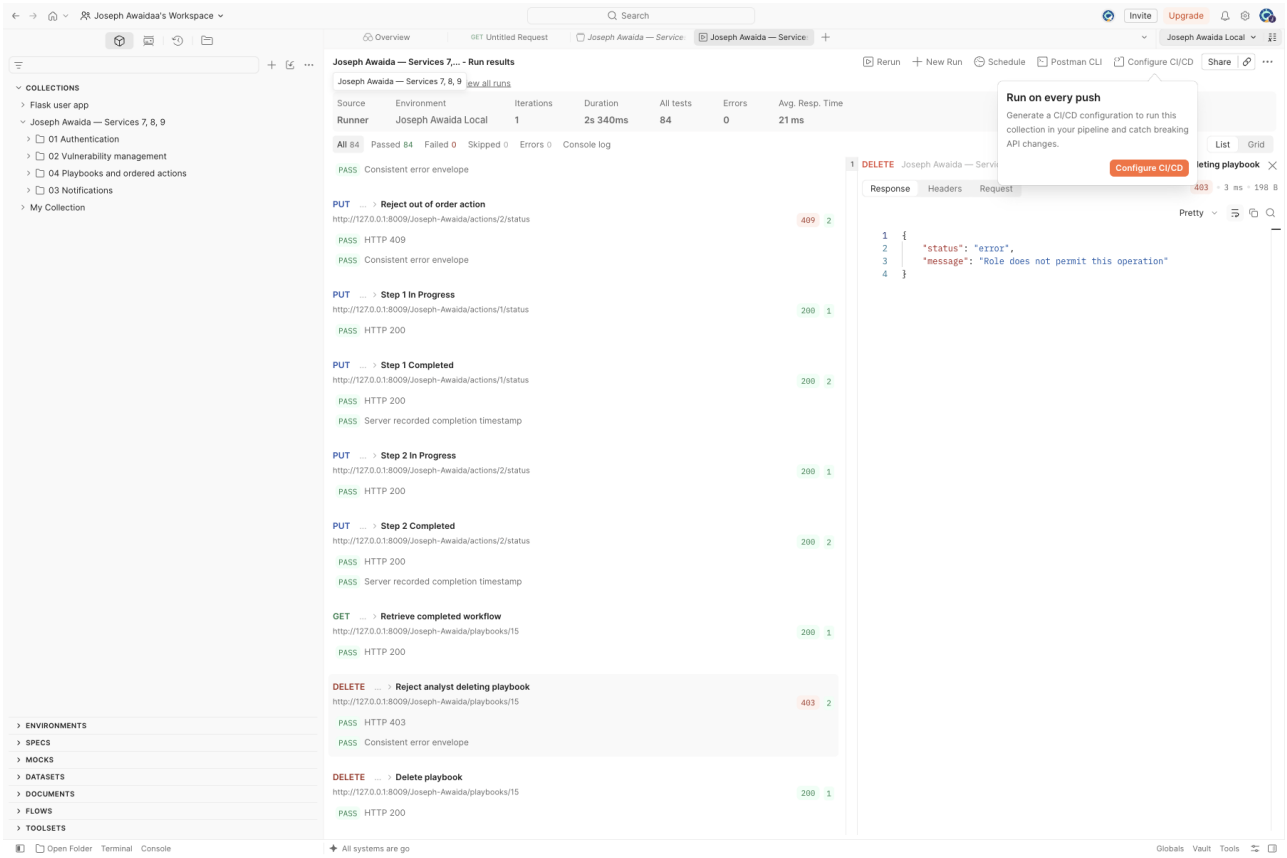

Joseph Awaida — Step 2 Completed: actual PUT result and HTTP status.



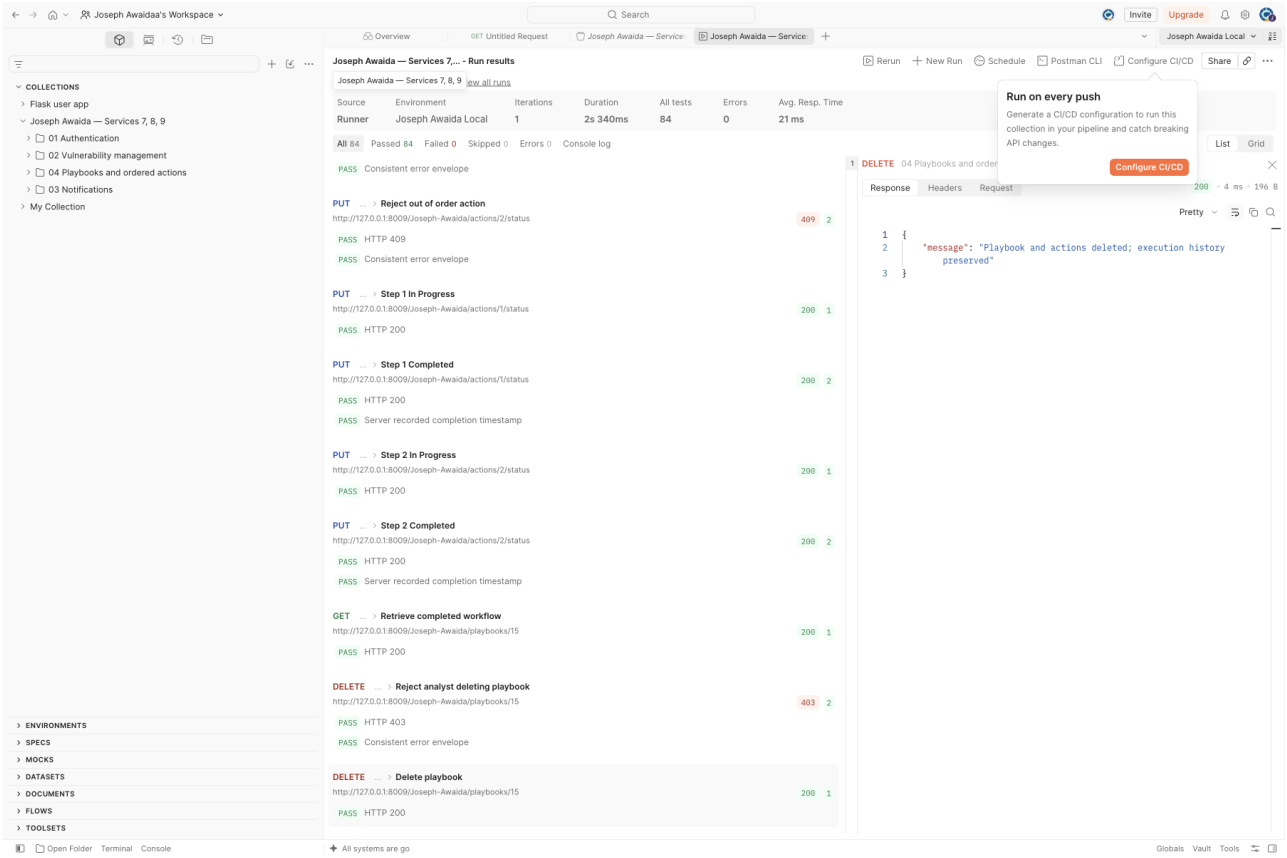
Joseph Awaida — Step 2 Completed: actual request body shown in the Postman runner.



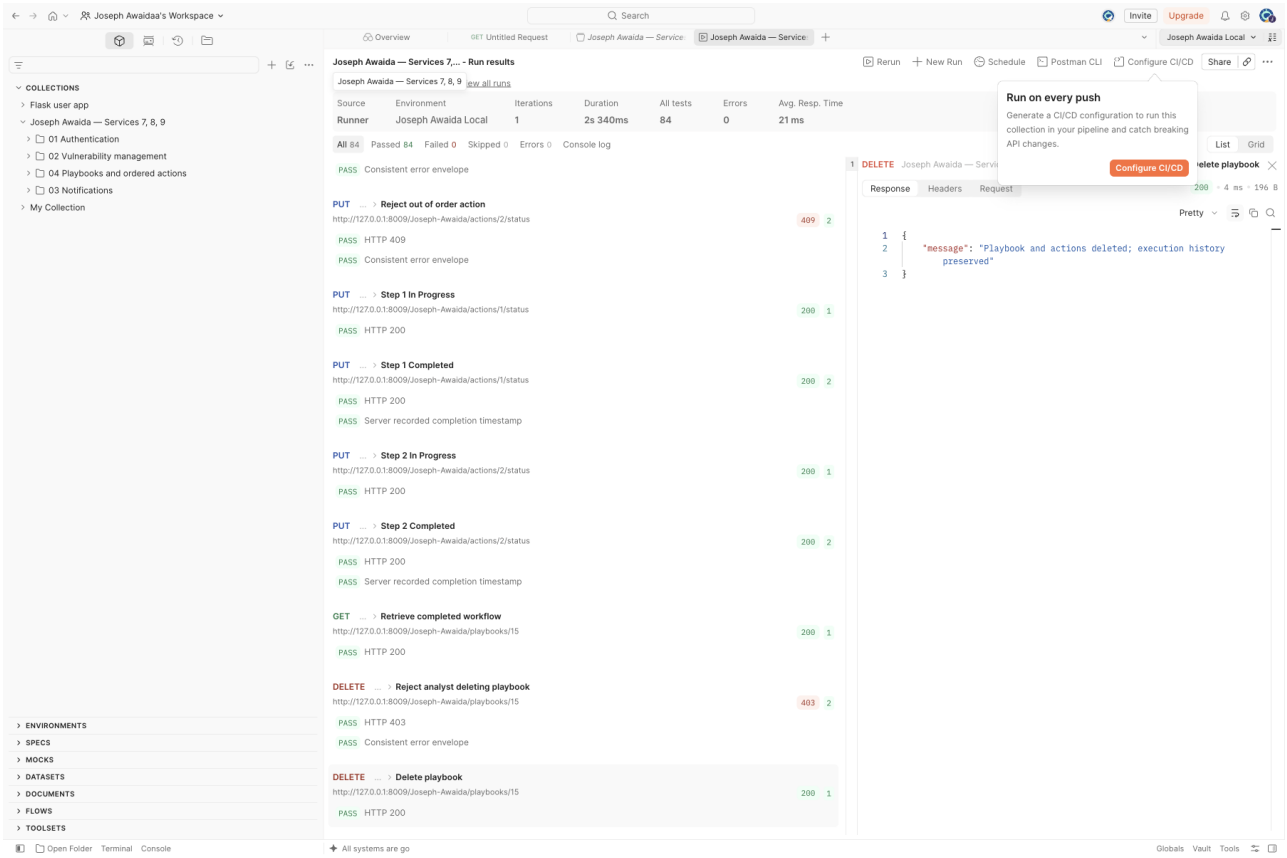
Joseph Awaida — Retrieve completed workflow: actual GET result and HTTP status.



Joseph Awaida — Reject analyst deleting playbook: actual DELETE result and HTTP status.



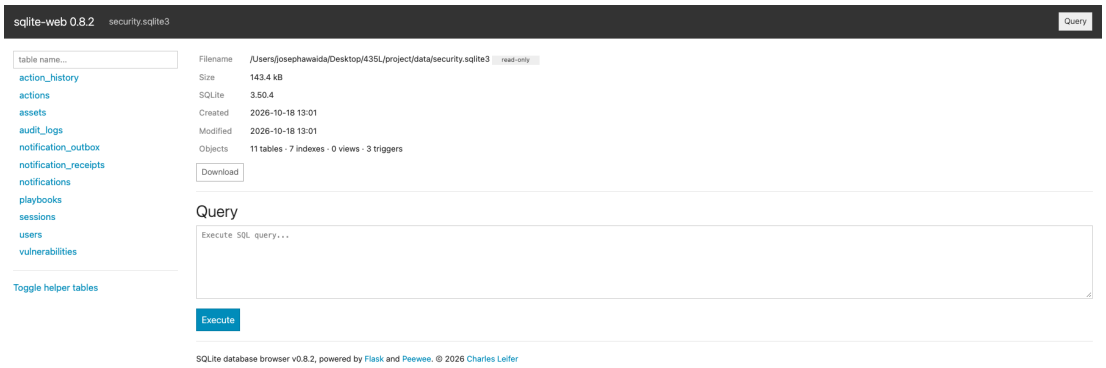
Joseph Awaida — Delete playbook: actual DELETE result and HTTP status.



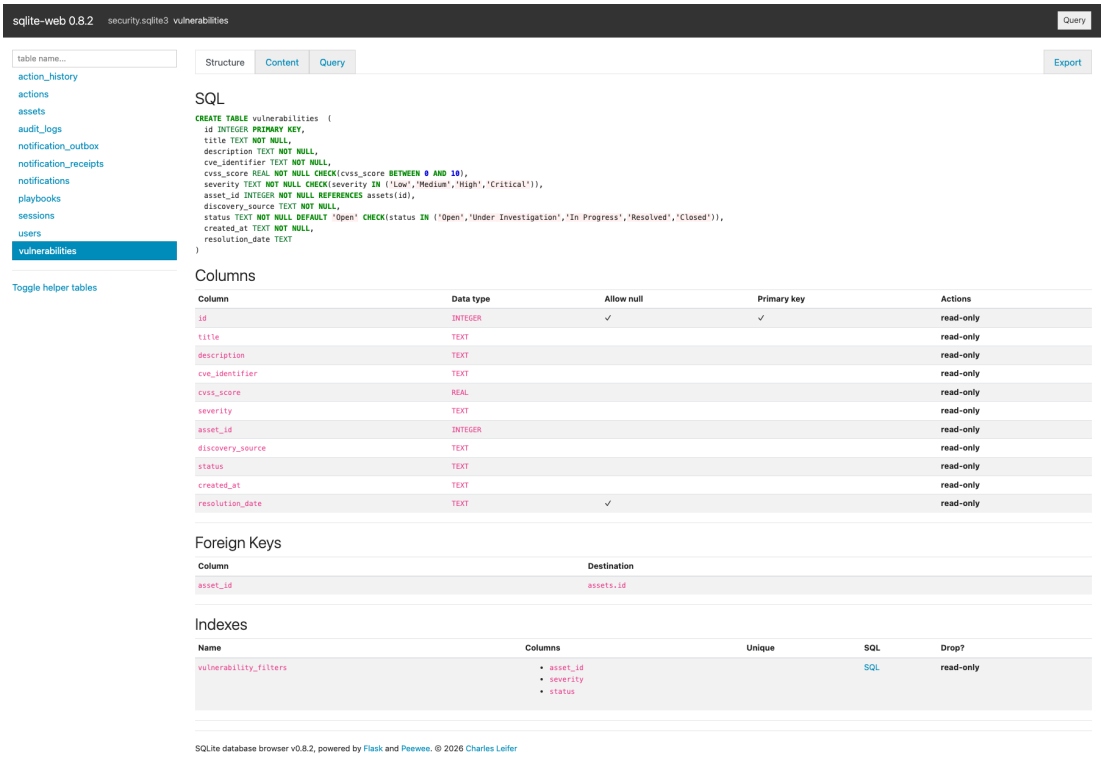
Joseph Awaida — Postman desktop run: 84 assertions passed, zero errors.

Genuine database and GitHub screenshots (Joseph Awaida)

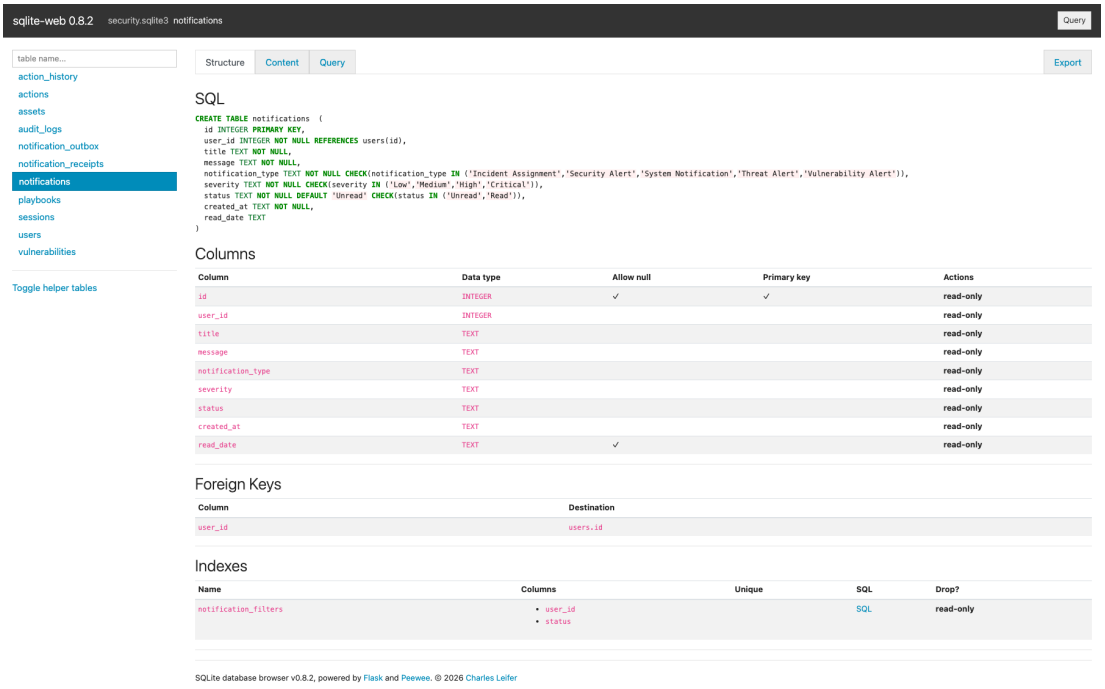
Unedited captures from actual running applications. Captions identify the student, operation and purpose. Token-bearing authentication responses use the Headers view; private password bodies are omitted.



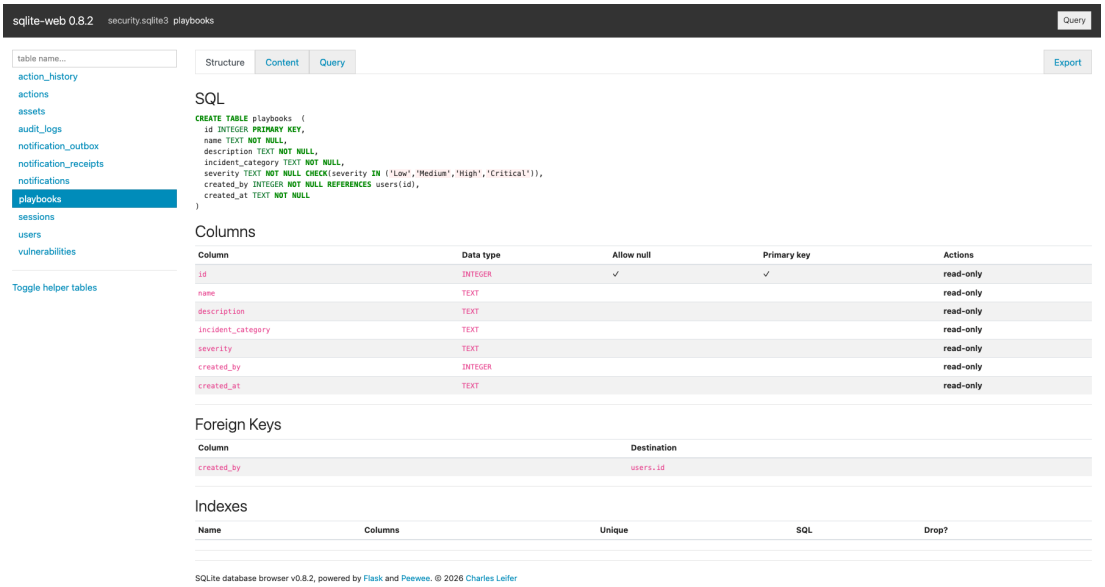
Joseph Awaida — Database tables: the implemented local relational schema.



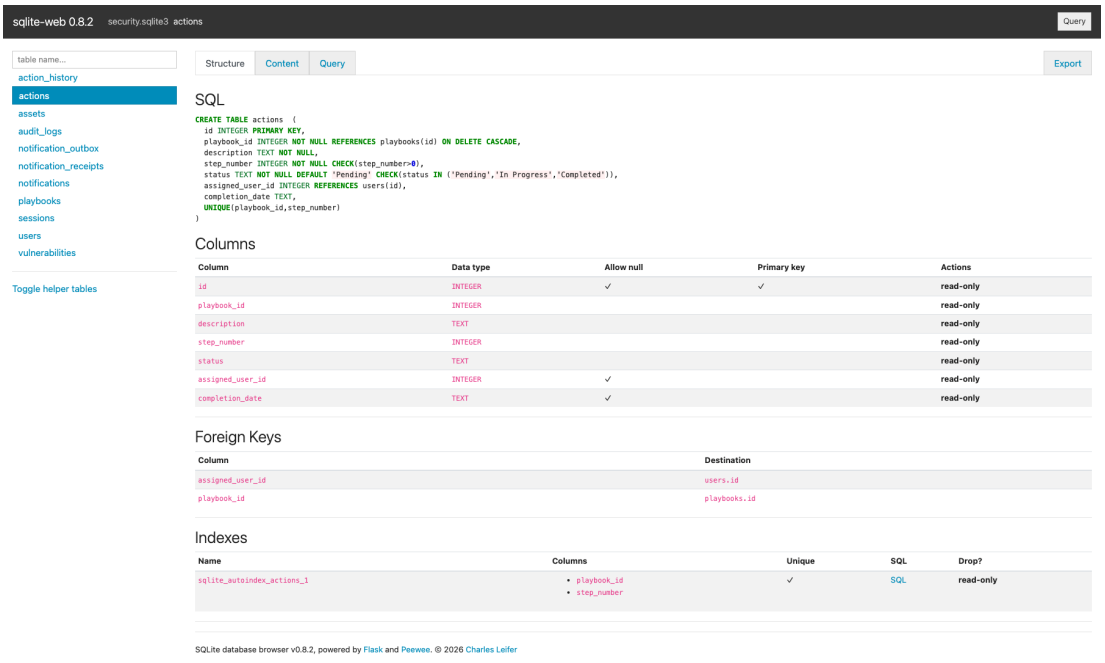
Joseph Awaida — vulnerabilities structure: actual fields, keys and constraints.



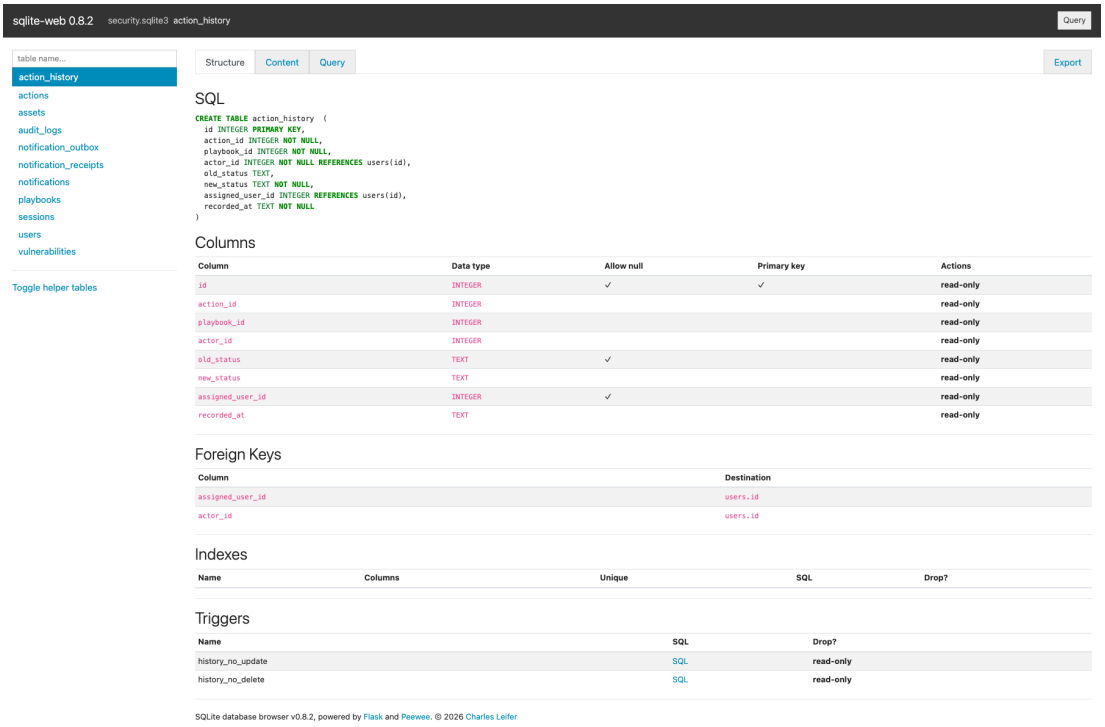
Joseph Awaida — notifications structure: actual fields, keys and constraints.



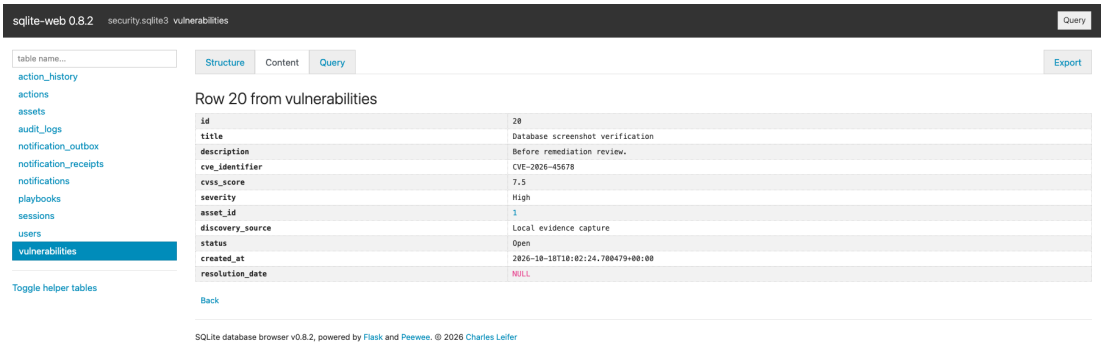
Joseph Awaida — playbooks structure: actual fields, keys and constraints.



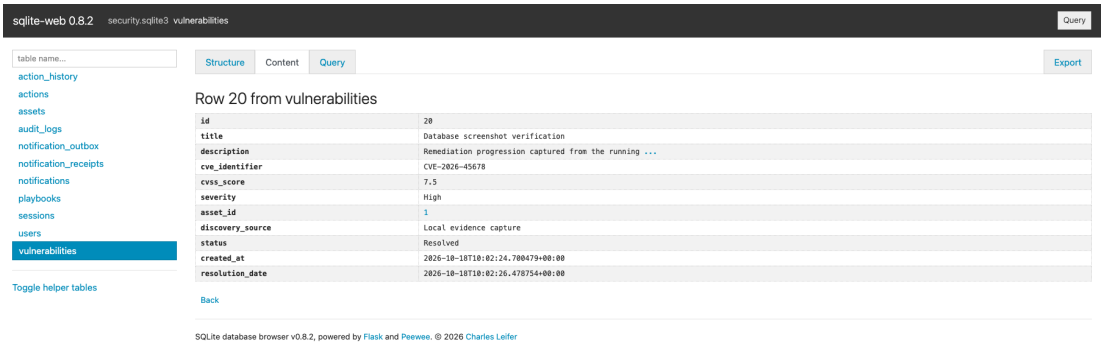
Joseph Awaida — actions structure: actual fields, keys and constraints.



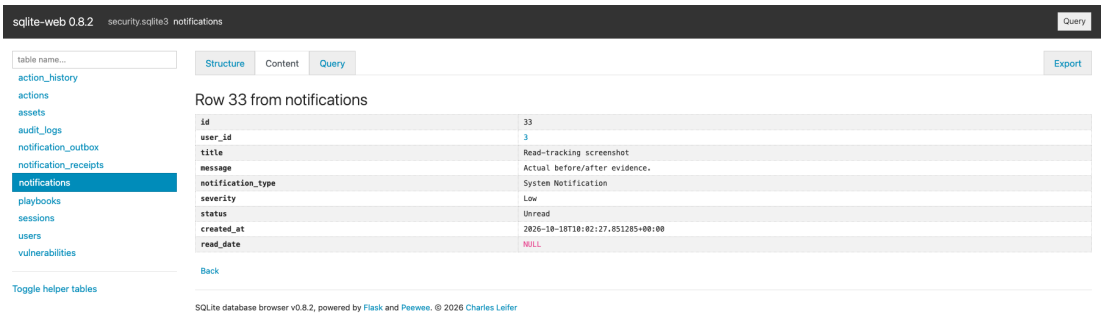
Joseph Awaida — action_history structure: actual fields, keys and constraints.



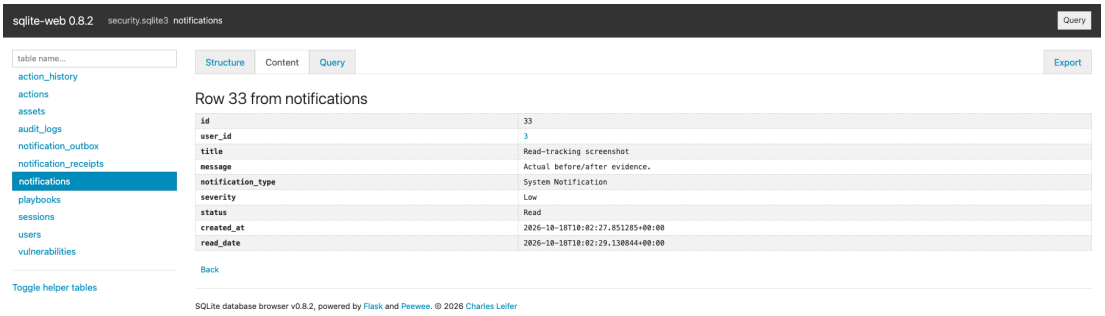
Joseph Awaida — Vulnerability before update: Open with no resolution date.



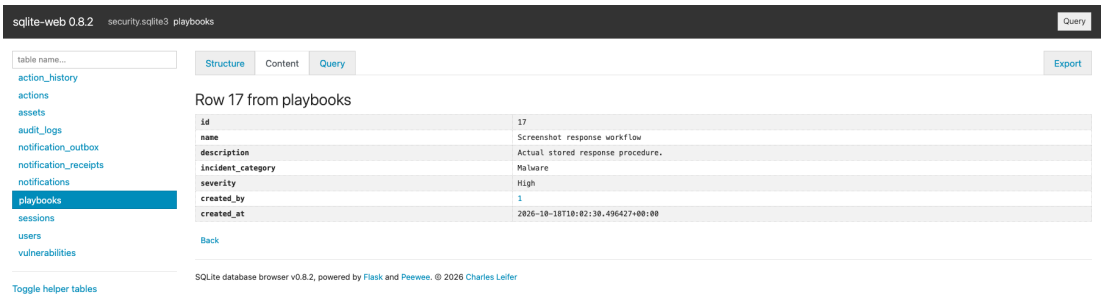
Joseph Awaida — Vulnerability after valid lifecycle progression: Resolved with server-recorded resolution date.



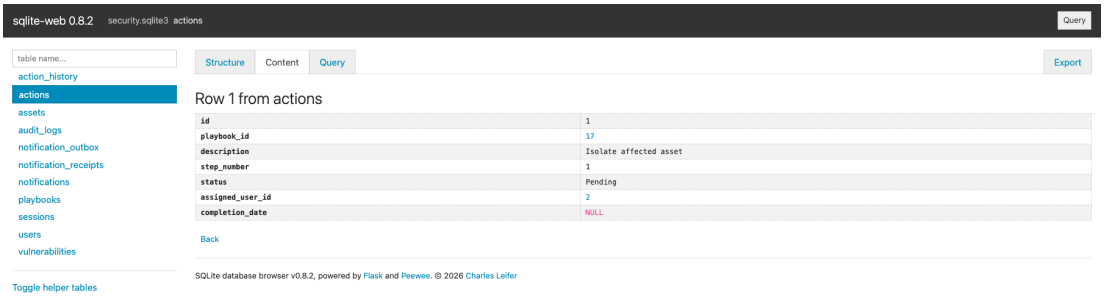
Joseph Awaida — Notification before read: Unread with no read date.



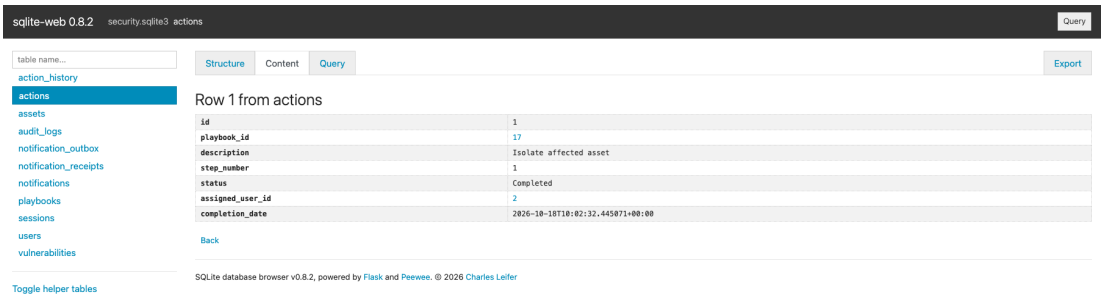
Joseph Awaida — Notification after mark-read: Read with its original read timestamp.



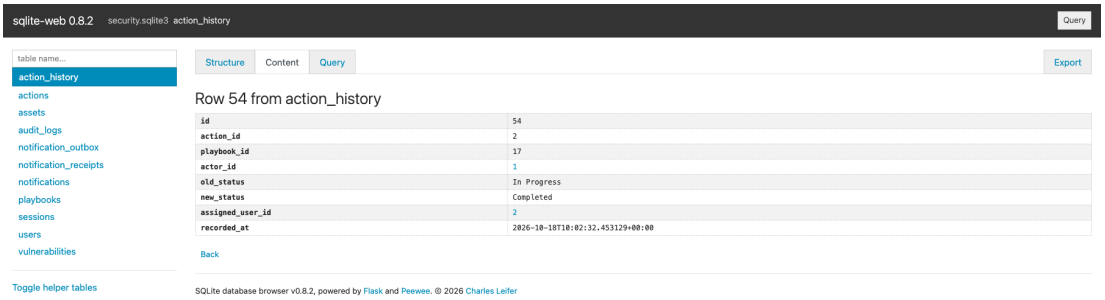
Joseph Awaida — Stored playbook: category, severity, creator and creation timestamp.



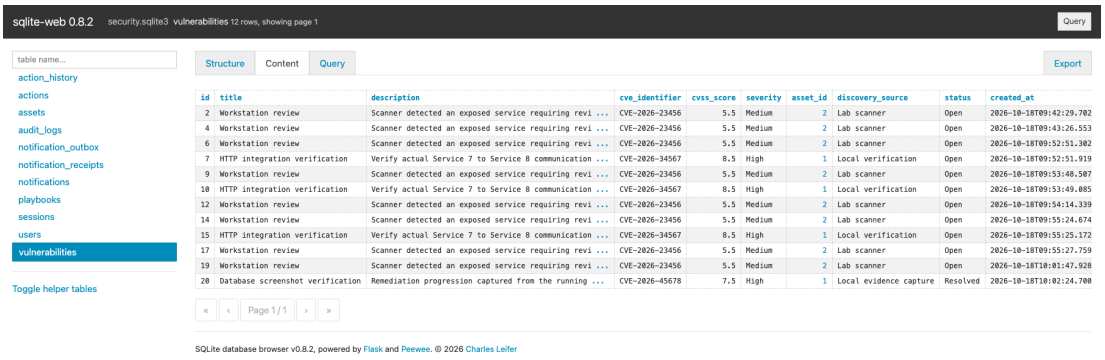
Joseph Awaida — Response action before execution: Pending and assigned to analyst 2.



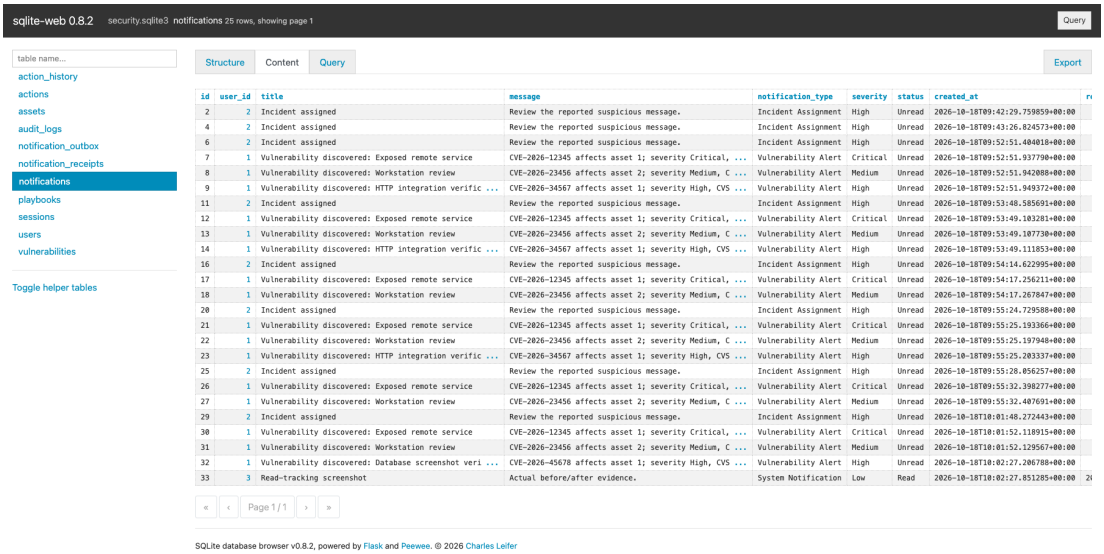
Joseph Awaida — Response action after ordered execution: Completed with server completion time.



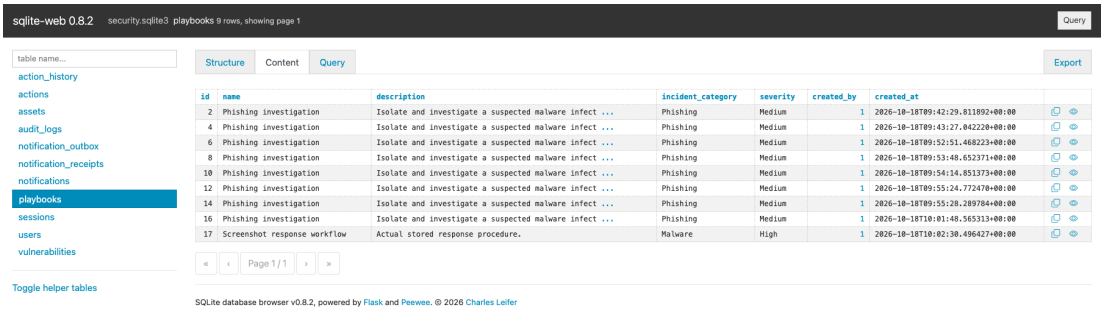
Joseph Awaida — Execution history: an actual immutable response-action transition and its actor.



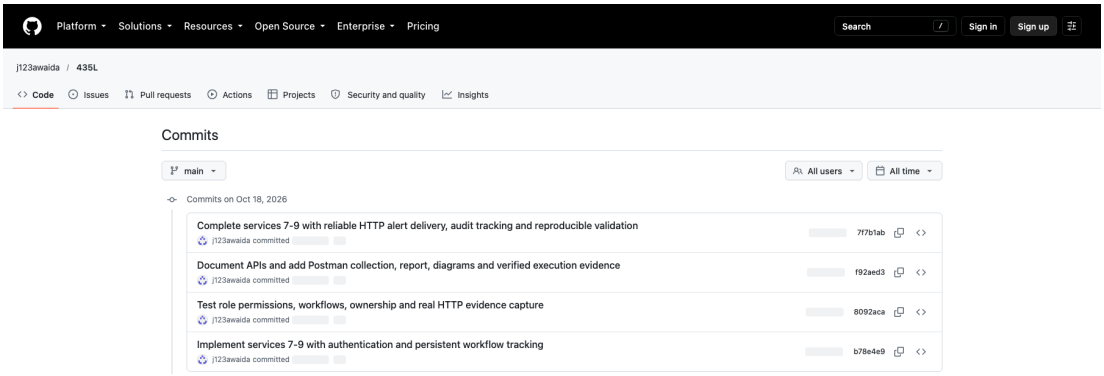
Joseph Awaida — vulnerabilities records: actual stored rows from the implementation.



Joseph Awaida — notifications records: actual stored rows from the implementation.



Joseph Awaida — playbooks records: actual stored rows from the implementation.



Joseph Awaida — GitHub commit history: genuine contributions in the supplied repository.

Submission status and remaining evidence (Joseph Awaida)

- All eighteen required vulnerability, notification and playbook/action operations.

- Authentication, per-role permissions, notification ownership, input validation, consistent errors, database foreign keys and workflow rules.

- Automatic vulnerability alerts delivered from Service 7 to Service 8 over HTTP, durable retries and duplicate protection.

- Successful-change and permission-denial audit records; immutable response-action history retrievable with playbooks and preserved after deletion.

- Database schema, seed command, structure records and actual before/after execution records.

- Described Postman collection with automated assertions and local environment template; actual Newman execution results.

- Automated tests, API reference, OpenAPI schemas, architecture/ER diagrams, editable DOCX report and paginated PDF report.

- AI-assistance disclosure, integration handoff, demonstration guide and contribution ZIP packaging script.

- Genuine Postman desktop request/result screenshots, actual database structure and before/after screenshots, and a GitHub commit-history screenshot, embedded in the reports with identifying captions.

- Work uploaded to the supplied GitHub repository; actual commit history is available there.

- **Student review and explanation:** read the demonstration guide and understand the implementation before presenting it.

- **Team integration and common report:** teammates must supply their actual Services 1–6 code, names and agreed identity/asset/audit contracts. Merge this contribution into one common report and include genuine teammate history. Their implementations and contributions are not included or fabricated here.

- **Final project name:** use StudentFullName1-StudentFullName2-StudentFullName3-430L for the complete team submission after the three names are known. The current archive is explicitly Joseph's contribution.

- **CREATES:** personally attend the required team session at least three days before the actual deadline and apply its feedback. Attendance cannot be completed by Codex.

- **Moodle:** submit the complete team source, schema/database scripts, common report, Postman collection and GitHub link through the course account. No course portal session or final team submission was supplied.

The README contains setup commands. docs/TEAM-INTEGRATION.md defines the notification integration contract. docs/DEMONSTRATION.md explains how to show the implemented behavior. evidence/validation-summary.json contains the actual final validation counts.